ESCUELA POLITÉCNICA NACIONAL

FACULTAD DE SISTEMAS

CORE DE SERVICIOS WEB PARA INTEROPERAR ENTRE EL ECOSISTEMA DE LA FACULTAD DE INGENIERÍA EN SISTEMAS Y LA PLATAFORMA DE SISTEMAS ACADÉMICOS DE LA ESCUELA POLITÉCNICA NACIONAL

COMPONENTE DE DESARROLLO DEL CORE DE SERVICIOS WEB

TRABAJO DE INTEGRACIÓN CURRICULAR PRESENTADO COMO REQUISITO PARA LA OBTENCIÓN DEL TÍTULO DE INGENIERO DE SOFTWARE

VICTOR SEBASTIÁN MORENO CORAL

victor.moreno01@epn.edu.ec

DIRECTOR: PATRICIO MICHAEL PACCHA ANGAMARCA

patricio.paccha@epn.edu.ec

DMQ, enero 2025

CERTIFICACIONES

Yo, VICTOR SEBASTIÁN MORENO CORAL declaro que el trabajo de integración curricular aquí descrito es de mi autoría; que no ha sido previamente presentado para ningún grado o calificación profesional; y, que he consultado las referencias bibliográficas que se incluyen en este documento.

VICTOR SEBASTIAN MORENO CORAL

Certifico que el presente trabajo de integración curricular fue desarrollado por VICTOR SEBASTIÁN MORENO CORAL, bajo mi supervisión.

PATRICIO MICHAEL PACCHA ANGAMARCA

DECLARACIÓN DE AUTORÍA

A través de la presente declaración, afirmamos que el trabajo de integración curricular aquí descrito, así como el (los) producto(s) resultante(s) del mismo, son públicos y estarán a disposición de la comunidad a través del repositorio institucional de la Escuela Politécnica Nacional; sin embargo, la titularidad de los derechos patrimoniales nos corresponde a los autores que hemos contribuido en el desarrollo del presente trabajo; observando para el efecto las disposiciones establecidas por el órgano competente en propiedad intelectual, la normativa interna y demás normas.

VICTOR SEBASTIÁN MORENO CORAL

PATRICIO MICHAEL PACCHA ANGAMARCA

DEDICATORIA

Dedico este trabajo con todo mi amor y gratitud a mis padres quienes con su esfuerzo, apoyo incondicional y enseñanzas han sido mi mayor fuente de inspiración. Mi abuela cuyo amor y sabiduría han sido un pilar fundamental en mi vida. Mis tías por su cariño constante y por estar siempre presentes con palabras de aliento. Cada logro alcanzado en este proyecto es también de ustedes por todo lo que han hecho por mí.

AGRADECIMIENTO

Agradezco profundamente a todas las personas que hicieron posible la realización de este proyecto. A mis padres por su amor incondicional, sacrificios y constante apoyo en cada etapa de mi vida. A mi queridísima abuela por ser una guía inquebrantable en mi camino. Extiendo mi gratitud a mis profesores y amigos quienes con su orientación y ayuda contribuyeron a este logro. Los sueños no se conquistan en soledad sino con el eco del amor y el apoyo de quienes creen en nosotros

ÍNDICE DE CONTENIDO

CERTIFICACIONES I
DECLARACIÓN DE AUTORÍA II
DEDICATORIA III
AGRADECIMIENTO IV
ÍNDICE DE CONTENIDO V
ÍNDICE DE FIGURAS VII
ÍNDICE DE TABLAS VIII
RESUMEN IX
ABSTRACT X
1 DESCRIPCIÓN DEL COMPONENTE DESARROLLADO 1
1.1 Objetivo general 2
1.2 Objetivos específicos 2
1.3 Alcance 2
1.4 Marco teórico 3
Definición de Servicios Web 3
Modelado de Datos 4
Arquitectura de Servicios Web RESTful con .NET 5
Definición de Recursos y Endpoints 5
Controladores 7
Modelos 7
Acceso a Datos con Entity Framework Core 7
Repositorios 7
Estrategia Domain-Driven Design 8
Contenedorización con Docker 8
Orquestación para Escalabilidad y Alta Disponibilidad 9
Tendencias y Futuro del Desarrollo de los Servicios Web 10
2 METODOLOGÍA 10
2.1 Enfoque Metodológico 10
Plan Metodológico 10
Gestión de proyectos de desarrollo de software en la DGIP 12
2.2 Fases de la Metodología 12
Fase de Análisis y Requisitos 12
Fase de Diseño 13

Fase de Desarrollo 13
Fase de Pruebas 14
2.3 Descripción funcional del monolito 14
Funcionamiento del sistema actual 15
Limitaciones del sistema actual 17
2.4 Requisitos en el proceso de desarrollo 18
Estándares de desarrollo 19
Estándares para crear el Core de servicios 22
2.5 Diseño para el desarrollo del Core de Servicios 25
Estrategia de migración a microservicios 26
Arquitectura Propuesta para el Core de servicios 32
Selección de protocolos, seguridad y estándares RESTFul 33
Diseño de una API Gateway centralizada 36
2.6 Desarrollo e Implementación del Core de servicios Web 37
Herramientas utilizadas para desarrollo 38
Ecosistema de desarrollo 39
Desarrollo de microservicios 42
Estrategias de integración de datos en microservicios 45
Integración de Servicios Específicos y API gateway 50
Configuración de Docker y orquestación 54
2.7 Pruebas de desarrollo 55
Herramientas utilizadas para pruebas 56
Metodología de pruebas de desarrollo 56
Validación de Contenedores y orquestación 57
Pruebas de seguridad 58
Pruebas Unitarias 62
3 RESULTADOS, CONCLUSIONES Y RECOMENDACIONES 64
3.1 Resultados 64
3.2 Conclusiones 77
3.3 Recomendaciones 79
4 REFERENCIAS BIBLIOGRÁFICAS 80
5 ANEXOS 82
5.1 ANEXO I. Código Fuente del Proyecto de Agendamiento de Citas Médicas en .NET 82
5.2 ANEXO II. Diagrama de Clases del Proyecto de Agendamiento de Citas Médicas 88
5.3 ANEXO III. Estándares de Desarrollo del Proyecto 89

5.4 ANEXO IV. Metodología de Desarrollo Utilizada.....89
5.5 ANEXO V. Documentación caso de estudio 89
5.6 ANEXO VI. Colecciones de Pruebas de API en Postman.....89
5.7 ANEXO VII. Reporte Generado con Swagger.....89
5.8 ANEXO VIII. Resultados de Pruebas de API utilizando Newman 89

ÍNDICE DE FIGURAS

Figura 1. Orquestación de Contenedores con Docker..... 9
Figura 2. Estructura de Carpetas del Monolito 16
Figura 3. Diagrama Entidad-Relación del Monolito 17
Figura 4. Diagrama de Contexto Sistema de Agendamiento de Citas.....27
Figura 5. Diagrama de Clases del Microservicio de Gestión de Citas 29
Figura 6. Diagrama de Clases del Microservicio de Personal Médico 29
Figura 7. Diagrama de Clases del Microservicio de Pacientes.....30
Figura 8. Diagrama de Clases del Microservicio de Catálogos 30
Figura 9. Diagrama de Clases Enfocado a DDD 31
Figura 10. Diagrama de Microservicios para Agendamiento de Citas Médicas 32
Figura 11. Métodos HTTP en el Microservicio de Citas.....34
Figura 12. Flujo de Seguridad con JWT 35
Figura 13. Diagrama API Gateway.....37
Figura 14. Configuración de Docker con Visual Studio 40
Figura 15. Integración de Docker al monolito.....42
Figura 16. Estructura de Carpetas de Microservicios DDD 42
Figura 17. Estructura de Carpetas Centrada en Dominios 43
Figura 18. Capas de Microservicios.....44
Figura 19. Referencias de la Capa API.....45
Figura 20. Referencias de la Capa Aplicación 45
Figura 21. Referencias de la Capa Infraestructura.....45
Figura 22. Código de Unificación de Tablas.....46
Figura 23. Estructura de División de DataContext.....47
Figura 24. Código de consultas SQL directas 47
Figura 25. Código para Consulta de un API.....48
Figura 26. Código para publicar evento 48
Figura 27. Código para consumir evento 49
Figura 28. Estructura de Archivos de Microservicios.....50
Figura 29. Estructura de Archivos de API Gateway 51
Figura 30. Código de Configuración de Middleware.....53
Figura 31. Archivo docker-compose.yml 54
Figura 32. Contenedor del Monolito 57
Figura 33. Orquestación de Microservicios 57
Figura 34. Código de Ingreso de Encabezados 59
Figura 35. Contenedor de Generación de Firmas 59
Figura 36. Autorización con JWT 60
Figura 37. Prueba de Autenticación.....61

Figura 38. Mensaje de Error de Autenticación Fallida 61
Figura 39. Documentación de Endpoint en Swagger 62
Figura 40. Respuesta de endpoint gate 63
Figura 41. Colección de Microservicios en Postman 63
Figura 42. Prueba Unitaria en Postman 64
Figura 43. Solicitudes de Microservicios en Newman 65
Figura 44. Validación del endpoint GET /api/Catalogo 65
Figura 45. Resultados del endpoint GET /api/Catalogo.....65
Figura 46. Validación del endpoint GET /api/Catalogo/1 66
Figura 47.Resultados del endpoint GET /api/Catalogo/1.....66
Figura 48. Validación del endpoint GET /api/CatalogoHoras.....67
Figura 49.Resultados del endpoint GET /api/CatalogoHoras 67
Figura 50. Validación del endpoint GET /api/Cita.....68
Figura 51.Resultados del endpoint GET /api/Cita 68
Figura 52. Validación del endpoint GET /api/CitaEstadoActual 69
Figura 53.Resultados del endpoint GET /api/CitaEstadoActual.....69
Figura 54. Validación del endpoint GET /api/Horario.....70
Figura 55.Resultados del endpoint GET /api/Horario 70
Figura 56. Validación del endpoint GET /api/Paciente 72
Figura 57.Resultados del endpoint GET /api/Paciente 72
Figura 58. Validación del endpoint GET /api/ListaPaciente 73
Figura 59.Resultados del endpoint GET /api/ListaPaciente 73
Figura 60. Validación del endpoint GET /api/Personal 74
Figura 61.Resultados del endpoint GET /api/Personal.....74
Figura 62. Validación del endpoint GET /api/Especialidad 75
Figura 63.Resultados del endpoint GET /api/Especialidad.....75
Figura 64. Validación del endpoint GET /api/Permisos/GetPermisosByPersonal/3906 76
Figura 65.Resultados del endpoint GET /api/Permisos/GetPermisosByPersonal/3906....77

ÍNDICE DE TABLAS

Tabla 1.Métodos HTTP y su uso común 6
Tabla 2. Estándares de Codificación.....20
Tabla 3. Estándares de Base de Datos.....21
Tabla 4. Estándares de Microservicios 24
Tabla 5. Configuración del API Gateway.....52
Tabla 6. Configuración de Docker Compose.....55

RESUMEN

Este proyecto aborda el desarrollo de un Core de Servicios Web que permite la interoperabilidad entre los diversos aplicativos académicos utilizados en el Subdecanato de la FIS utilizando las mejores prácticas y principios para la creación de servicios modulares. Este trabajo se centra en el caso de estudio del sistema la Dirección de Gestión de Información y Procesos (DGIP) de la Facultad de Ingeniería de Sistemas (FIS) con el objetivo de crear estándares de desarrollo que puedan seguirse en la creación de aplicaciones y el demostrar cómo una arquitectura orientada a servicios puede mejorar la alineación con los procesos del negocio.

En la fase de análisis y requisitos se recopiló información detallada sobre el sistema documentando su funcionamiento y creando estándares para la transición hacia microservicios. En la fase de diseño se creó la propuesta de arquitectura para el Core de Servicios, la selección de protocolos y estándares RESTful, y el diseño de un API Gateway que asegura la centralización y seguridad del acceso a los servicios. En la fase de desarrollo se implementó un Core de Servicios Web utilizando el caso de estudio con .NET como prueba de concepto de la nueva arquitectura. En la fase de pruebas se realizó un análisis de los endpoints y se ejecutaron pruebas unitarias y de seguridad para validar el correcto funcionamiento de los servicios.

PALABRAS CLAVE: Core de Servicios Web, DGIP, FIS, integración académica, Microservicios, Domain-Driven Design, arquitectura RESTful, .NET.

ABSTRACT

This project focuses on the development of a Web Services Core that enables interoperability among the various academic applications used in the Subdean's Office of the Faculty of Engineering Systems (FIS), employing the best practices and principles for creating modular services. This work focuses on the case study of the system managed by the Directorate of Information and Process Management (DGIP) of the Faculty of Engineering Systems (FIS), with the aim of creating development standards that can be followed in the creation of applications and demonstrating how a service-oriented architecture can enhance alignment with business processes.

During the analysis and requirements phase, detailed information about the system was gathered, documenting its operation and creating standards for the transition to microservices. In the design phase, the proposal for the architecture of the Services Core was created, along with the selection of RESTful protocols and standards, and the design of an API Gateway to ensure centralized and secure access to services. In the development phase, a Web Services Core was implemented using the case study with .NET as proof of concept for the new architecture. In the testing phase, an analysis of the endpoints was performed, and unit and security tests were executed to validate the correct functionality of the services.

KEYWORDS: Web Services Core, DGIP, FIS, academic integration, Microservices, Domain-Driven Design, RESTful architecture, .NET.

1 DESCRIPCIÓN DEL COMPONENTE DESARROLLADO

La Facultad de Ingeniería en Sistemas (FIS) de la Escuela Politécnica Nacional (EPN) enfrenta un desafío importante relacionado con la falta de interoperabilidad entre los diversos aplicativos académicos utilizados en el Subdecanato de la FIS. Uno de los ejemplos más evidentes es el Sistema de Planificación Académica (SPA-WEB), que actualmente implementa una Interfaz de Programación de Aplicaciones (API) para integrarse con el Free Timetabling Software (FET). Sin embargo, esta integración ha resultado insuficiente para resolver las redundancias de datos y las limitaciones en el acceso eficiente a información académica.

Aunque en los últimos años se han desarrollado diversos componentes informáticos para optimizar algunos servicios específicos del Subdecanato, estos esfuerzos no han alcanzado un nivel de integración que permita un manejo centralizado y colaborativo de la información. Para abordar este problema, es fundamental lograr una interoperabilidad robusta con la Dirección de Gestión de Información y Procesos (DGIP), institución clave en la administración de datos institucionales. Esta interoperabilidad se logrará mediante la creación de un ecosistema tecnológico basado en servicios RESTful, aprovechando una arquitectura escalable que garantice la adaptabilidad y la sostenibilidad del sistema en el tiempo.

Este proyecto requiere una atención particular a la seguridad de la información, considerando que se procesará información académica sensible. Los estándares de JSON Web Token (JWT), técnica de autenticación criptográfica como HMAC, y aplicar principios de diseño de API seguros. Además, la implementación será guiada por una metodología hibrida siguiendo el ciclo de vida del software, fomentando entregas incrementales basada en retroalimentación constante de los stakeholders, incluidos los usuarios finales, el Subdecanato y la DGIP.

Desde una perspectiva técnica, el desarrollo del Core de Servicios Web se basará en las mejores prácticas de la ingeniería de software, incluyendo la modularización de componentes, el uso de tecnologías modernas de desarrollo Backend (como .NET Core o Node.js), y el despliegue en entornos escalables mediante contenedores y orquestadores como Kubernetes. Este enfoque no solo asegurará un rendimiento óptimo, sino también una fácil mantenibilidad y expansión futura.

El impacto del proyecto será significativo: al eliminar duplicidades de datos, mejorar el acceso a la información y optimizar la gestión de procesos académicos y administrativos, se espera un fortalecimiento de la toma de decisiones estratégicas en la facultad. La

integración tecnológica propuesta posicionará a la EPN como líder en innovación tecnológica dentro del ámbito académico nacional, estableciendo un precedente para otras instituciones educativas en Ecuador y la región.

1.1 Objetivo general

Desarrollar un Core de servicios web que permita la interoperabilidad entre sistemas informáticos del Subdecanato en la Facultad de Ingeniería de Sistemas de la Escuela Politécnica Nacional y las aplicaciones académicas existentes facilitando así un entorno colaborativo y accesible.

1.2 Objetivos específicos

1. Desarrollar un Core de Servicios Web utilizando una Arquitectura de Servicios Web RESTful con .NET.
2. Asegurar modularidad y escalabilidad en la integración de las aplicaciones académicas del Subdecanato de la Facultad de Ingeniería de Sistemas.
3. Diseñar e integrar medidas de seguridad para proteger los datos y servicios del sistema incluyendo autenticación y autorización robustas.
4. Implementar la transmisión segura de datos mediante el uso de protocolos como HTTPS.

1.3 Alcance

Este proyecto tiene como objetivo principal desarrollar un Core de Servicios Web que permita la interoperabilidad eficiente entre las aplicaciones académicas utilizadas en el Subdecanato de la Facultad de Ingeniería de Sistemas (FIS). Para ello se propone una arquitectura basada en microservicios diseñada para mejorar la flexibilidad del ecosistema tecnológico de la Dirección de Gestión de Información y Procesos (DGIP).

El alcance del proyecto incluye la migración de un sistema monolítico existente hacia una arquitectura moderna de servicios tomando como caso de estudio el sistema de agendamiento de citas médicas de la DGIP. Este sistema fue seleccionado por su complejidad y relevancia dentro de los procesos académicos y administrativos permitiendo demostrar la viabilidad y los beneficios de un diseño modular que se ajuste a las necesidades institucionales.

La creación de un conjunto de estándares y mejores prácticas para el desarrollo de software dentro de la DGIP permite que otras áreas de la institución puedan replicar este modelo en proyectos futuros fomentando un ecosistema tecnológico sostenible.

El desarrollo del Core de Servicios Web comprende varias fases: análisis, diseño, desarrollo y pruebas. En la fase de análisis se recopiló información sobre el sistema monolítico identificando sus limitaciones y definiendo los estándares de desarrollo que guiarán la transición a microservicios. En la fase de diseño se crearon los planos arquitectónicos alineados con los protocolos RESTful, y se diseñó un API Gateway para centralizar y garantizar la seguridad del acceso a los servicios.

En la fase de desarrollo se implementa microservicios independientes que reflejan las reglas del negocio y que están diseñados para ser reutilizables en futuros proyectos. Se hizo uso de técnicas y patrones de diseño como Domain-Driven Design (DDD) para que los servicios estén alineados con los procesos del negocio. La fase de pruebas incluyó la validación funcional y de seguridad de los microservicios mediante herramientas como Postman y Newman para comprobar que cada componente cumpla con los estándares definidos.

1.4 Marco teórico

Definición de Servicios Web

Los Servicios Web (Web Services) son aplicaciones informáticas autónomas y autocontenidas, son accesibles a través de una red que permite la comunicación e interacción de datos entre sistemas o aplicaciones. Se clasifican en dos categorías diferentes de acuerdo con el diseño del API, REST (Representational State Transfer) y SOAP (Simple Object Access Protocol)[1]. La Transferencia de Estado Representacional REST es un estilo arquitectónico para la creación de APIs en sistemas distribuidos basado en el uso de solicitudes en protocolo HTTP para interactuar con recursos representados en formatos JSON o XML [2], caracterizado por su simplicidad, escalabilidad y capacidad para respuestas cacheables. El protocolo simple de acceso a objetos SOAP es un protocolo estándar basado en XML para intercambiar información estructurada en entornos distribuidos [3], conocido robustez especialmente en transacciones seguras y complejas que requieren una estructura definida.

El desarrollar Servicios Web se trata de aplicaciones que interactúen mediante una interoperabilidad entre tecnologías y plataformas diversas, intercambiando datos y funcionalidades de manera segura que involucra aspectos como crear y gestionar el diseño

web, el desarrollo del lado del cliente (Front-end), el desarrollo del lado del servidor (Back-end) y la gestión de servidores, así como de las bases de datos.

El desarrollo del lado del cliente Front-end se refiere a definir los procesos de interacción con el usuario ya que es la capa de software que los clientes visualizan. El Front-end se encarga de interactuar con el servicio web a través de APIs. Las tecnologías y frameworks principales como React, Angular y Vue se utilizan para crear interfaces de usuario interactivas y responsivas. Los clientes pueden consumir vía herramientas de cliente los servicios web a través de llamadas HTTP y manejar los datos recibidos en formatos como JSON o XML.

Conocido como desarrollo del lado del servidor el desarrollo Back-end es la capa de software que gestiona la lógica y funcionalidad del lado del servidor además de la interacción con la base de datos. Utiliza frameworks y tecnologías que permiten la creación de APIs robustas y escalables. Frameworks modernos como ASP.NET Core, Node.js, y Django entre otros son los más usados en esta área [4]. Para el desarrollo del Back-end se manejan las operaciones de CRUD (Crear, Leer, Actualizar, Eliminar) en la base de datos y se utiliza tecnologías de acceso a datos como Entity Framework Core en .NET para gestionar la persistencia eficaz de datos.

Modelado de Datos

La gestión de la estructura de la base de datos es crucial por requerir el modelado y definición de relaciones entre entidades, como por ejemplo E-R entidad-relación que representa las entidades y relaciones entre ellas que se establece en una base de datos relacional. Se requiere la implementación de mecanismos de persistencia y recuperación de datos para almacenar y recuperar datos de manera eficiente [5], utilizando tecnologías como Entity Framework Core para .NET, Hibernate para Java, o Sequelize para Node.js. entre otros.

En Web Services cada microservicio opera dentro de un Bounded Context (contexto limitado) que significa que gestiona su propio dominio de datos de manera independiente. Esto permite que los servicios sean autónomos reduciendo el acoplamiento entre ellos. También cada microservicio puede tener su propia base de datos optimizada para sus necesidades específicas lo que mejora el rendimiento y facilita la escalabilidad. En estos entornos la consistencia eventual se utiliza con frecuencia para sincronizar los datos entre servicios mediante eventos y sistemas de mensajería.[6] [7]. Además, la computación y arquitectura serverless (sin servidor) permite a los desarrolladores ejecutar código sin preocuparse por la infraestructura subyacente y ser más eficientes. Herramientas

modernas como Node.js con Express, Spring Boot, Django, Flask, Ruby on Rails, Apollo Server, GraphQL-Ruby y Graphene son de las más utilizadas [6] [7].

Arquitectura de Servicios Web RESTful con .NET

La implementación de la arquitectura de servicios web RESTful en .NET permite la creación de aplicaciones web escalables, modulares y fáciles de mantener. En la arquitectura RESTful, cada recurso tiene una URI única y las solicitudes HTTP se utilizan para realizar operaciones sobre los recursos del servidor [8]. Por ejemplo, una solicitud GET a una URI específica puede devolver la representación del recurso, mientras que una solicitud POST puede crear un nuevo recurso, PUT puede actualizar un recurso existente y DELETE puede eliminar un recurso [1]. REST utiliza enlaces a través de hipermedios en APIs, donde los enlaces dentro de las respuestas pueden guiar a los clientes sobre las acciones disponibles y los estados de los recursos. El framework .NET proporciona un entorno de programación robusto y de alto rendimiento para la implementación de servicios web RESTful, especialmente con ASP.NET Core, una plataforma modular para construir aplicaciones modernas. También simplifica la creación y gestión de servicios RESTful, ofreciendo herramientas y funcionalidades que facilitan el desarrollo, la prueba y el despliegue de APIs RESTful [9].

Definición de Recursos y Endpoints

Cuando se trabaja con una arquitectura RESTful normalmente se habla de un recurso, el cual es una abstracción que representa una entidad o un conjunto de datos que puede ser manipulada a través de una interfaz de programación de aplicaciones (API). Los recursos son elementos fundamentales en la arquitectura REST (Representational State Transfer), ya que encapsulan la información y las operaciones disponibles en un servicio web [10].

Los endpoints se implementan en controladores utilizando atributos de routing de ASP.NET Core. En ASP.NET Core los controladores se encargan de implementar estos endpoints. Cada método dentro de un controlador se asocia con una operación HTTP y maneja las solicitudes entrantes procesando los datos según sea necesario y devolviendo las respuestas adecuadas al cliente.[12].

A cada recurso se puede acceder y gestionar utilizando métodos HTTP, Ver Tabla 1, como GET, POST, PUT y DELETE, que corresponden a las operaciones de lectura, creación, actualización y eliminación, respectivamente [11]. Los Endpoints RESTful son las rutas a través de las cuales se accede a los recursos. Cada endpoint está asociado con un método HTTP específico y representa una operación que se puede realizar sobre el recurso en

cuestión. Cada endpoint tiene una URI (Identificador Uniforme de Recursos) única que permite a los clientes acceder a recursos específicos [1].

Tabla 1.Métodos HTTP y su uso común

Métodos HTTP	Características	Uso	Propósito
El método GET se utiliza para solicitar datos de un recurso específico. No debe modificar el estado del recurso en el servidor y es considerado un método seguro e idempotente.	Idempotente realizar la misma solicitud GET varias veces produce el mismo resultado sin efectos secundarios adicionales.	Obtener datos o recursos. Por ejemplo, obtener información sobre un usuario en una aplicación. Ejemplo: GET /api/users/123 solicita los detalles del usuario con ID 123.	Permite a los clientes consultar datos del servidor sin alterar su estado. Es ideal para recuperar información.
El método POST se utiliza para enviar datos al servidor para crear un nuevo recurso o realizar una acción que puede modificar el estado del recurso. POST no es idempotente.	No idempotente realizar la misma solicitud POST varias veces puede crear varios recursos o tener efectos secundarios diferentes.	Crear nuevos recursos o enviar datos que deben procesarse en el servidor. Por ejemplo, enviar un formulario de registro. Ejemplo: POST /api/users con un cuerpo que contiene los datos del nuevo usuario	Permite a los clientes enviar datos al servidor para crear nuevos recursos o realizar acciones que alteren el estado del servidor.
El método PUT se utiliza para actualizar un recurso existente en el servidor o crear uno nuevo si no existe. Es idempotente, lo que significa que aplicar la misma solicitud PUT varias veces produce el mismo resultado.	Idempotente realizar la misma solicitud PUT varias veces tiene el mismo efecto que una sola vez.	Actualizar un recurso existente con datos completos o crear un nuevo recurso en una URI específica si no existe. Ejemplo: PUT /api/users/123 con un cuerpo que contiene los datos actualizados del usuario con ID 123.	Permite actualizar un recurso específico con nuevos datos, o crear un recurso en una URI determinada si aún no existe.
El método DELETE se utiliza para eliminar un recurso específico en el servidor. No debe devolver una representación del recurso eliminado y es idempotente.	Idempotente realizar la misma solicitud DELETE varias veces no cambia el resultado después de la primera eliminación.	Eliminar un recurso existente. Por ejemplo, eliminar un usuario de la base de datos. Ejemplo: DELETE/api/users/123 elimina el usuario con ID 123.	Permite a los clientes solicitar la eliminación de un recurso en el servidor. La eliminación del recurso es final y no reversible a través de esta solicitud.

Controladores

Los controladores son la parte de la aplicación que maneja las solicitudes HTTP entrantes, procesa los datos y devuelve las respuestas al cliente. En una arquitectura RESTful son responsables de definir los endpoints de la API y manejar las acciones CRUD (Create, Read, Update, Delete) sobre los recursos. [13]

Modelos

Los modelos en EF Core representan las entidades de dominio de la aplicación. Un modelo es una clase que corresponde a una tabla en la base de datos. Cada instancia de la clase representa una fila en la tabla. Las clases de modelo se definen en el código y se configuran con propiedades que se corresponden con las columnas de las tablas de la base de datos. Los modelos de transferencia de datos (DTOs) son patrones de arquitectura que se utilizan para definir la estructura de los datos que se intercambian entre el cliente y el servidor. Son modelos diseñados específicamente para transferir datos a través de la API, simplificando y optimizando las respuestas y solicitudes.[13]

Acceso a Datos con Entity Framework Core

Entity Framework Core (EF Core) es un marco de trabajo de acceso a datos para .NET que permite a los desarrolladores trabajar con bases de datos utilizando objetos .NET y un enfoque de programación orientada a objetos. EF Core es una versión ligera, extensible y de código abierto de Entity Framework, el cual es un Object-Relational Mapper (ORM) que facilita la interacción con bases de datos relacionales.

DbContext es la clase principal en EF Core que se utiliza para gestionar las operaciones de persistencia en la base de datos. Es una clase que proporciona una conexión a la base de datos y permite realizar operaciones CRUD (Crear, Leer, Actualizar, Eliminar). Se define una clase que hereda de DbContext y se configura con DbSet<TEntity> propiedades para cada entidad que se quiere gestionar. Por ejemplo, un DbContext para una aplicación de comercio electrónico podría tener propiedades como DbSet<Producto>, DbSet<Pedido>, etc. La configuración de la cadena de conexión a la base de datos se realiza en el archivo de configuración (appsettings.json en ASP.NET Core) y se inyecta en el DbContext a través del contenedor de servicios de ASP.NET Core.[14]

Repositorios

Los repositorios son responsables de la interacción con la base de datos. Actúan como una capa intermedia entre la aplicación y los datos persistentes, proporcionando métodos para

realizar operaciones CRUD de manera abstracta y eficiente Los repositorios utilizan Entity Framework Core para interactuar con la base de datos.

La implementación del repositorio usa DbContext para interactuar con la base de datos. Por ejemplo, un método Add en el repositorio puede usar DbContext.Add para agregar una nueva entidad a la base de datos. Usar repositorios ayuda a mantener el código de acceso a datos separado de la lógica de negocio, facilita las pruebas unitarias al permitir el uso de repositorios simulados (mocks), y proporciona una ubicación central para la lógica de acceso a datos.[15]

Para una implementación de una aplicación basada en Entity Framework, se utiliza los patrones de diseño Modelo-Controlador-Repositorio para gestionar la persistencia y comportamiento de los datos.

Estrategia Domain-Driven Design

Domain-Driven Design (DDD) es una técnica ampliamente valorada por su enfoque en organizar los servicios alrededor de las necesidades del negocio en lugar de preocupaciones técnicas. DDD introduce principios, prácticas y patrones como guías para la descomposición del dominio de aplicación desde una perspectiva contextual del negocio. Facilita la creación de sistemas que reflejan de manera fiel los procesos y reglas de la organización a través de capas.[16]

Contenedorización con Docker

El desarrollo y la implementación de servicios web RESTful con .NET se benefician enormemente de la contenedorización y la orquestación con Docker y Kubernetes [9]. Estas tecnologías permiten gestionar aplicaciones de manera más eficiente y consistente a través de entornos variados [10]

Docker es una plataforma de contenedorización que permite a los desarrolladores empaquetar aplicaciones junto con todas sus dependencias en contenedores ligeros y portátiles.

La plataforma Docker utiliza archivos de configuración, como el DockerFile y el docker-compose.yml, para definir el entorno de ejecución de una aplicación. Esto asegura que la aplicación se ejecute de manera consistente en cualquier entorno, ya sea desarrollo, pruebas o producción.

Al encapsular la aplicación y sus dependencias dentro de un contenedor, se eliminan problemas relacionados con la configuración y las versiones de las bibliotecas. Esto reduce la probabilidad de errores causados por diferencias entre entornos.

Los contenedores Docker pueden ser iniciados y detenidos con rapidez, lo que simplifica el proceso de despliegue. Esto permite una implementación más ágil y eficiente, facilitando la integración y el despliegue continuos (CI/CD).

Orquestación para Escalabilidad y Alta Disponibilidad

La gestión y orquestación de contenedores automatiza y simplifica la implementación, el escalado y la gestión de aplicaciones contenerizadas. Para este apartado se usó la herramienta Docker Compose que permitió gestionar diferentes servicios creados y aumentar la eficiencia en el desarrollo de contenedores.[17]

Docker Compose permite controlar de una manera fácil diferentes contenedores y orquestarlos para cumplir funciones específicas que se hacer a través de un archivo único de configuración de tipo YAML, lo que permite crear y escalar aplicaciones fácilmente. Esta herramienta se lo utiliza a menudo para configurar un entorno local, pero también puede formar parte de un flujo de trabajo de continuous integration / continuous delivery (CI/CD**)**.

Una arquitectura básica con Docker con orquestación luce igual que el diagrama mostrado en la Figura 1. Orquestación de Contenedores con Docker [17].

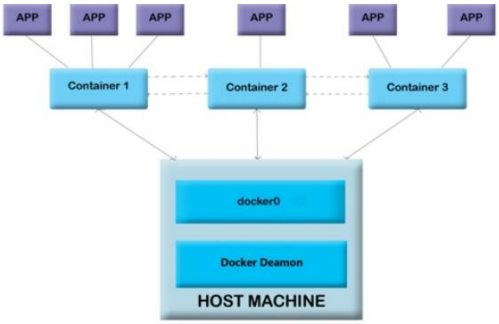


Figura 1. Orquestación de Contenedores con Docker

En este diagrama se observa un grupo lógico de contenedores que contienen servicios de aplicación que proveen toda la lógica de un negocio estos son gestionados y coordinados por el Docker Daemon el cual es responsable de crear, iniciar, detener y eliminar contenedores según las necesidades y también interactúa con la herramienta Docker que proporciona la interfaz de usuario para ejecutar comandos y gestionar los contenedores.[17]

Tendencias y Futuro del Desarrollo de los Servicios Web

El desarrollo de servicios web ha evolucionado significativamente en las últimas décadas, impulsado por avances tecnológicos y cambios en las necesidades del mercado.

Una de las tendencias más importantes en el desarrollo de servicios web es la arquitectura de microservicios. A diferencia de las arquitecturas monolíticas donde todas las funcionalidades se agrupan en una única aplicación, los microservicios descomponen una aplicación en servicios pequeños e independientes que se comunican a través de APIs.

La computación serverless permite a los desarrolladores construir y ejecutar aplicaciones sin considerar la infraestructura subyacente. Los proveedores de servicios en la nube gestionan la infraestructura y el escalado automáticamente, permitiendo a los desarrolladores centrarse en la lógica del negocio.[18]

El uso de GraphQL permite a los clientes solicitar exactamente los datos que necesitan, proporcionando un único endpoint que responde con datos específicos solicitados en una sola consulta. A diferencia de REST, que utiliza múltiples endpoints.[4]

La integración de inteligencia artificial (IA) y machine learning (ML) en servicios web está permitiendo el desarrollo de aplicaciones más personalizadas. Las APIs de IA y ML permiten incorporar capacidades avanzadas, como el procesamiento de lenguaje natural, la visión por computadora y el análisis predictivo.[19]

2 METODOLOGÍA

2.1 Enfoque Metodológico

El diseño y desarrollo del Core de Servicio Web de la Facultad de Ingeniería de Sistemas (FIS) se basa en un enfoque metodológico mixto. Este enfoque integra métodos cualitativos para el análisis de requisitos y diseño conceptual, y métodos cuantitativos para evaluar el desempeño y eficacia del sistema.

También dado que el enfoque es híbrido se combinó elementos de metodologías ágiles y tradicionales para garantizar que las distintas etapas del proyecto de desarrollo de software se ejecuten de manera organizada, eficiente y adaptable.

Plan Metodológico

Se tuvieron prácticas iterativas y flexibles propias de metodologías ágiles, así como la incorporación de retroalimentación constante y adaptaciones rápidas, con la estructura formal y documentada que ofrecen las metodologías tradicionales lo que permite

desarrollar con facilidad un proyecto académico como el desarrollo del arquetipo de microservicios.

Se siguieron los siguientes lineamientos para el plan metodológico híbrido:

- **Identificación de requisitos:** En este contexto se llevaron a cabo reuniones semanales con la Dirección de Gestión de la Información y Procesos (DGIP) para realizar un seguimiento detallado del desarrollo del proyecto. Estas reuniones sirvieron para identificar requisitos alineados con las necesidades de la DGIP como es definir estándares de construcción del software y ajustar los entregables a medida que avanzaban las etapas del proyecto. Por este motivo la interacción constante con la DGIP facilitó la identificación temprana de problemas y la incorporación de sugerencias clave para el éxito del arquetipo.
- **Documentación de caso de estudio:** Se selecciono como caso de estudio representativo el aplicativo de agendamiento de citas médicas que incluye operaciones críticas como la creación, modificación y consulta de citas médicas de la FIS. Este caso sirve como base para la creación de una prueba de concepto (POC) que permitió evaluar la modularización del servicio y su interoperabilidad con otras aplicaciones académicas.
- **Análisis comparativo:** Se realiza un análisis teórico sobre las ventajas de una arquitectura de servicios frente a la monolítica donde se destacó el modularidad, la independencia de componentes y la facilidad para realizar actualizaciones y mantenimiento en un futuro.

Para identificar las necesidades y prioridades del ecosistema se tuvieron reuniones con los interesados:

- **DGIP (Dirección de Gestión de la Información y Procesos):**
    - En las reuniones se identificó las necesidades actuales del sistema monolítico.
    - Se obtuvo requerimientos y estándares que deben cumplir las aplicaciones de software dentro de la DGIP.
- **Tutor académico y directivos:**
    - Validación de objetivos metodológicos y técnicos.
    - Alcance esperado para la prueba de concepto (POC).

Durante el desarrollo se analizaron múltiples documentos y referencias proporcionadas en las diferentes reuniones que se tuvieron con los interesados para que el nuevo diseño de la aplicación cumpla con los estándares establecidos por la institución y al mismo tiempo facilite la interoperabilidad en las futuras aplicaciones de la FIS. Los elementos revisados son:

- **Normativa universitaria:**
    - Políticas y modelo de desarrollo de software de la DGIP.
    - Estándares de desarrollo de software.
- **Documentación técnica de las plataformas existentes:**
    - Esquemas de base de datos del caso de estudio.
    - Modelos de integración utilizados actualmente.
    - Informe general del sistema de agendamiento de citas médicas.

Gestión de proyectos de desarrollo de software en la DGIP

La metodología presentada para la gestión de proyectos de desarrollo de software en la DGIP es un marco estructurado que garantizo la correcta ejecución del desarrollo de la aplicación hacia servicios.

La metodología de desarrollo guía al ciclo de vida del software dentro de la DGIP y también establece una relación directa con los requisitos de los componentes del ecosistema de la facultad, como plataformas tecnológicas, recursos humanos y políticas de gobernanza que están divididas en varias fases para la ejecución de proyectos (Anexo IV).

2.2 Fases de la Metodología

La metodología sugerida y acordada con la DGIP se rige a la política de gestión y desarrollo de software la cual fue utilizada en fases que cumplan con el ciclo de vida del desarrollo del software de manera secuencial y tener transición ordenada del sistema monolítico hacia una arquitectura basada en microservicios

Fase de Análisis y Requisitos

En esta fase inicial se lleva a cabo una recopilación de los requerimientos del Core de Servicios, se documentan detalladamente las funciones principales y las interacciones con otros sistemas y los puntos críticos que requerían mejoras. Con esto se permite identificar los requisitos funcionales y no funcionales necesarios para la creación de un núcleo de

servicios. También en esta fase se mantuvieron reuniones semanales con los responsables e interesados para ajustar los entregables según las necesidades específicas del proyecto como la identificación de estándares de desarrollo y la alineación con los objetivos organizacionales, esto fue clave para establecer una base sólida para las etapas posteriores.

Entregables:

- Documento de estándares de desarrollo de software adaptados a las necesidades de la DGIP.
- Identificación de los requerimientos funcionales y no funcionales del sistema.

Fase de Diseño

En esta fase se desarrollaron los lineamientos técnicos y arquitectónicos que guiarían la implementación del Core de Servicios. Se diseñó una propuesta de arquitectura basada en microservicios que permita en un futuro la interoperabilidad de sus componentes. Se definieron los estándares RESTful que regirían las interacciones entre los servicios y se seleccionaron los protocolos de comunicación más adecuados para garantizar eficiencia y seguridad. También se diseñó un API Gateway como un punto centralizado para la gestión de acceso a los servicios incluyendo mecanismos de autenticación y autorización.

Entregables:

- Diagramas de Arquitectura Propuesta para el Core de Servicios.
- Selección de protocolos, estándares RESTful, y medidas de seguridad aplicables.
- Diseño de una API Gateway como punto central de comunicación entre los microservicios.

Fase de Desarrollo

En esta etapa se llevó a cabo la implementación del Core de Servicios utilizando .NET como plataforma tecnológica principal. A partir del caso de estudio del sistema de agendamiento de citas médicas se desarrolló una prueba de concepto que demostró la viabilidad de la arquitectura propuesta. Se construyeron servicios independientes y modulares que reemplazaron las funciones del sistema monolítico siguiendo la estrategia planteada de Domain-Driven Design (DDD). En esta fase se tuvo la creación y configuración de contenedores Docker para facilitar la implementación y escalabilidad de

los servicios para cumplir con el objetivo de que los componentes desarrollados fueran independientes y fácilmente desplegables en entornos modernos.

Entregables:

- Migración del sistema monolítico al modelo de microservicios como una prueba de concepto.

Fase de Pruebas

La fase de pruebas se centró en validar la funcionalidad, seguridad y desempeño de los microservicios desarrollados. Para esto se realizaron pruebas unitarias que demuestra los requisitos cumplidos por cada servicio. De igual manera se llevó a cabo pruebas de seguridad para evaluar la robustez de los mecanismos de autenticación y autorización implementados en el API Gateway. Como resultado se obtuvo un reporte que documentó el correcto funcionamiento de los endpoints de los microservicios evidenciando la interoperabilidad entre ellos y su capacidad para integrarse con otros sistemas académicos de la FIS.

Entregables:

- Resultados documentados de las pruebas unitarias y de seguridad realizadas al caso de estudio.

2.3 Descripción funcional del monolito

El caso de estudio de este componente de desarrollo fue la aplicación de agendamiento de citas médicas la cual se escogió debido a su relevancia en el sector de la salud de la FIS y a la posibilidad de aplicar conceptos como la migración de microservicios para solucionar los retos presentes en los sistemas actuales (Anexo V).

En el caso de la aplicación de citas médicas se maneja un gran volumen de datos en donde los sistemas monolíticos frecuentemente enfrentan problemas de escalabilidad y mantenimiento. El objetivo principal al seleccionar esta aplicación como caso de estudio fue demostrar la viabilidad de la transición desde sistemas tradicionales hacia arquitecturas más ágiles y escalables para mejorar la eficiencia operativa dentro de las aplicaciones de la FIS, también tuvo el objetivo de crear estándares de desarrollo y mejores prácticas que sirvan como referencia para la creación de aplicaciones futuras.

A través de la implementación del Core de servicios en este caso de estudio se busca garantizar que el sistema de agendamiento pueda manejar de manera eficiente altos

volúmenes de solicitudes, ofrecer integraciones con otros sistemas (como registros médicos electrónicos) y permitir una personalización adaptable a las necesidades futuras de la DGIP y que puedan ser utilizados igualmente por la FIS.

Funcionamiento del sistema actual

El funcionamiento de la aplicación se centra en simplificar el proceso de agendamiento para los usuarios finales y los administradores del sistema. Los pacientes pueden consultar la disponibilidad de horarios médicos en tiempo real, seleccionar una especialidad, y confirmar su cita con un médico específico. También el personal médico y administrativo puede gestionar eficientemente los horarios y realizar ajustes en los calendarios sobre las citas.

Para este caso el back-end de la aplicación de agendamiento de citas médicas se diseñó inicialmente como un sistema monolítico donde toda la lógica de negocio y el acceso a datos operan como una única unidad. Esta estructura de carpetas puede ser funcional y sencilla de implementar, pero en un futuro puede presentar retos en términos de escalabilidad.

El sistema monolítico de agendamiento de citas médicas se compone de dos partes principales:

- **Lógica de Negocio**:
  - Maneja las reglas para agendar, modificar y cancelar citas.
  - Contiene validaciones como la disponibilidad de horarios y restricciones de tiempo.
- **Acceso a Datos**:
  - Utiliza una única base de datos centralizada para almacenar usuarios, horarios, y registros de citas.
  - La interacción con la base de datos está acoplada directamente a la lógica de negocio, lo que aumenta la dependencia entre componentes

El monolito estudiado agrupa todas sus funciones en varias carpetas como lo indica la Figura 2. Estructura de Carpetas del Monolito, cada una de las cuales tiene funcionalidades relacionadas como:

- Gestión de servicios y controladores.

- Acceso a datos y operaciones CRUD.
- Transferencia de datos estructurada.
- Manejo de solicitudes HTTP y middleware

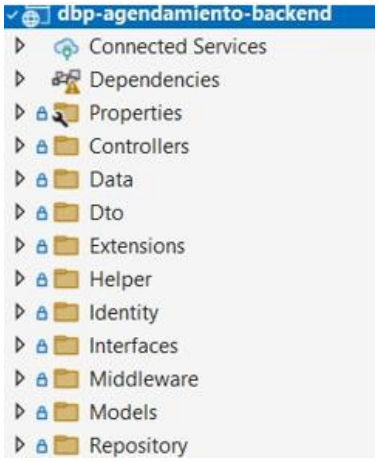


Figura 2. Estructura de Carpetas del Monolito

Este diseño implica que cualquier cambio en un módulo puede requerir la recopilación y despliegue del sistema completo aumentando los tiempos de inactividad y los riesgos asociados a fallos durante las actualizaciones.

Se realizó el análisis de la base de datos indicada en la Figura 3. Diagrama Entidad-Relación del Monolito, del sistema monolítico, con el fin de identificar las estructuras y entidades clave que se van a manejar para la migración hacia microservicios.

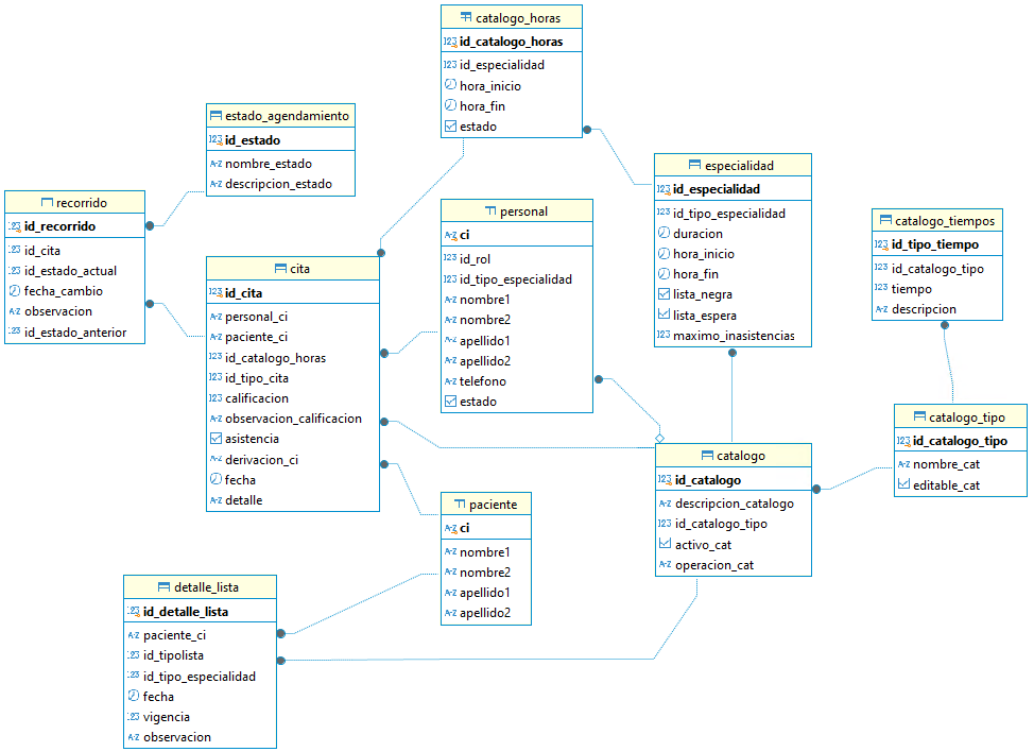


Figura 3. Diagrama Entidad-Relación del Monolito

Existen múltiples claves foráneas y relaciones cruzadas (por ejemplo, id_especialidad en varias tablas) lo cual puede dificultar la migración a microservicios si no se desacoplan correctamente.

También existe una alta cohesión de datos dado que algunas tablas como cita y personal concentran mucha información relacionada, lo que indica que sus funcionalidades están centralizadas.

Algunos nombres de atributos no siguen un patrón consistente (fecha_cambio, detalle, nombre1). Por eso al tratarse de una base relacional monolítica cualquier cambio en una tabla clave como cita o personal puede afectar múltiples dependencias.

Limitaciones del sistema actual

El sistema actual del caso de estudio se basa en una arquitectura monolítica centralizada para la gestión de citas médicas que no permite la interoperabilidad entre los distintos servicios que puede ofrecer en un futuro. Si bien al inicio permitió consolidar procesos clave las crecientes necesidades académicas en la FIS y los avances tecnológicos han expuesto limitaciones significativas que afectan el desarrollo, mantenimiento y escalabilidad del sistema.

Cualquier modificación en las reglas de disponibilidad de citas requiere la recompilación y despliegue completo del sistema. Este mismo problema ocurre en los sistemas monolíticos de la FIS cuando se actualizan procesos como la inscripción o la generación de reportes académicos lo que incrementa los tiempos de mantenimiento y el riesgo de errores en producción.

El alto acoplamiento entre los módulos en el monolito complica significativamente su mantenimiento incluso para cambios menores implica pruebas extensas en todo el sistema para garantizar que no se introduzcan errores en otros módulos. Esta problemática se replica en los sistemas de la FIS donde las dependencias entre componentes generan un efecto dominó en cada actualización. Lo cual puede resultar en:

- **Tiempo prolongado de desarrollo**: Cambios simples, como actualizar una regla de validación, pueden generar impactos en módulos no relacionados.
- **Riesgo de errores**: Alteraciones en un módulo pueden introducir errores inesperados en otras áreas del sistema debido a dependencias implícitas.

En el caso de estudio el sistema no está diseñado para integrar fácilmente nuevas tecnologías o plataformas externas. Por ejemplo, si se requiere añadir capacidades de teleconsulta o sincronización con calendarios personales se requiere modificaciones significativas en el código base.

La dependencia de tecnologías específicas como un único tipo de base de datos o frameworks restringe la capacidad de innovación y adaptación a nuevas necesidades, lo que afecta la competitividad y la eficiencia operativa.

2.4 Requisitos en el proceso de desarrollo

Este apartado establece los pasos seguidos para recopilar y analizar los requisitos funcionales y no funcionales, así como normas y estándares para crear un Core de servicios que permitirá integrarse con otros aplicativos que la DGIP considere.

Para garantizar que el desarrollo del Core de Servicios Web responda adecuadamente a las necesidades de la Facultad de Ingeniería en Sistemas (FIS) se hizo como primer paso la alineación de los estándares y políticas de desarrollo de la Dirección de Gestión de la Información y Procesos (DGIP).

Estándares de desarrollo

Dentro de los documentos desarrollados para la DGIP está el documento de estándares de desarrollo, este entregable se centra en explicar la forma codificar una aplicación para garantizar la calidad, la sostenibilidad y la interoperabilidad de los sistemas y aplicaciones que se desarrollan dentro de la Facultad de Ingeniería en Sistemas.

Dado el gran alcance y la complejidad de los servicios ofrecidos por la Facultad, se vio en la necesidad de contar con directrices claras que orienten las actividades de diseño, codificación, documentación, pruebas y mantenimiento de software asegurando en todo momento que los componentes trabajen de manera armónica para responder a las necesidades académicas de la facultad.

El documento creado sobre estándares de desarrollo de software de la Dirección General de Innovación y Proyectos (DGIP) de la Escuela Politécnica Nacional (EPN) encontrado en el Anexo III, se convierte en una referencia que proporciona un marco metodológico que no solo promueve el cumplimiento de buenas prácticas internacionales según la tecnología usada, sino que también se adaptan a las necesidades específicas de una institución educativa de alto nivel.

Normas de codificación

En el documento de estándares de desarrollo se establece un conjunto de normas específicas para garantizar la calidad del código Backend destacando el uso del framework .NET 8.0 como base tecnológica principal.

El estándar también incluye lineamientos para el uso de paréntesis en las expresiones, que deben colocarse de manera explícita para evitar ambigüedades incluso cuando no sean estrictamente requeridos por el compilador se establecen reglas claras para nombrar y declarar variables y constantes como el uso de convenciones como CamelCase para variables locales y PascalCase para constantes y nombres de clases, así como limitar la reutilización innecesaria de nombres de variables.

A continuación, se describen algunos de los puntos clave incluidos en este marco normativo en la

Tabla 2. Estándares de Codificación.

Tabla 2. Estándares de Codificación

Tema	Descripción	Ejemplo/Referencia
Idioma	El idioma oficial para nombres de clases, variables, métodos y comentarios es el **inglés**. Esto facilita la comprensión internacional y alinea con estándares globales.	Variables: int studentAge; Comentarios XML: /// <summary> calculates the age. </summary>
Sangría y Formateo de Código	Uso de formato **UTF-8** y sangrías consistentes configuradas en el IDE (como Visual Studio). Mejora la legibilidad y colaboración.	Sangría estándar: 4 espacios por nivel.
Documentación	Uso de **XML Documentation Comments** en .NET para describir propósito, entradas, salidas y comportamientos. Los comentarios deben estar en tercera persona y ser claros.	/// <summary>return the actual date. </summary>
Análisis de Código	Revisión obligatoria con herramientas como **SonarQube**. Clases no deben exceder las 2000 líneas. Realizar pruebas de carga para rendimiento.	**Herramientas:** SonarQube, PMD. **Práctica:** Dividir clases grandes en módulos.
Manejo de Excepciones	Capturar y registrar solo mensajes relevantes con _logger.LogError. Evitar el uso excesivo de excepciones genéricas.	catch (Exception e) { _logger.LogError(e.Message); }
Variables	Uso de **lowerCamelCase** para nombres descriptivos y claros. Evitar caracteres especiales o abreviaturas poco claras.	int maxStudentCount;
Constantes	Declaración como public static readonly, usando nombres,**MAYÚSCULAS_CON_GUIONES_BAJOS** para separaciones.	public static readonly string PROPERTY_URL_SERVICE = "https://example.com";
Uso de Paréntesis	Siempre utilizar paréntesis para mejorar la legibilidad en expresiones complejas, incluso si	if (a == b && c == d)

	no son requeridos por el compilador.	
Accesibilidad Web	Cumplir con WCAG 2.1. Etiquetas <img> deben tener ALT, las páginas deben incluir <title> descriptivo, asegurando accesibilidad a personas con discapacidades.	<img src="logo.png" alt="Logo de la institución"> <title>Sistema Académico</title>

Estándares de desarrollo de base de datos

Para la migración del monolito hacia el Core de servicios fue necesario establecer un estándar con respecto a la base de datos que se debería usar, la DGIP estableció como requisito para el caso de estudio que todos los servicios se desarrollen apuntando a una única base de datos. Sin embargo, se crearon directrices para aplicaciones futuras donde se incluyen las pautas sobre la normalización, la nomenclatura y las buenas prácticas para la creación de objetos de base de datos, como tablas, vistas, procedimientos almacenados, índices y triggers.

En la Tabla 3. Estándares de Base de Datos se puede ver algunos elementos que sirvieron de ayuda para establecer lineamientos para cumplir con normativas de accesibilidad y optimización dentro de las aplicaciones de la DGIP.

Tabla 3. Estándares de Base de Datos

Objeto	Reglas	Formato	Ejemplo	Prefijo
Tablas	Nombre en singular, UpperCamelCase, tabla principal primero en caso de relacional	<TableName>	CategoriaJudicatura	**N/A**
Columnas	Nombre en singular, UpperCamelCase, prefijo Id para PK, FK nombradas como en la tabla referenciada	<ColumnName>	IdProvince, IdCountry, IdLocation	**Id**
Vistas	Nombre en singular, UpperCamelCase, prefijo VW_	VW_<NombreVista>	VW_SearchLocationByProvince	**VW_**

Primary Key Constraint	Prefijo PK_ + nombre de la columna	PK_<NombreColumna>	PK_IdProvincia	**PK_**
Unique Constraint	Prefijo UC_ + campo o campos en UpperCamelCase	UC_<NombreColumnas>	UC_LoginCedula	**UC_**
Foreign Keys Constraint	Prefijo FK_ + campo o campos en UpperCamelCase	FK_<NombreColumna>	FK_IdProvincia	**FK_**
Defaults Constraint	Prefijo DC_ + tabla + campo + valor	DC_<TableName>_<ColumnName>_<Value>	DC_Judicatura_Nombre_Nulos	**DC_**
Check Constraint	Prefijo CC_ + tabla + campo + valor	CC_<TableName>_<ColumnName>_<Value>	CC_MovimientoJuicio_Cantidad_NoNegativo	**CC_**
Parámetros	Prefijo i/o/io, seguido de UpperCamelCase para el nombre	<tipodato><NombreParámetro>	iCedulaEcuatoriana	**i, o, io**
Variables	LowerCamelCase para los nombres	<NombreParámetro>	NumeroAutorizacion	**N/A**

Un principio fundamental de los estándares es la estructura de las bases de datos dado que en futuras aplicaciones deben ser separadas según el contexto de negocio para asociarlo a los servicios que se vayan creando, lo que permite minimizar la duplicidad de información y optimizar la gestión del almacenamiento.

Las bases de datos deben diseñarse teniendo en cuenta que cada tabla represente una entidad única y que las relaciones entre tablas sean consistentes. Además, se especifica el uso de tipos de datos eficientes, como bit para valores booleanos, y columnas Unicode (nvarchar, nchar) para garantizar la compatibilidad con múltiples idiomas y soportar las necesidades internacionales de la comunidad académica de la FIS. También se incluye dentro de los estándares el registro de auditorías para rastrear cambios y accesos, y la generación de diagramas de entidad-relación que faciliten la comprensión de las estructuras y relaciones entre los datos.

Estándares para crear el Core de servicios

Se creo para la DGIP una serie de estándares y buenas prácticas para el diseño, desarrollo, y despliegue de microservicios, con el objetivo de garantizar aplicaciones robustas, escalables y alineadas con las necesidades de la institución.

Estas normas se las creo para marcar un estándar al momento de crear servicios dentro de la DGIP siguiendo el patrón de diseño de Domain Driven Design (DDD), así como el uso de API RESTful como parte de la arquitectura en sus aplicaciones y también basada en la manipulación de recursos, que pueden contener datos actualizados de forma estática o dinámica.

Como estándar se recomienda usar ASP.NET Core para el desarrollo de APIs RESTful debido a su rendimiento, modularidad y compatibilidad con contenedores Docker. También se creó el uso de ciertas tecnologías al momento de crear microservicios:

- **Docker:** Para empaquetar cada microservicio en contenedores independientes, asegurando la portabilidad entre entornos de desarrollo y producción.
- **OpenAPI:** Para documentar y probar las APIs de manera automática y estandarizada, se recomienda utilizar OpenAPI. Esta especificación no solo facilita la generación de documentación clara y estructurada, sino que también permite pruebas automatizadas y la simulación de entornos.
- **Swagger**: Se recomienda usarlo conjunto de herramientas basado en OpenAPI ya que proporciona interfaces gráficas intuitivas que simplifican la interacción con las APIs y potencian el desarrollo colaborativo.
- **Entity Framework Core:** Se recomienda utilizarlo como el ORM base para gestionar la persistencia de datos, optimizando la productividad en el manejo de bases de datos relacionales.

Los estándares de nomenclatura de microservicios establecen un esquema idéntico al de codificación que incluye:

- **Nombres en PascalCase** para proyectos, clases y métodos.
- **Prefijos específicos** para los controladores (e.g., UsuariosController) y repositorios (IUsuariosRepository).
- **Organización en carpetas por capas** (/Domain, /Application, /Infrastructure, /Api), asegurando la claridad y la cohesión del proyecto.

La configuración de seguridad se realiza mediante:

- **Autenticación y Autorización:** Uso de JWT para proteger las APIs y el uso del servicio de autenticación mediante el middleware de ASP.NET Core.

- **HTTPS:** Implementación obligatoria para garantizar comunicaciones seguras con certificados proporcionados por la institución.
- **Validación de Entradas:** Control exhaustivo en las solicitudes HTTP para prevenir vulnerabilidades como inyecciones SQL o XSS.

También se desarrolló estrategias de comunicación entre microservicios como la comunicación entre el Backend utilizando:

- **Eventos de Integración:** Sincronización de datos mediante eventos en tiempo real.
- **API Gateway:** Centralización de las solicitudes hacia los microservicios, mejorando la gestión y seguridad de las conexiones.
- **CQRS:** Separación de consultas y comandos, lo que permite manejar altas demandas de lectura/escritura sin comprometer la arquitectura.

El diseño que se recomienda seguir para los microservicios se basará en la estrategia de diseño de Domain-Driven Design (DDD). Ya que permitió estructurar cada microservicio en torno a contextos delimitados que representan los dominios principales del sistema. De esta manera se obtuvo autonomía de los servicios, un alto nivel de desacoplamiento y una alineación directa entre la lógica de negocio y la implementación técnica.

Todos los aspectos y normas aplicadas que servirán para la DGIP se lo dan de una forma resumida en la Tabla 4. Estándares de Microservicios.

Tabla 4. Estándares de Microservicios

Aspecto	Descripción	Ejemplo/Referencia
Arquitectura Basada en DDD	La arquitectura de microservicios sigue el enfoque Domain-Driven Design (DDD), con capas como Dominio, Aplicación, Infraestructura y API para garantizar cohesión y bajo acoplamiento.	Ejemplo de capa de API: UsuariosController en ASP.NET Core.
Herramientas y Tecnologías	Uso de **ASP.NET Core** para APIs RESTful, **Docker** para contenedores, **OpenAPI (Swagger)** para documentar APIs y **Entity Framework Core** para persistencia de datos.	Docker Compose para levantar múltiples servicios; Swagger para probar rutas HTTP.
Nomenclatura y Organización	Nombres en **PascalCase** para clases y métodos; prefijos como IUsuariosRepository para repositorios. Organización en carpetas por capas: /Domain, /Application, /Infrastructure.	public class UsuariosController { }

Prácticas de Seguridad	Implementación de OAuth2 o JWT para autenticación. HTTPS obligatorio en todas las conexiones. Validación exhaustiva de las entradas para prevenir vulnerabilidades.	HTTPS configurado con certificados SSL; validación de entradas con reglas en ASP.NET Core.
Documentación de APIs	Uso obligatorio de OpenAPI (Swagger) para generar documentación estandarizada y pruebas automáticas. Las APIs se verifican con herramientas como Postman.	"paths": { "/usuarios": { "get": { "description": "Obtiene usuarios" } } }
Estrategias de Comunicación	Patrones recomendados: **Eventos de Integración** para sincronización, **API Gateway** para centralizar solicitudes y **CQRS** para separar consultas y comandos.	Uso de RabbitMQ para eventos de integración; implementación de API Gateway con herramientas como Ocelot o Kong.
Autenticación y Autorización	Uso de tokens JWT para proteger las APIs. Configuración de roles y permisos basados en Claims.	{ "Authorization": "Bearer token" }
Validación de Entradas	Control exhaustivo de las solicitudes HTTP mediante filtros y middlewares para prevenir inyecciones SQL y XSS.	if (input.Contains("DROP")) { throw new ValidationException(); }
Pruebas y Validación	Las APIs deben pasar pruebas automatizadas y manuales. Se verifican los endpoints principales con herramientas como Postman y se evalúan errores de carga o latencia.	Pruebas de carga con JMeter; verificación de rutas mediante suites en Postman.
Uso de Docker	Cada microservicio se empaqueta en un contenedor independiente, facilitando la portabilidad entre entornos de desarrollo, pruebas y producción.	Archivo Dockerfile con instrucciones de compilación: FROM mcr.microsoft.com/dotnet/aspnet:8.0

Con estos estándares la DGIP garantiza que los microservicios cumplan con las expectativas funcionales y que también estén alineados con los principios modernos de software dentro del desarrollo del Core de servicios.

2.5 Diseño para el desarrollo del Core de Servicios

Para comenzar el diseño del Core de Servicios Web para la Facultad de Ingeniería en Sistemas (FIS) se tuvo que responder a la necesidad de modernizar y optimizar la gestión agendamiento de citas médicas, alineándose con los estándares y políticas representadas en el capítulo anterior por la DGIP.

Este diseño se basa en una arquitectura de microservicios adaptada a partir del caso de estudio del sistema de agendamiento de citas médicas. También otro punto fue descomponer este sistema monolítico en servicios independientes, especializados en funciones específicas, que trabajan de manera conjunta para ofrecer una solución robusta y adaptable. Cada microservicio sigue principios de Domain-Driven Design (DDD) lo que asegura una alta cohesión dentro de cada módulo y un bajo acoplamiento entre ellos.

Se contemplaron elementos clave como la implementación de una API Gateway que centraliza las comunicaciones entre los clientes y los microservicios, facilitando la seguridad, el enrutamiento dinámico y el balanceo de carga. También se adoptaron estándares modernos como RESTful para la comunicación entre servicios y la integración de herramientas de orquestación como Docker Compose para garantizar la portabilidad y escalabilidad del sistema.

Estrategia de migración a microservicios

El primer paso en la estrategia de migración es la identificación de los contextos limitados según las reglas de negocio, un concepto central en el patrón de Domain-Driven Design (DDD) que permite segmentar el sistema en dominios delimitados y bien definidos. Cada contexto representa una unidad lógica que encapsula una parte específica de la lógica del negocio y se convierte en la base para definir los microservicios.

El diagrama de clases del caso de estudio ubicado en el Anexo II destaca las entidades principales y sus relaciones. Las clases como cita, paciente, personal y catalogo_horas son el núcleo del sistema, mientras que otras clases como estado_agendamiento y recorrido gestionan los estados y cambios asociados al flujo de las citas. Tambien el sistema incorpora elementos como los catalogo, catalogo_tiempos y especialidad, que definen reglas de negocio para los horarios y restricciones para las citas.

La separación por dominios específicos permite trabajar de forma independiente en cada contexto, reducir la complejidad global y facilitar la implementación de nuevas características.

En el caso de estudio de la DGIP, los contextos limitados identificados en la Figura 4. Diagrama de Contexto Sistema de Agendamiento de Citas incluyen:

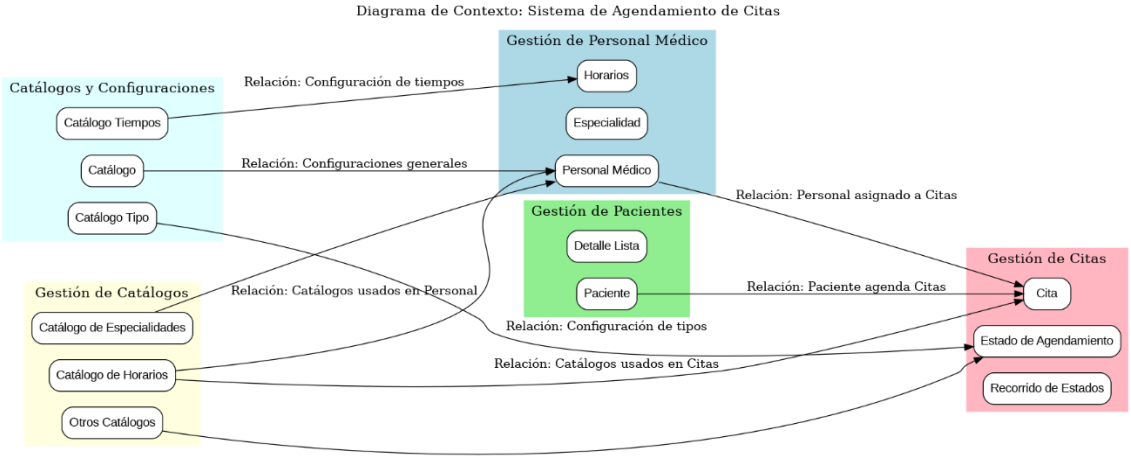


Figura 4. Diagrama de Contexto Sistema de Agendamiento de Citas

- Gestión de Pacientes

Aquí se administra la información del personal médico (personal), sus roles y especialidades. Este contexto incluye reglas relacionadas con los horarios y asignaciones del personal en la atención de pacientes también las operaciones relacionadas con los pacientes, como el registro de sus datos y la creación de listas detalladas para consultas futuras. Incluye las entidades "Paciente" y "Detalle Lista". Al crear este contexto se enfocó en gestionar la información personal de los pacientes de manera independiente, cumpliendo con normativas de privacidad y permitiendo flexibilidad para adaptarse a nuevos requerimientos del dominio, como integraciones con otros sistemas de salud.

- Gestión de Personal Médico

Este contexto maneja toda la información relacionada con los pacientes, como sus datos personales (paciente) y sus listas de historial o preferencias (detalle_lista).

Incluye entidades como "Personal Médico", "Especialidad" y "Horarios". Su principal función es garantizar que el sistema cuente con información precisa sobre los médicos, sus especialidades y las disponibilidades de horario. Se separo esta lógica del negocio debido que el manejo del personal médico tiene reglas de negocio específicas, como la asignación de horarios según especialidad, que no están directamente relacionadas con otros aspectos como los pacientes o las citas.

- Gestión de Citas

Este es el corazón del sistema e incluye la entidad cita, que conecta pacientes, personal médico y disponibilidad de horarios (catalogo_horas). Incorpora funcionalidades de cambio de estado mediante recorrido y estado_agendamiento.

Ccentraliza la lógica relacionada con la creación, modificación y seguimiento de citas médicas. Las entidades clave aquí son "Cita", "Estado de Agendamiento" y "Recorrido de Estados". Este contexto opera como un punto de interacción entre todo el sistema de agendamiento dado que requiere información del personal médico (disponibilidad y especialidad) y de los pacientes (datos y solicitudes).

- Catálogos

Este contexto engloba todas las configuraciones del sistema mediante las tablas catálogo, catalogo_tiempos y catalogo_tipo. Permite configurar parámetros dinámicos como tipos de citas, tiempos y otros elementos relacionados con las operaciones.

Las conexiones entre los contextos están diseñadas para representar dependencias mínimas y claras, las conexiones que se dan son:

- "Gestión de Personal Médico" provee información sobre los médicos y sus horarios a "Gestión de Citas", permitiendo asignar personal médico a las citas.
- "Gestión de Pacientes" se relaciona con "Gestión de Citas" para que los pacientes puedan agendar y gestionar sus citas.
- "Gestión de Catálogos" soporta tanto a "Gestión de Personal Médico" como a "Gestión de Citas" con información estructurada y centralizada.

Una vez identificados los contextos limitados de la aplicación se procede al diseño del sistema basado en microservicios.

Cada uno de estos contextos encapsula tareas específicas dentro del dominio del agendamiento. Como ejemplo se tiene contexto de Gestión de Citas se centra en la lógica para programar, modificar y cancelar citas, asegurando que estas operaciones estén aisladas de otros procesos, como la gestión de pacientes. Catálogos centraliza parámetros dinámicos como tipos de citas o tiempos predeterminados, permitiendo que el sistema sea flexible frente a cambios y actualizaciones.

Si en el futuro se requiere integrar nuevos servicios o funcionalidades como un módulo para notificaciones automáticas estos pueden añadirse sin alterar los contextos existentes ya que el sistema se adapta fácilmente a nuevas necesidades sin comprometer su estabilidad.

Para dividir en microservicios se agrupan las entidades en contextos funcionales según sus responsabilidades, la clasificación se la hizo de esta manera:

- **Microservicio de Gestión de Citas**: Gestiona el proceso decitas médicas. Incluye la lógica para verificar la disponibilidad de horarios, asignar citas a especialidades específicas y actualizar estados de las citas (Figura 5. Diagrama de Clases del Microservicio de Gestión de Citas).
    - Entidades: cita, estado_agendamiento, recorrido.
    - Funcionalidad: Manejo de citas y asignación de horarios.

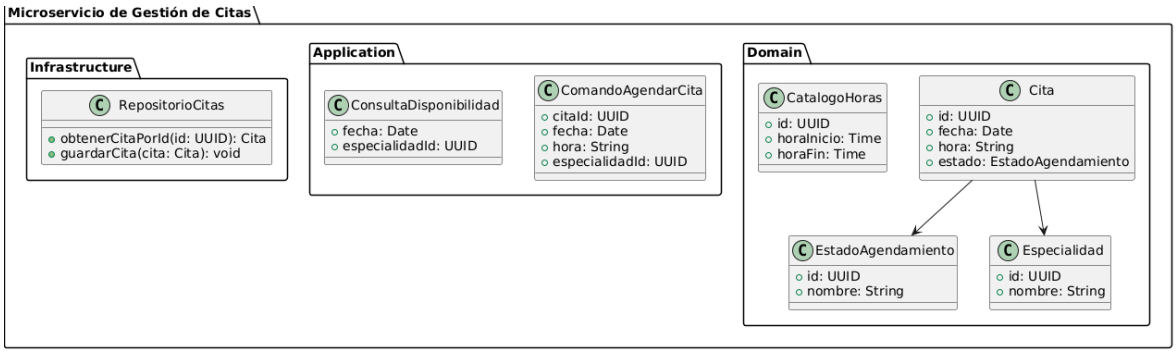


Figura 5. Diagrama de Clases del Microservicio de Gestión de Citas

- **Microservicio de Personal Médico**: Administra la información del personal médico como su nombre, especialidades y disponibilidad. Permite registrar nuevos médicos y asociarlos con especialidades específicas, también permite consultar las especialidades disponibles en el sistema (Figura 6. Diagrama de Clases del Microservicio de Personal Médico).
    - Entidades: personal, especialidad.
    - Funcionalidad: Gestión de información del personal médico y sus especialidades.

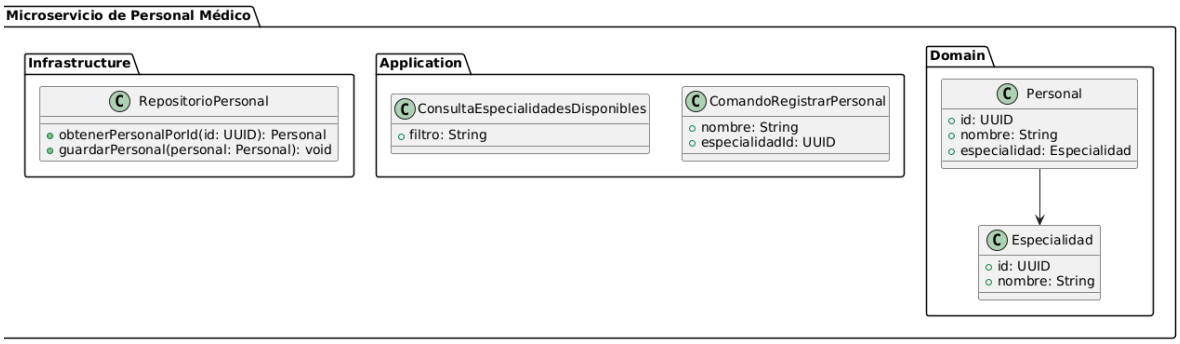


Figura 6. Diagrama de Clases del Microservicio de Personal Médico

- **Microservicio de Pacientes**: Centraliza la gestión de la información de los pacientes incluyendo su identificación, nombre e historial médico. Este historial se compone de registros detallados de eventos médicos pasados (Figura 7. Diagrama de Clases del Microservicio de Pacientes).
    - Entidades: paciente, detalle_lista.
    - Funcionalidad: Gestión de pacientes y sus listas o historiales.

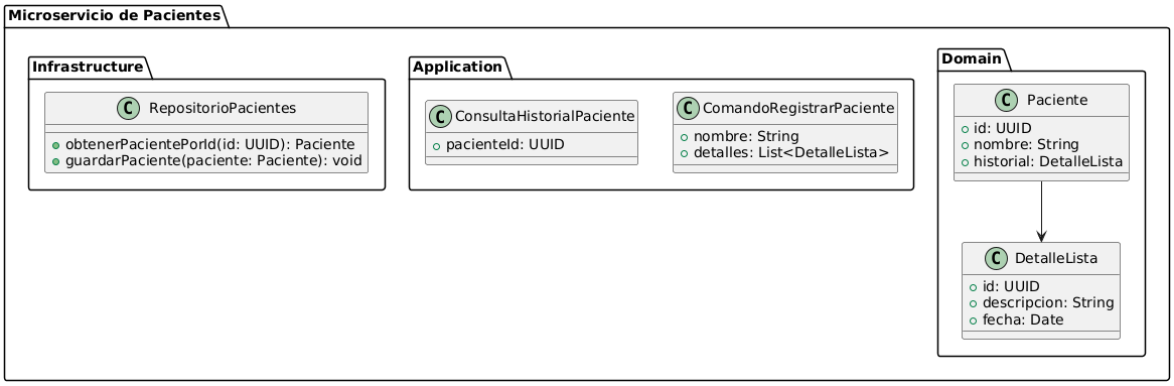


Figura 7. Diagrama de Clases del Microservicio de Pacientes

- **Microservicio de Catálogos**: Proporciona una estructura para manejar catálogos de información estática. Se incluye elementos como tipos de catálogos y configuraciones de tiempos (Figura 8. Diagrama de Clases del Microservicio de Catálogos).
    - Entidades: catalogo, catalogo_tipo, catalogo_tiempos. catalogo_horas
    - Funcionalidad: Configuración de catálogos y tiempos operativos.

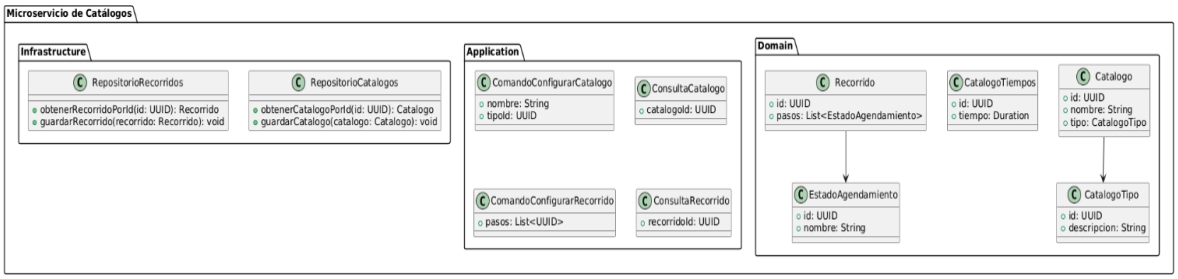


Figura 8. Diagrama de Clases del Microservicio de Catálogos

Patrón de diseño principal

El patrón principal utilizado en el diseño de microservicios ejemplificado con el Microservicio de Gestión de Citas en la Figura 9. Diagrama de Clases Enfocado a DDD, se basa en el enfoque de Domain-Driven Design (DDD). Aquí la arquitectura del sistema se basa

en capas y componentes cada una con sus responsabilidades alineadas a un dominio específico del negocio.

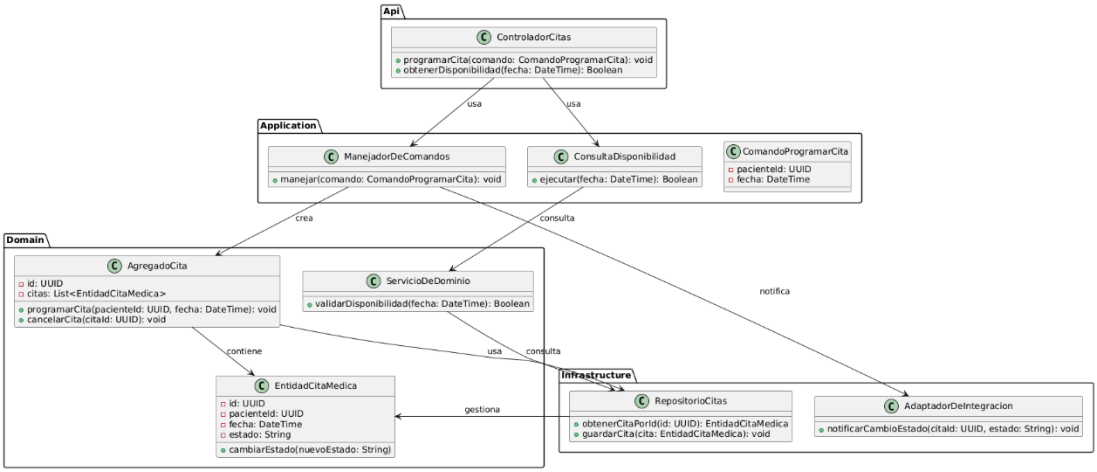


Figura 9. Diagrama de Clases Enfocado a DDD

El microservicio se estructura en cuatro capas principales: API, Aplicación, Dominio e Infraestructura.

La capa de API contiene los controladores que exponen la funcionalidad del microservicio al mundo exterior mediante endpoints REST. En el caso de de ControladorCitas se maneja solicitudes como programarCita o obtenerDisponibilidad delegando estas solicitudes a las operaciones de las capas internas para procesarlas.

En el núcleo del microservicio está la capa de Dominio que contiene toda la lógica de negocio, modelos y agregados. La clase AgregadoCita es el encargado de manejar operaciones clave como programar o cancelar citas las cuales son reglas de negocio.

En la capa de Aplicación se da la conexión entre las capas API y el dominio. Aquí no se implementa lógica de negocio directamente, sino que utiliza casos de uso, manejadores de comandos y consultas para coordinar las operaciones.

Por último, la capa de Infraestructura se ocupa de las interacciones con bases de datos y otros servicios. En este caso el RepositorioCitas gestiona las operaciones de almacenamiento y recuperación en la base de datos mientras que el AdaptadorDeIntegración notifica los cambios en el estado de las citas a otros sistemas.

Un ejemplo concreto del flujo de trabajo es cuando se necesita "Programar una cita". Todo comienza con una solicitud al endpoint programarCita, manejada por el ControladorCitas, que traduce esta solicitud en un ComandoProgramarCita y lo pasa a la capa de Aplicación. Aquí, el ManejadorDeComandos valida la información y delega la lógica de negocio al

AgregadoCita en la capa de Dominio. Este último verifica la disponibilidad, aplicando las reglas definidas en los servicios de dominio. Una vez validado, se actualiza la base de datos mediante el RepositorioCitas, y se envía una notificación a otros sistemas a través del AdaptadorDeIntegración.

Arquitectura Propuesta para el Core de servicios

La arquitectura propuesta en la Figura 10. Diagrama de Microservicios para Agendamiento de Citas Médicas para el Core de está diseñada bajo un enfoque de microservicios que se conectan a través de un API Gateway para simplificar el acceso y la gestión de solicitudes. Dada la conexión se centraliza las operaciones del sistema manejando la autenticación, autorización y enrutamiento hacia los microservicios correspondientes proporcionando una comunicación eficiente y segura entre los componentes del sistema.

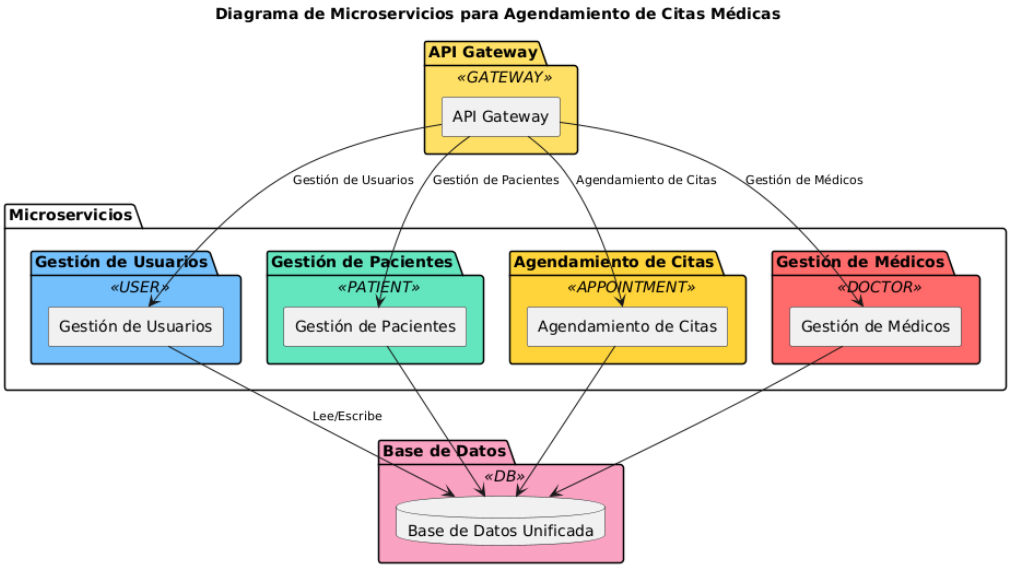


Figura 10. Diagrama de Microservicios para Agendamiento de Citas Médicas

El diseño del Back-end está estandarizado para funcionar con el Framework de .NET Core 8.0 que soporta la implementación de microservicios y orquestación de contenedores.

El sistema se compone de cuatro microservicios con diferentes contextos principales: Gestión de Usuarios, Gestión de Pacientes, Agendamiento de Citas y Gestión de Médicos. Estos microservicios están diseñados para ser autónomos con responsabilidades claramente delimitadas.

Todos los microservicios están diseñados bajo el principio de API RESTful y se comunican mediante el protocolo HTTPS siguiendo el estándar.

Al utilizar un API Gateway podemos mejorar significativamente la seguridad del sistema ya que se asegura que las solicitudes externas no accedan directamente a los microservicios individuales protegiendo la lógica interna.

Todos los microservicios interactúan directamente con una base de datos unificada, representada en el diagrama que centraliza el almacenamiento de datos teniendo en cuenta que fue uno de los requisitos de la DGIP al momento de trabajar con este caso de estudio para reducir la complejidad en la sincronización de información.

La contenerización mediante Docker es otro punto clave en la arquitectura dado que se permite encapsular cada microservicio junto con sus dependencias en contenedores, facilitando su despliegue y portabilidad entre entornos.

Selección de protocolos, seguridad y estándares RESTFul

El siguiente paso es definir los protocolos y estándares para los microservicios establecidos un ejemplo en el caso de estudio es el Microservicio de Gestión de Citas.

El protocolo de comunicación seleccionado es HTTP/HTTPS, ya que permite implementar los principios RESTful de manera eficiente, cabe destacar que para el uso de HTTPS se requiere un certificado SSL o TLS para poder implementarlo haciendo que las solicitudes sean seguras protegiendo datos sensibles como información de citas y pacientes.

La Figura 11. Métodos HTTP en el Microservicio de Citas describe los métodos HTTP utilizados en el microservicio de citas en el caso de estudio destacando cómo un cliente API interactúa con el controlador principal para gestionar las operaciones relacionadas con las citas, siguiendo también el enfoque de RESTful que se encarga de definir recursos entre la comunicación del cliente y el servidor.

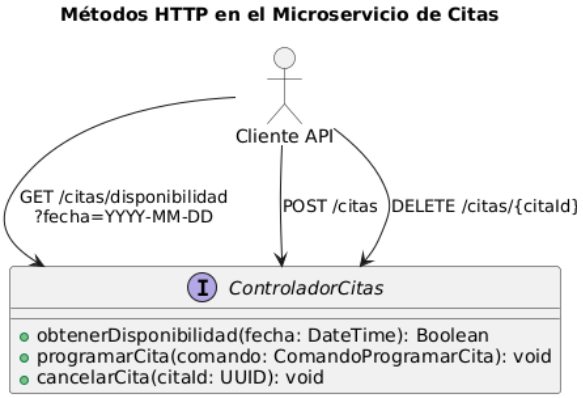


Figura 11. Métodos HTTP en el Microservicio de Citas

Aquí el Cliente API es un actor que representa cualquier aplicación o sistema externo que necesite interactuar con el microservicio.

Este cliente realiza solicitudes HTTP al controlador principal que se denomina "ControladorCitas", este controlador expone las operaciones necesarias para gestionar las citas.

Estos controladores a nivel general se basan en tres puntos clave

1. **Obtener disponibilidad**: En el ejemplo el cliente realiza una solicitud GET al endpoint /citas/disponibilidad con un parámetro de consulta fecha en el formato YYYY-MM-DD para verificar las citas disponibles para la fecha especificada y devuelve un valor booleano indicando la disponibilidad.

2. **Programar una cita**: Para crear una nueva cita el cliente realiza una solicitud POST al endpoint /citas. Esta operación recibe un comando con los datos necesarios para programar la cita asegurando que se cumplan las reglas de negocio definidas en el dominio.

3. **Cancelar una cita**: Si el cliente desea anular una cita ya programada, realiza una solicitud DELETE al endpoint /citas/{citaId}, donde {citaId} representa el identificador único de la cita que se desea cancelar.

El estándar seleccionado para la transferencia de datos es JSON ya que es ligero y fácil de integrar.

También la seguridad dentro de los microservicios es primordial, a continuación, la Figura 12. Flujo de Seguridad con JWT detalla el flujo de autenticación de usuarios mediante JWT, cabe destacar que si se requiere usar otro estándar de autorización como OAuth2 se lo

puede implementar en donde los tokens de acceso emitidos por OAuth2 pueden están en formato JWT.

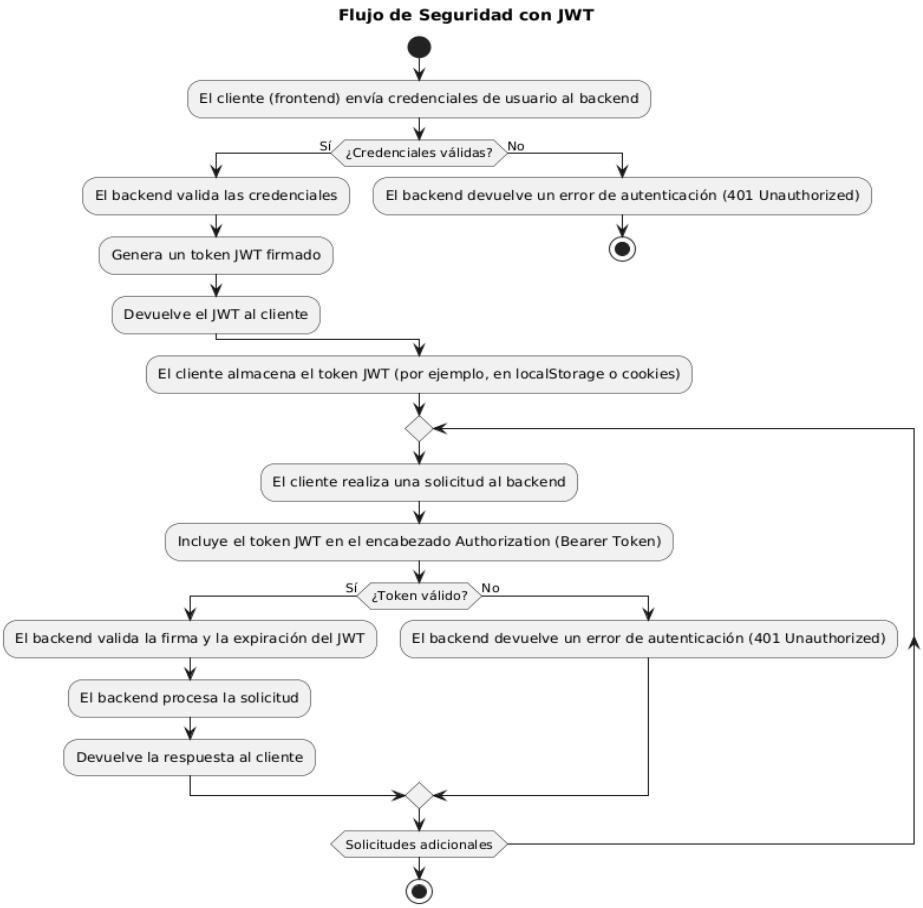


Figura 12. Flujo de Seguridad con JWT

Se puede observar el flujo que tendrá la autenticación y autorización dentro de cada uno de los microservicios en el caso de estudio, esto se logra utilizando JWT (JSON Web Tokens), lo importante es garantizar que las solicitudes realizadas a los recursos protegidos del backend sean autenticadas sin necesidad de mantener sesiones de usuario en el servidor.

Cuando un usuario intenta autenticarse envía sus al servidor a través de un endpoint de autenticación. El backend valida estas credenciales contra la base de datos y si las credenciales son correctas el backend genera un token JWT firmado que contiene información relevante sobre el usuario como su identificador único (userId), roles, permisos y una fecha de expiración.

El token JWT tiene tres partes codificadas en Base64: la cabecera (header), el cuerpo (payload) y la firma (signature). La cabecera especifica el algoritmo utilizado para la firma en este caso se usará una función hash SHA-256 que sirve como código de autenticación de mensajes basado en hash (HMAC). El cuerpo contiene la información del usuario (también llamada "claims"), como su ID, roles, y otros datos relevantes. La firma se genera utilizando una clave secreta definida en el backend, lo que asegura que el token no pueda ser modificado sin invalidarlo.

Diseño de una API Gateway centralizada

Cada servicio tiene su propia API diseñada para ser consumida por aplicaciones académicas y administrativas.

En el proyecto del caso de estudio se empleó Ocelot como API Gateway para gestionar la comunicación entre los clientes y los microservicios centralizando así las operaciones comunes como la autenticación, autorización y el enrutamiento.

La implementación del API Gateway permite abstraer y unificar el acceso a múltiples microservicios alineándose con el diseño arquitectónico del proyecto. Se lo desarrollo para que cada microservicio pueda tener la exposición de sus endpoints consolidados y la delegación de responsabilidades específicas sin que el cliente conozca los detalles de su ubicación o implementación.

Se eligió hacer con esta tecnología debido a su integración nativa con el ecosistema .NET y su capacidad para manejar configuraciones complejas de enrutamiento, autenticación y balanceo de carga de manera eficiente.

El diseño de la API Gateway se puede visualizar en la Figura 13. Diagrama API Gateway mostrando el flujo desde el cliente hasta los microservicios a través de la validación y el enrutamiento.

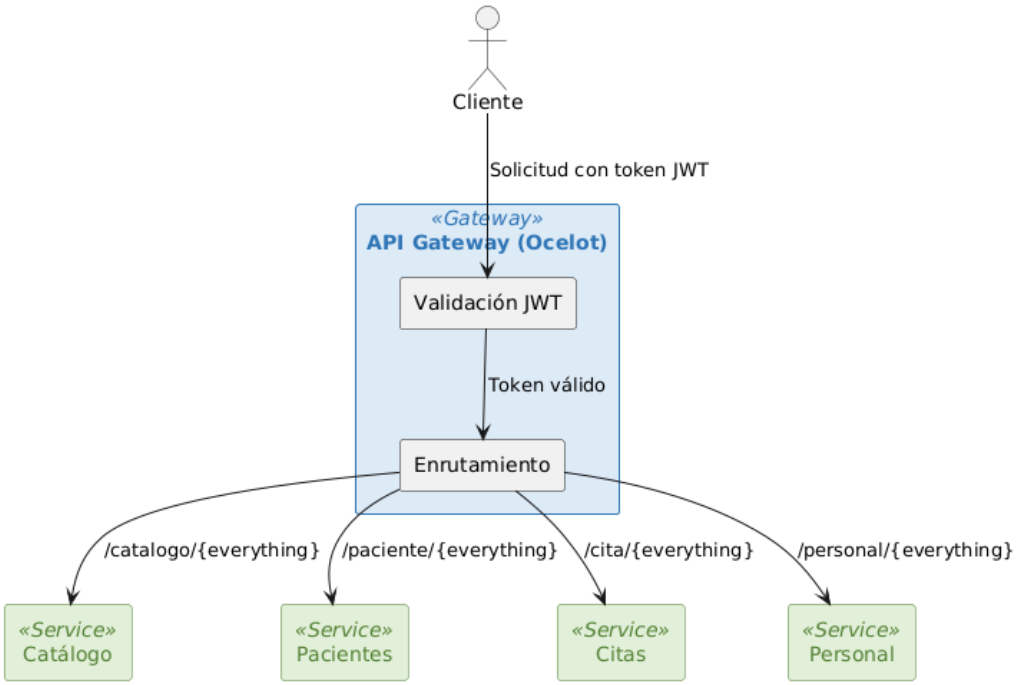


Figura 13. Diagrama API Gateway

Este diseño también maneja la validación de tokens JWT antes de redirigir las solicitudes a los microservicios lo que cual asegura que cada solicitud sea autenticada, delegando la responsabilidad de la autorización a la Gateway y simplificando la lógica en los microservicios individuales.

2.6 Desarrollo e Implementación del Core de servicios Web

El desarrollo del Core de servicios se enfocó en transformar el monolito de la aplicación de agendamiento de citas médicas de la DGIP hacia una arquitectura modular basada en microservicios. Como resultado, se creó una prueba de concepto (POC) que cumple con los estándares previamente establecidos y servirá como base fundamental para las futuras aplicaciones desarrolladas en la FIS, siguiendo el patrón de diseño Domain-Driven Design (DDD). Los microservicios implementados están diseñados para abordar casos de uso específicos, como la gestión de citas médicas y la consulta de horarios, entre otros.

El desarrollo inicial se lo realizó en un entorno virtual permitiendo validar la interacción de los servicios antes de su integración con el ecosistema académico completo. Este entorno provisto por la DGIP fue configurado como una máquina virtual dedicada con recursos escalables.

Herramientas utilizadas para desarrollo

Framework .NET 8

La elección de .NET 8 como framework principal para este desarrollo estuvo motivada por sus características avanzadas como el soporte para Minimal APIs, mejoras en el rendimiento y una mayor flexibilidad para desarrollar aplicaciones modernas y escalables. Además de la facilidad con la que permitió implementar servicios RESTful de manera eficiente, con soporte nativo para autenticación, autorización y conexiones a bases de datos.

Visual Studio y Visual Studio Code

- **Visual Studio**: Se utilizó como el IDE principal para el desarrollo y depuración de los microservicios durante la migración. Sus herramientas integradas, depuración avanzada y soporte para proyectos Docker, así como la facilidad de creación dentro del IDE permitieron un flujo de trabajo fluido.
- **Visual Studio Code**: Se empleó como un editor ligero para editar configuraciones, referencias entre distintas capas del diseño, dependencias del proyecto y archivos JSON como ocelot.json. Su amplia gama de extensiones facilitó tareas de mantenimiento y configuración.

Beaver

Esta herramienta permite analizar cualquier tipo de base de datos relacional de una manera fácil e intuitiva, se conectó a la base de datos utilizada para la aplicación de agendamiento de citas médicas de manera remota a través de Beaver, también fue utilizada para la gestión y administración de las bases de datos PostgreSQL y proporcionó una interfaz intuitiva para diseñar esquemas, ejecutar consultas SQL y realizar análisis de rendimiento en las operaciones de base de datos.

PostgreSQL

El sistema de gestión de bases de datos elegido fue PostgreSQL, debido a su robustez y soporte para características avanzadas como transacciones ACID. Se utilizo de manera local para hacer consultas y pruebas durante la migración a microservicios también esta base de datos almacena entidades críticas como pacientes, citas y configuraciones del sistema.

Swagger

Para la documentación de las APIs, se utilizó Swagger mediante el paquete Swashbuckle.AspNetCore. Así durante las pruebas unitarias durante el desarrollo esta herramienta genero automáticamente documentación interactiva basada en OpenAPI, facilitando resultados de las APIs al explorar y probar los servicios directamente desde el navegador.

Docker y **Docker Desktop**

Son herramientas clave para la contenedorización y orquestación de los microservicios que conforman el sistema. Docker permitió empaquetar cada microservicio junto con sus dependencias en contenedores aislados. Con Docker Desktop se facilitó la gestión local de los contenedores y su ejecución proporcionando una interfaz sencilla para controlar las imágenes, redes y volúmenes necesarios para el despliegue. Además, se utilizó estas herramientas para poder orquestar los diferentes microservicios creados y así monitorear los contenedores de manera eficiente visualizando una alta disponibilidad y un manejo dinámico de los recursos del sistema.

Ecosistema de desarrollo

La DGIP otorgo un entorno virtual el cual serviría como un entorno unificado de desarrollo y pruebas:

- **Infraestructura:** Máquinas virtuales proporcionadas por la DGIP.
- **Recursos del entorno:**
    - CPU: 8 núcleos.
    - RAM: 10 GB.
    - Almacenamiento: 150 GB SSD.
    - Sistema Operativo: Windows Server.

El primer paso en el desarrollo fue la implementación del sistema monolítico utilizando Docker para la contenedorización y Docker Compose para la orquestación. Con esto se permitió levantar rápidamente el entorno necesario para pruebas el cual será utilizado en el componente de pruebas de este tema de estudio.

El propósito principal de esta etapa fue crear un punto de referencia que se utilizará para realizar comparaciones directas durante las pruebas del nuevo sistema basado en microservicios.

Para comenzar se configuro el proyecto para que pudiera ejecutarse de manera eficiente dentro de un contenedor Docker, por lo que se tuvo que crear y añadir un Dockerfile con la ayuda de Visual Studio Code como lo muestra la Figura 14. Configuración de Docker con Visual Studio.

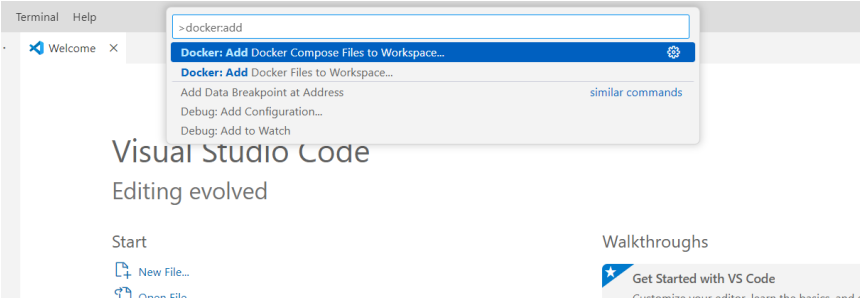


Figura 14. Configuración de Docker con Visual Studio

Este archivo definió las instrucciones necesarias para construir la imagen de la aplicación. Entre los aspectos destacados se incluyeron:

- La selección de ASP.NET Runtime como una imagen base adecuada para el entorno de ejecución.
- La copia de los archivos del proyecto al contenedor.
- La restauración de dependencias y recompilación del proyecto.
- La exposición del puerto en el que la aplicación escucha solicitudes en este caso se lo creo para el puerto 8080

Una vez creado el contenedor se creó un archivo docker-compose.yml para definir y orquestar el funcionamiento del sistema.

En el Anexo I se puede observar el archivo que se creó que permite al proyecto ejecutarse desde un contenedor, el servicio principal en este archivo es dbp-agendamiento-backend, que representa el backend monolítico de la aplicación.

El servicio se construye utilizando un Dockerfile ubicado en el directorio dbp-agendamiento-backend. La opción build define el contexto (carpeta raíz del proyecto) y la ubicación del Dockerfile correspondiente.

También se definieron puertos para la conexión exterior con el proyecto.

- El puerto 8080 dentro del contenedor se mapea al puerto 5000 en la máquina anfitriona para el tráfico HTTP.

- El puerto 8081 se mapea al puerto 5001 para tráfico HTTPS.

Dentro del archivo también se especifica una clave secreta para la generación y validación de tokens JWT.JwtSettings__Issuer y JwtSettings__Audience estas dos establecen la entidad emisora y los destinatarios del token.

Al igual fue necesario definir la conexión a la base de datos PostgreSQL alojada en el servidor 172.31.203.223. Incluye credenciales y parámetros como el nombre de la base de datos las cuales en el anexo no aparecen completas por acuerdos de confidencialidad. Esto garantiza que el backend pueda interactuar con la base de datos y realizar operaciones CRUD.

La línea de código ASPNETCORE_ENVIRONMENT está establecido en Development para asegurar que el backend utilice configuraciones apropiadas para el entorno de desarrollo, como logs más detallados y tolerancia a errores.

El servicio se ejecuta dentro de una red llamada external-network, configurada con el controlador bridge. Este tipo de red permite que los contenedores dentro de la misma red se comuniquen entre sí de manera aislada, mientras que aún pueden interactuar con el mundo exterior si es necesario, esto servirá como punto de apoyo al momento de configurar la orquestación de contenedores.

En la Figura 15. Integración de Docker al monolito se tiene la estructura final para el sistema monolítico incluyendo la orquestación.

Figura 15. Integración de Docker al monolito

Tras iniciar el contenedor, se realizaron pruebas de conectividad para asegurar que la aplicación pudiera comunicarse correctamente con la base de datos. Esto incluyó verificar que las credenciales configuradas en el archivo docker-compose.yml coincidieran con las especificadas en el appsettings.json del proyecto.

Desarrollo de microservicios

Una vez que el proyecto original estuviera con Docker se procedió a la separación de microservicios. A partir del análisis y el diseño anterior se procedió a examinar el contenido de cada una de las carpetas del proyecto original para poder separarlas de acuerdo con el patrón de diseño.

El patrón de diseño de cada uno de los microservicios que estarán desarrollándose es DDD, la cual nos permite dividir la estructura del proyecto en cuatro capas, cada una con una responsabilidad diferente.

Como lo indica la Figura 16. Estructura de Carpetas de Microservicios DDD la estructura general del proyecto dentro de las aplicaciones de la DGIP para los microservicios estará dividida en las capas de Api, Aplicación, Dominio e Infraestructura.

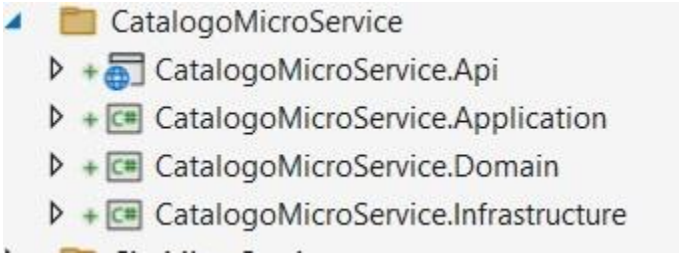


Figura 16. Estructura de Carpetas de Microservicios DDD

Cada una de estas capas cumplirá una función clave para la creación del microservicio ya que tienen un contexto delimitado con su propia lógica de negocio y datos.

Cabe destacar que cada una de estas capas es un proyecto creado en .NET, la capa de API es un proyecto ASP.NET Core Web Api mientras que las demás capas son de tipo ConsoleApp dado que estos complementan el funcionamiento de los microservicios.

La transición comenzó con la identificación de los dominios principales de la aplicación en el monolito. Estos dominios fueron extraídos y organizados en una nueva estructura de carpetas, alineada con los principios de DDD.

En lugar de mantener todo el código en un único proyecto se separaron las responsabilidades en cuatro proyectos diferentes dentro de cada microservicio, de ejemplo se utiliza el microservicio de catálogos para esta parte del desarrollo, la Figura 17. Estructura de Carpetas Centrada en Dominios muestra la estructura de carpetas en la cual se separó el proyecto original, cada carpeta destinada a un dominio diferente.

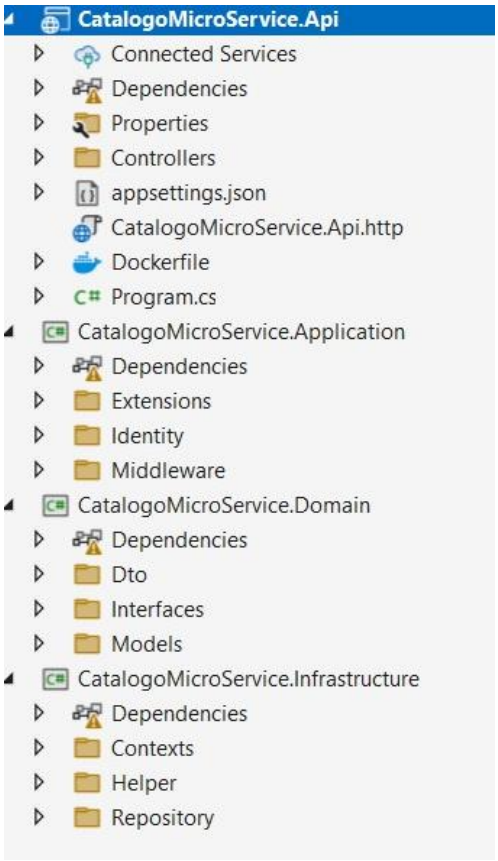


Figura 17. Estructura de Carpetas Centrada en Dominios

- **/Domain:** Agrupa todo lo relacionado con el dominio del negocio, incluyendo entidades y objetos de valor que representan conceptos clave.
    - **/Models**: Aquí se mueven las entidades desde la carpeta Models.
    - **/Interfaces**: Se mantiene aquí cualquier interfaz relacionada con el dominio.
    - **/DTOs:** Contiene objetos simples usados para transportar datos entre capas sin exponer las entidades del dominio.
- **/Application:** Contiene la lógica de negocio y los servicios que operan sobre el dominio.
    - / **Middleware**: Para manejar el middleware de la aplicación como la generación de tokens.
    - /**Identity**: Gestión de usuarios y autenticación.

- /**Extensions**: Métodos de extensión para mejorar el modularidad.

- **/Infrastructure:** Maneja la infraestructura del proyecto, incluyendo acceso a datos, repositorios y configuraciones relacionadas con la infraestructura.
  - **/Repositories:** Define e implementa los repositorios para interactuar con la persistencia de datos.
  - **/Contexts**: Contendrá los contextos de base de datos (por ejemplo, DbContext)
  - **/Helpers**: Utilidades necesarias para las operaciones de la aplicación.
- / **Apis:** El proyecto implementa la interacción del microservicio, el acceso a redes remotas y las API web externas utilizadas desde aplicaciones cliente o de interfaz de usuario.
  - **/Program.cs:** Define el punto de entrada de la aplicación.
  - **/Controllers**: Los controladores se mantienen aquí.

Con cada una de las carpetas ya creadas en su respectivo dominio lo que sigue es armar correctamente las dependencias de los servicios de DDD, como indica la Figura 18. Capas de Microservicios, la capa de aplicación depende del dominio y la infraestructura, y la infraestructura depende del dominio, pero el dominio no depende de ninguna capa.

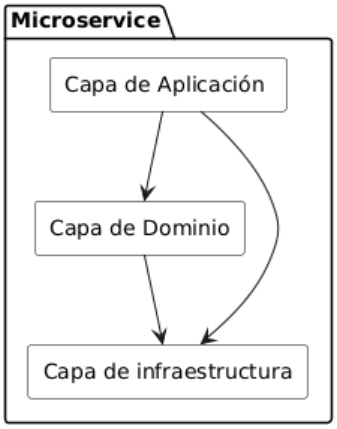


Figura 18. Capas de Microservicios

Este diseño de nivel debe ser independiente para cada microservicio. Se puede implementar microservicios más complejos siguiendo patrones DDD, al mismo tiempo que se puede implementar microservicios guiados por datos más simples (un único CRUD en un solo nivel) de una forma más sencilla.

Para esto dentro del código para estructurar un proyecto basado en estas capas en .NET, se deben agregar referencias explícitas entre los proyectos utilizando comandos de .NET CLI para mantener estas relaciones bien definidas.

La capa de aplicación necesita interactuar con las entidades y repositorios definidos en el dominio. En la Figura 19. Referencias de la Capa API, se puede ver las referencias agregadas al dominio API que utiliza la lógica de los casos de uso definida en la capa de aplicación.

```
<ItemGroup>
  <ProjectReference Include="..\AgendamientoMicroService.Application\CatalogoMicroService.Application.csproj" />
  <ProjectReference Include="..\AgendamientoMicroService.Domain\CatalogoMicroService.Domain.csproj" />
  <ProjectReference Include="..\AgendamientoMicroservice.Infrastructure\CatalogoMicroService.Infrastructure.csproj" />
</ItemGroup>
```

Figura 19. Referencias de la Capa API

Si la API necesita acceder directamente a implementaciones de infraestructura, como conectarse a una base de datos o un proveedor de servicios, se agrega esta referencia con el comando:

dotnet add Api/Api.csproj reference Infrastructure/Infrastructure.csproj

La capa de aplicación necesita interactuar con las entidades y repositorios definidos en el dominio y la infraestructura como indica la Figura 20. Referencias de la Capa Aplicación.

```
<ItemGroup>
  <ProjectReference Include="..\CatalogoMicroService.Domain\CatalogoMicroService.Domain.csproj" />
  <ProjectReference Include="..\CatalogoMicroService.Infrastructure\CatalogoMicroService.Infrastructure.csproj" />
</ItemGroup>
```

Figura 20. Referencias de la Capa Aplicación

La Figura 21. Referencias de la Capa Infraestructura indica en cambio las referencias de la capa de infraestructura la cual implementa los repositorios definidos en el dominio, por lo que debe depender de esta.

```
<ItemGroup>
  <ProjectReference Include="..\CatalogoMicroService.Domain\CatalogoMicroService.Domain.csproj" />
  <ProjectReference Include="..\CatalogoMicroService.Application\CatalogoMicroService.Application.csproj" />
</ItemGroup>
```

Figura 21. Referencias de la Capa Infraestructura

Mientras que la capa de dominio no contiene ninguna referencia debido a que esta es la que define los modelos y las interfaces que van a requerir las otras capas del proyecto.

Estrategias de integración de datos en microservicios

Para desarrollar el Core de servicios basadas en Domain-Driven Design (DDD) es fue necesario mantener la independencia y el aislamiento de cada servicio. Sin embargo, en escenarios donde se requiere integrar datos o unificar consultas que pertenecen a

diferentes servicios y contextos es necesario considerar estrategias que equilibren la autonomía de los dominios con la eficiencia y la simplicidad operativa.

- **Unificación de tablas en un único DataContext:** En la Figura 22. Código de Unificación de Tablas se muestra que una solución a las consultas fue combinar las entidades de diferentes microservicios en un solo contexto para simplificar consultas y operaciones. Aunque facilita el manejo de datos relacionados, aumenta el acoplamiento y compromete los principios de DDD. Esta estrategia solo es recomendable en casos excepcionales donde la complejidad técnica lo justifique.

```
22 references | 0 changes | 0 authors, 0 changes
public partial class DataContext : DbContext
{
    0 references | 0 changes | 0 authors, 0 changes
    public DataContext()
    {
    }

    0 references | 0 changes | 0 authors, 0 changes
    public DataContext(DbContextOptions<DataContext> options)
        : base(options)
    {
    }

    5 references | 0 changes | 0 authors, 0 changes
    public virtual DbSet<Catalogo> Catalogos { get; set; }

    4 references | 0 changes | 0 authors, 0 changes
    public virtual DbSet<CatalogoHora> CatalogoHoras { get; set; }

    0 references | 0 changes | 0 authors, 0 changes
    public virtual DbSet<CatalogoTiempo> CatalogoTiempos { get; set; }

    1 reference | 0 changes | 0 authors, 0 changes
    public virtual DbSet<Catalogotipo> Catalogotipos { get; set; }

    8 references | 0 changes | 0 authors, 0 changes
    public virtual DbSet<Cita> Cita { get; set; }

    8 references | 0 changes | 0 authors, 0 changes
    public virtual DbSet<DetalleLista> DetalleLista { get; set; }

    6 references | 0 changes | 0 authors, 0 changes
    public virtual DbSet<Especialidad> Especialidades { get; set; }

    0 references | 0 changes | 0 authors, 0 changes
    public virtual DbSet<EstadoAgendamiento> EstadoAgendamientos { get; set; }

    6 references | 0 changes | 0 authors, 0 changes
    public virtual DbSet<Paciente> Pacientes { get; set; }

    6 references | 0 changes | 0 authors, 0 changes
    public virtual DbSet<Personal> Personales { get; set; }

    0 references | 0 changes | 0 authors, 0 changes
    public virtual DbSet<Recorrido> Recorridos { get; set; }
```

Figura 22. Código de Unificación de Tablas

- **División de DataContexts según dominios:** Mantener múltiples DataContexts, cada uno enfocado en su propio dominio como lo indica la Figura 23. Estructura de División de DataContext. Esto preserva la independencia de los microservicios y respeta los límites del modelo DDD. Sin embargo, puede complicar las transacciones distribuidas y el manejo de dependencias cruzadas.

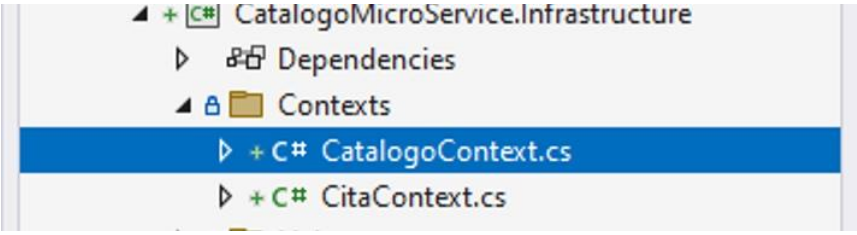


Figura 23. Estructura de División de DataContext

- **Consultas SQL directas:** Realizar consultas optimizadas directamente a la base de datos, sin pasar por un ORM como Entity Framework. Esta estrategia permite un mejor control y rendimiento en operaciones específicas, pero reduce la portabilidad y la abstracción. Debe aplicarse solo en casos donde se busque optimización puntual.(Figura 24. Código de consultas SQL directas)

```
2 references | 0 changes | 0 authors, 0 changes
public async Task<ICollection<Cita>> GetCitasByEspecialidadId(int especialidadId)
{
    var citas = new List<Cita>();

    var sql = "SELECT c.* FROM Medico.cita c INNER JOIN Medico.catalogohoras ch ON c.id_catalogohoras = ch.id_catalogohoras WHERE ch.id_especialidad = @especialidadId";

    using (var connection = _context.Database.GetDbConnection())
    {
        await connection.OpenAsync();
        using (var command = connection.CreateCommand())
        {
            command.CommandText = sql;
            command.Parameters.Add(new NpgsqlParameter("@especialidadId", especialidadId));

            using (var reader = await command.ExecuteReaderAsync())
            {
                while (await reader.ReadAsync())
                {
                    citas.Add(MapToCita(reader));
                }
            }
        }
    }

    return citas;
}
```

Figura 24. Código de consultas SQL directas

- **Consultas a través de APIs:** Usar APIs de otros microservicios para obtener datos relacionados en lugar de acceder directamente a sus bases de datos. Esto mantiene la autonomía y el desacoplamiento entre servicios, aunque introduce latencia y requiere manejar fallos mediante estrategias. (

- Figura 25. Código para Consulta de un API)

```
// Nuevo método para llamar al servicio externo
2 references | 0 changes | 0 authors, 0 changes
private async Task<Especialidad?> GetEspecialidadById(int especialidadId)
{
    var response = await _httpClient.GetAsync($"http://url-del-microservicio-especialidad/api/especialidades/{especialidadId}");
    if (response.IsSuccessStatusCode)
    {
        var content = await response.Content.ReadAsStringAsync();
        return JsonSerializer.Deserialize<Especialidad>(content);
    }
    return null;
}
```

Figura 25. Código para Consulta de un API

- **Eventos de integración:** Implementar comunicación entre microservicios mediante eventos de integración para separar las operaciones de lectura y escritura. Esta estrategia promueve la escalabilidad y el desacoplamiento, pero añade complejidad en la gestión de eventos y la sincronización de datos. Es ideal para sistemas con alta demanda de consultas complejas. Se lo hace mediante:
    - **Publicar un Evento**: Cuando CitaMedica es creada, se publica un evento CitaMedicaCreadaEvent que contenga la información básica del médico (como un ID), sin incluir directamente la entidad Medico.(Figura 26. Código para publicar evento)

```
    // Publicar el evento
    var citaCreadaEvent = new CitaMedicaCreadaEvent(
        nuevaCita.Id,
        nuevaCita.MedicoId,
        nuevaCita.Fecha,
        nuevaCita.PacienteId
    );

    await _eventBus.PublishAsync(citaCreadaEvent);
}
```

Figura 26. Código para publicar evento

    - **Consumir el Evento**: El microservicio responsable de la entidad Medico escucha este evento y puede reaccionar, si es necesario, sin acoplarse directamente a la entidad CitaMedica.(Figura 27. Código para consumir evento)

```
public async Task Handle(CitaMedicaCreadaEvent @event)
{
    // 1. Validar que el evento tenga los datos necesarios
    if (@event == null )
    {
        throw new ArgumentException("Evento inválido recibido.");
    }

    // 2. Buscar información del médico
    var medico = await _medicoRepository.GetMedicoById(@event.MedicoId);
    if (medico == null)
    {
        // Log o manejo de error: El médico no existe
        Console.WriteLine($"Médico con ID {@event.MedicoId} no encontrado.");
        return;
    }

    // 3. Registrar información relacionada con la cita médica
    var citaDetails = new CitaMedicaDetails
    {
        CitaId = @event.CitaId,
        MedicoId = @event.MedicoId,
        MedicoNombre = medico.NombreCompleto,
        Fecha = @event.Fecha,
        PacienteId = @event.PacienteId
    };

    // Guardar información en el repositorio de citas (u otro flujo necesario)
    await _citaRepository.SaveCitaMedicaDetails(citaDetails);
```

Figura 27. Código para consumir evento

Se desarrollo todos los microservicios identificados en la etapa de diseño teniendo en cuenta los principios de separación de responsabilidades y bajo acoplamiento anteriormente planteados en donde cada microservicio sigue la misma estructura en base de capas (Dominio, Aplicación, Infraestructura y API) la cual se observa en la Figura 28. Estructura de Archivos de Microservicios.

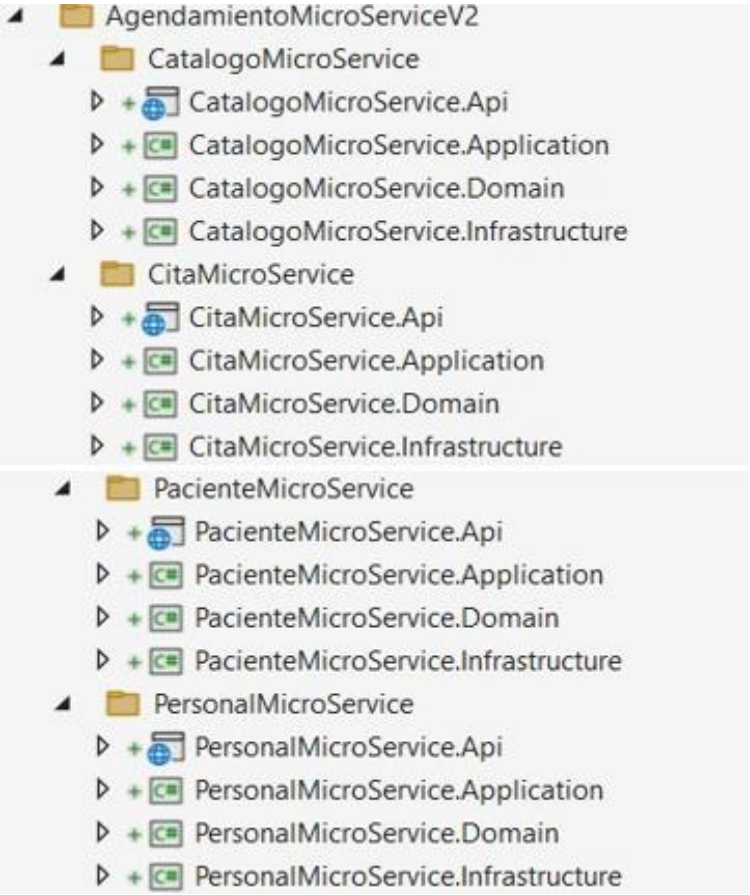


Figura 28. Estructura de Archivos de Microservicios

Para el microservicio de gestión de citas se definió un dominio que contenía las entidades principales como Cita y las reglas de negocio relacionadas con su programación y cancelación. Este mismo proceso de separación de entidades se replicó en cada uno de los microservicios como el de gestión de pacientes donde se modelaron entidades como Paciente y se implementaron reglas específicas para su creación y actualización.

Cada microservicio se construyó como un módulo independiente, pero siguiendo una estructura homogénea con el fin de facilitar la integración de estos microservicios en otras aplicaciones dentro de la DGIP en un futuro.

Integración de Servicios Específicos y API gateway

En el proyecto de microservicios se implementó un API Gateway como punto de entrada único para todas las solicitudes hacia los servicios individuales, se lo creo como otro proyecto nuevo en el que solo estará la configuración de las rutas de los distintos microservicios al que igualmente se lo integro con Docker. Con esta integración se facilita el enrutamiento de las solicitudes hacia los microservicios correspondientes y también permite implementar políticas centralizadas de seguridad y control de acceso.

API Gateway

El API Gateway se lo creo como un proyecto API totalmente separado dentro de .NET, aquí se lo configuro con la estructura de archivos que se aprecia en la Figura 29. Estructura de Archivos de API Gateway, además se añadió un archivo ocelot.json que refleja las rutas internas de los microservicios basadas en los puertos definidos en el launchSettings.json se siguieron varios pasos clave para integrar y configurar correctamente los microservicios.

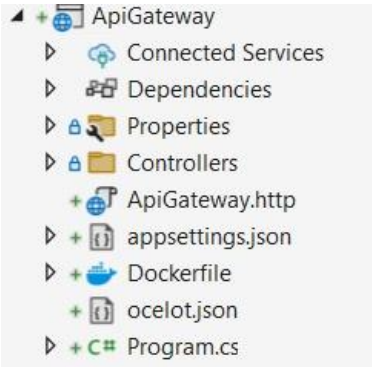


Figura 29. Estructura de Archivos de API Gateway

En primer lugar, se establecieron las configuraciones individuales de cada microservicio en su respectivo archivo launchSettings.json. Aquí se define los perfiles de inicio para cada uno de los servicios: CatalogoMicroService, CitaMicroService, PersonalMicroService y PacienteMicroService. Cada perfil contiene información específica como el comando de inicio (commandName), la URL de inicio del navegador (launchBrowser y launchUrl), las variables de entorno necesarias para configurar el entorno de desarrollo, y el puerto asignado para cada servicio en el atributo applicationUrl.

Un ejemplo es el microservicio de catálogo que fue configurado para ejecutarse en http://localhost:5101, mientras que los demás servicios se asignaron a puertos consecutivos: 5102, 5103 y 5104 (Anexo I).

El siguiente paso consistió en la creación del archivo ocelot.json, que es el núcleo de la configuración del API Gateway donde se define las rutas necesarias para redirigir las solicitudes entrantes hacia los microservicios correspondientes.

Cada entrada en el atributo ReRoutes especifica un conjunto de parámetros, en el mismo caso de catalago la ruta de entrada (UpstreamPathTemplate) /catalogo/{everything} se asigna a la ruta de destino (DownstreamPathTemplate) /api/catalogo/{everything} en el puerto 5101.

También se establece el protocolo de comunicación como HTTP mediante DownstreamScheme y se permite el uso de métodos HTTP comunes como GET, POST, PUT y DELETE. Este patrón se repite para los otros microservicios asignándoles rutas de entrada como /cita, /personal y /paciente, vinculadas a sus respectivos puertos como lo especifica la Tabla 5. Configuración del API Gateway.

Tabla 5. Configuración del API Gateway

Microservicio	**Puerto**	**URL de acceso (launchSettings)**	**Ruta de entrada (UpstreamPathTemplate)**	**Ruta de destino (DownstreamPathTemplate)**
CatalogoMicroService	5101	http://localhost:5101/swagger	/catalogo/{everything}	/api/catalogo/{everything}
CitaMicroService	5102	http://localhost:5102/swagger	/cita/{everything}	/api/cita/{everything}
PersonalMicroService	5103	http://localhost:5103/swagger	/personal/{everything}	/api/personal/{everything}
PacienteMicroService	5104	http://localhost:5104/swagger	/paciente/{everything}	/api/paciente/{everything}
API Gateway	5000	http://localhost:5000	N/A	N/A

Al final del archivo ocelot.json se incluye una sección global denominada GlobalConfiguration, donde se define la URL base del API Gateway, configurada como http://localhost:5000. Esta URL actúa como el punto central desde donde se canalizan todas las solicitudes hacia los microservicios (Anexo I).

Seguridad y control de acceso

La seguridad en el sistema se basa en el uso de tokens JWT y una capa adicional de validación mediante HMAC (Hash-based Message Authentication Code) incluida dentro del API Gateway.

Este middleware se implementó para añadir una capa adicional de seguridad a las solicitudes entrantes en el sistema. Aunque si bien los tokens JWT proporcionan autenticación y autorización, pueden ser interceptados en redes inseguras por eso el uso de HMAC permite asegurar que las solicitudes no hayan sido manipuladas durante su

tránsito, ya que verifica su integridad mediante una firma criptográfica generada con una clave secreta compartida.

Primero se verifica que la cabecera X-Signature contenga una firma HMAC válida y que la cabecera Authorization incluya un token JWT. Luego genera una firma HMAC utilizando el token JWT como entrada y una clave secreta predefinida. Esta firma calculada se compara con la firma recibida en la solicitud y si coinciden se garantiza que la solicitud no fue alterada esto se lo puede ver en el Anexo I.

Además, en la configuración del archivo Program.cs, se definen los middlewares encargados de procesar las solicitudes antes de que estas lleguen a los controladores.

```
app.UseHttpsRedirection();
app.UseCors("CorsPolicy");
app.UseCors("AllowAll");
app.UseMiddleware<HmacAuthenticationMiddleware>(); //Middleware para la autenticacion HMAC
app.UseAuthentication(); // Asegura que el middleware de autenticacion esta configurado
```

Figura 30. Código de Configuración de Middleware

En la Figura 30. Código de Configuración de Middleware se puede ver lo usado en la línea de código de app.UseHttpsRedirection() se asegura que todas las solicitudes sean redirigidas de HTTP a HTTPS, lo que protege la comunicación al evitar conexiones inseguras y posibles ataques de intermediarios. La configuración de CORS se establece con dos políticas diferentes. La llamada a app.UseCors("CorsPolicy") aplica una política de control de acceso que permite especificar qué dominios, métodos y cabeceras tienen permiso para interactuar con el servicio y también se incluye la política AllowAll mediante app.UseCors("AllowAll"), que permite acceso desde cualquier origen que fue fundamental en el desarrollo.

Por último esta la linea app.UseMiddleware<HmacAuthenticationMiddleware>() que añade una capa adicional de seguridad a las solicitudes entrantes y se incluye de igual manera el middleware estándar de autenticación de ASP.NET Core mediante app.UseAuthentication(). Este middleware utiliza esquemas como JWT para validar las credenciales de los usuarios. Su objetivo es autenticar a los usuarios antes de que puedan acceder a los recursos protegidos del sistema.

Configuración de Docker y orquestación

Una vez completada la configuración del API Gateway se implementó la configuración y orquestación de los microservicios de este proyecto, se lo implemento mediante el uso de Docker y Docker Compose.

Cada microservicio tiene su propio archivo Dockerfile que sigue un patrón común como se lo ve en el Anexo I en cada Dockerfile se define una etapa base para configurar el entorno de ejecución, una etapa de construcción para compilar el código fuente, una etapa de publicación para optimizar los artefactos compilados y una etapa final para preparar la ejecución del microservicio.

Para gestionar la orquestación de los contenedores de microservicios, se utiliza un archivo docker-compose.yml como esta en la Figura 31. Archivo docker-compose.yml, aquí se integra configuraciones específicas para el entorno, cada servicio cuenta con su propia imagen, contexto de construcción, puertos expuestos y variables de entorno.

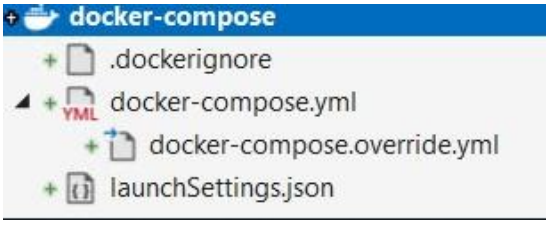


Figura 31. Archivo docker-compose.yml

En el Anexo I se puede observar el archivo docker-compose.yml el cual está configurado para orquestar los contenedores de cada uno de los microservicios.

Todos los servicios están conectados a la misma red apigateway-network lo que les permite comunicarse de manera eficiente entre sí. Esta red utiliza el controlador bridge, que es el controlador predeterminado en Docker para la creación de redes de contenedores por lo que los microservicios pueden enviarse solicitudes HTTP mutuamente sin necesidad de exponer puertos adicionales fuera de la red local.

También se ha configurado el entorno de desarrollo con ASPNETCORE_ENVIRONMENT=Development para que las aplicaciones se ejecuten en modo de desarrollo y permitan un ciclo de vida más flexible para pruebas y depuración.

Por último, estos microservicios escuchan en el puerto 80 como lo indica la Tabla 6. Configuración de Docker Compose, dentro de cada contenedor se lo ha configurado con la variable ASPNETCORE_URLS=http://+:80.

Tabla 6. Configuración de Docker Compose

Contenedor	Puerto Expuesto (Host)	Puerto Interno (Contenedor)	Imagen Docker	Dependencias
ApiGateway	8080, 8081	8080, 8081	apigateway	Depende de catalogo, cita, personal, paciente
Catalogo	5101	80	catalogomicroservice	No depende de otros microservicios
Cita	5102	80	citamicroservice	No depende de otros microservicios
Paciente	5104	80	pacientemicroservice	No depende de otros microservicios
Personal	5103	80	personalmicroservice	No depende de otros microservicios

2.7 Pruebas de desarrollo

Este apartado describe las pruebas realizadas para validar los módulos desarrollados en el caso de estudio de la DGIP en un entorno controlado. El objetivo principal es asegurar la conexión de los endpoints de los diferentes microservicios desarrollados, así como validar y verificar el código mediante pruebas unitarias según los requisitos funcionales y no funcionales antes de su transición al entorno de pruebas formal.

Herramientas utilizadas para pruebas

Swagger

Fue empleado como una herramienta principal para la documentación y validación de las APIs desarrolladas dada su capacidad para generar una interfaz interactiva permitió:

- Explorar y probar los endpoints en tiempo real.
- Verificar la estructura de los datos enviados y recibidos.
- Confirmar que los métodos implementados cumplieran con los contratos definidos previamente.
- Identificar posibles errores en la especificación de los servicios antes de realizar pruebas más exhaustivas.

Postman

Se utilizo para realizar pruebas manuales y automatizadas en los endpoints del sistema como:

- Envío de solicitudes HTTP con diferentes parámetros y cuerpos.
- Validación de respuestas, códigos de estado y tiempos de ejecución.
- Automatización de pruebas con la creación de colecciones para verificar secuencias de endpoints mediante scripts.

Newman

La principal herramienta CLI de Postman se la empleo para la ejecución automatizada de las colecciones anteriormente creadas, esta herramienta sirvió para:

- Generar reportes detallados sobre el desempeño y errores detectados durante las ejecuciones.

Metodología de pruebas de desarrollo

La metodología de pruebas en el componente de desarrollo se basó en pruebas unitarias con validación funcional. Al probar los microservicios en Postman se enfocó en validar de manera aislada cada funcionalidad individual de los servicios. Se lo desarrollo mediante el uso de scripts para validaciones y verificaciones automatizadas de resultados en donde los scripts actúan como aserciones para determinar si las respuestas del servicio cumplen con

los criterios establecidos (por ejemplo, códigos de estado, estructura de datos o valores específicos).

La documentación del resultado se basó en el modelo pruebas de caja negra ya que solo se evaluaron las salidas del sistema para asegurar que las funcionalidades individuales cumplan con los criterios técnicos establecidos como el tiempo de respuesta o el cumplimiento de una acción esperada bajo un escenario particular.

Por último, se realizaron pruebas de seguridad del sistema especialmente en lo relacionado con la autenticación y el manejo de tokens. Se probaron los procesos de generación y validación de los tokens JWT utilizados para autenticar usuarios y proteger las rutas de los microservicios.

Validación de Contenedores y orquestación

Antes de realizar las pruebas se verificó que el contenedor del backend del monolito estuviera correctamente levantado utilizando el comando docker-compose.

dockercompose109767664	-	-	-	0%	23 minutes ago
dbp-agendamiento-back	7069a1161cd9	dbpagendamientobackend:		0%	23 minutes ago

Figura 32. Contenedor del Monolito

La Figura 32. Contenedor del Monolito muestra el contenedor del monolito como dbpagamientobackend indicando que el entorno está preparado y configurado adecuadamente para proceder con las pruebas.

Después para realizar las pruebas de cada servicio como primer paso se levantaron todos los contenedores correspondientes a los microservicios y al API Gateway utilizando docker-compose para comprobar que cada componente estuviera operativo y accesible desde el entorno configurado para las pruebas. La lista de contenedores activos se lo muestra en la Figura 33. Orquestación de Microservicios.

CatalogoMicroService.Api	324499a43e5d	catalogomicroserviceapi:de	Show all ports (2)	0.06%	1 hour ago
CitaMicroService.Api	ab15c7d4b626	citamicroserviceapi:dev	32776:8080 Show all ports (2)	0.06%	1 hour ago
DetalleMicroService.Api	a597c0fcb9da	detallemicroserviceapi:dev	32778:8080 Show all ports (2)	0.06%	1 hour ago
PacienteMicroService.Api	6ff0a9fc5140	pacientemicroserviceapi:de	32786:8080 Show all ports (2)	0%	1 second ago
PersonalMicroService.Api	f37a6a735d17	personalmicroserviceapi:de	32782:8080 Show all ports (2)	0.05%	42 minutes ago
ApiGateway	5921d85da16c	apigateway:dev	32784:8080 Show all ports (2)	0.09%	36 minutes ago

Figura 33. Orquestación de Microservicios

1. **CatalogoMicroService.Api**: El servicio relacionado a los catálogos se levantó correctamente en el puerto configurado para la comunicación con los demás contenedores.
2. **CitaMicroService.Api**: Este servicio también fue iniciado correctamente y estaba disponible en el puerto correspondiente para realizar operaciones relacionadas con la gestión de citas.
3. **PacienteMicroService.Api**: Este contenedor también fue iniciado y configurado correctamente para manejar solicitudes de pacientes.
4. **PersonalMicroService.Api**: El servicio también quedó operativo para las pruebas lo que permite la interacción relacionada con el personal.
5. **ApiGateway**: Actua como el punto central de enrutamiento para coordinar todas las solicitudes a los diferentes microservicios. Fue configurado en el puerto 32784:8080, confirmando su disponibilidad para gestionar las comunicaciones entre los servicios y los clientes.

Cada contenedor fue verificado mediante los comandos de Docker para comprobar que estuviera correctamente iniciado, con los puertos asignados y sin errores en los logs iniciales.

Para la verificación se incluyó la revisión de los estados individuales de cada microservicio y del API Gateway confirmando que estuvieran en estado activo (indicador running). También se comprobó que los puertos configurados fueran accesibles desde el entorno local para que todas las solicitudes entrantes fueran correctamente redirigidas al microservicio correspondiente.

Pruebas de seguridad

Como parte de la metodología de pruebas se implementaron configuraciones específicas en el código fuente del API Gateway para garantizar la seguridad en la comunicación entre los microservicios. Para la herramienta de Swagger de cada microservicio se añadió soporte para encabezados personalizados en las solicitudes HTTPS utilizando JWT (JSON Web Token) para la autenticación y HMAC (Hash-based Message Authentication Code) para la validación de la integridad de los mensajes.

```
c.AddSecurityDefinition("Bearer", new OpenApiSecurityScheme
{
    Description = "Ingrese el token JWT en el formato 'Bearer {token}'",
    Name = "Authorization",
    In = ParameterLocation.Header,
    Type = SecuritySchemeType.ApiKey,
    Scheme = "Bearer"
});

c.AddSecurityDefinition("X-Signature", new OpenApiSecurityScheme
{
    Description = "Ingrese la firma HMAC en el encabezado X-Signature",
    Name = "X-Signature",
    In = ParameterLocation.Header,
    Type = SecuritySchemeType.ApiKey
});

c.AddSecurityRequirement(new OpenApiSecurityRequirement
```

Figura 34. Código de Ingreso de Encabezados

El código representa definiciones de seguridad utilizando AddSecurityDefinition en Swagger lo que permite que los usuarios especifiquen los valores requeridos directamente. La configuración define:

- **Bearer Token**: Se utiliza el encabezado Authorization para proporcionar el token JWT que autentica la solicitud.
- **HMAC Signature**: Se utiliza el encabezado X-Signature para validar la firma HMAC y garantizar la integridad de los datos enviados.

También se implementó una función para generar firmas HMAC. Este código utiliza un algoritmo SHA256 y una clave secreta compartida entre los servicios para crear una firma única basada en el contenido del token JWT.

```
// Método para generar la firma HMAC
1 reference
private string GenerateHMAC(string data, string secret)
{
    using (var hmac = new HMACSHA256(Encoding.UTF8.GetBytes(secret)))
    {
        var hash = hmac.ComputeHash(Encoding.UTF8.GetBytes(data));
        return BitConverter.ToString(hash).Replace("-", "").ToLower();
    }
}
```

Figura 35. Contenedor de Generación de Firmas

En una prueba realizada se usó un token JWT de ejemplo y una clave secreta para generar una firma HMAC la cual fue incluida en las solicitudes a través del encabezado X-Signature

para obtener la autorización de consulta del API Gateway como indica la Figura 36. Autorización con JWT.

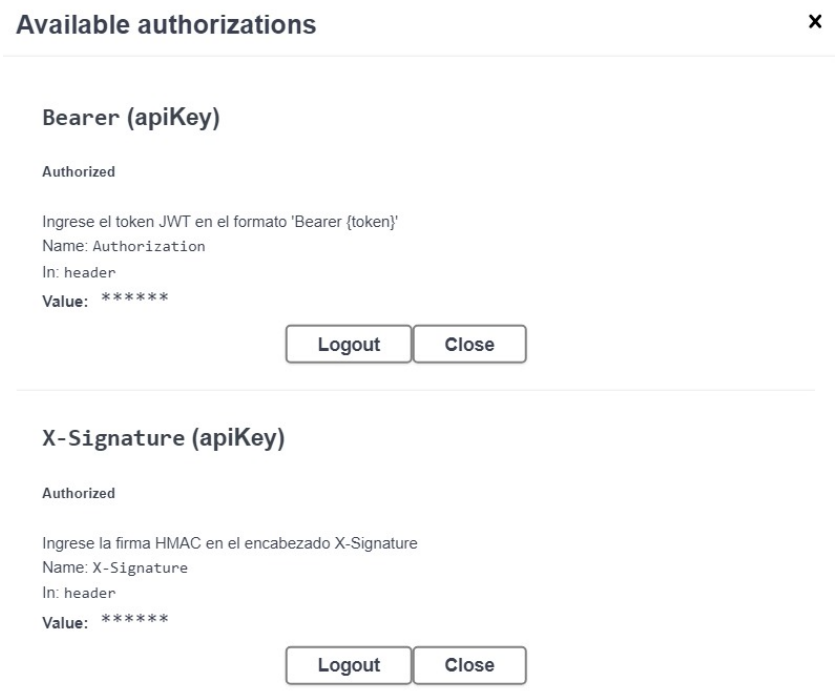


Figura 36. Autorización con JWT

En la Figura 37. Prueba de Autenticación se observa que se permitió la autenticación mediante los encabezados seguros generados al momento de la consulta hacia uno de los endpoints de los microservicios, fortaleciendo la seguridad del sistema y asegurando su correcto funcionamiento bajo escenarios diversos.

Curl

```
curl -X 'GET' \
  'https://localhost:5001/api/Catalogo' \
  -H 'accept: text/plain' \
  -H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwiYWRtaW4iOnRydWV9.nnn2HhXU-5NkfyGoDgiDlo1slO62Y2SE1EXBOzSEvEQ' \
  -H 'X-Signature: f8444e550da5018073e64c7b0b1651c88d7e7c65d1423943da0036c88c19359b'
```

Request URL

```
https://localhost:5001/api/Catalogo
```

Server response

Code	Details
200	Response body


```
[
  {
    "idCatalogo": 1,
    "idCatalogotipo": 1,
    "descripcionCat": "ESTUDIANTE                                    ",
    "operacionCat": null,
    "activoCat": true
  },
  {
    "idCatalogo": 3,
    "idCatalogotipo": 1,
    "descripcionCat": "ADMINISTRATIVO                                ",
    "operacionCat": null,
    "activoCat": true
  },
  {
    "idCatalogo": 5,
    "idCatalogotipo": 1,
    "descripcionCat": "OTRO                                          ",
```


Figura 37. Prueba de Autenticación

En el caso de que no se tenga autorización o que validación de JWT y HMAC sea incorrecta no permitirá el paso de información del microservicio y proporciona igualmente mensajes de error como indica la Figura 38. Mensaje de Error de Autenticación Fallida.

Curl

```
curl -X 'GET' \
  'https://localhost:5001/api/Catalogo' \
  -H 'accept: text/plain'
```

Request URL

```
https://localhost:5001/api/Catalogo
```

Server response

Code	Details
400 *Undocumented*	Error: response status is 400

Response headers

```
date: Sat,11 Jan 2025 02:12:53 GMT
server: Kestrel
```

Responses

Code	Description
200	Success

Figura 38. Mensaje de Error de Autenticación Fallida

Pruebas Unitarias

Se utilizó la herramienta de Swagger para validar los servicios expuestos por la API del monolito Agendamiento de Citas Médicas y corroborar la correcta implementación del contenedor con Docker y Docker-Compose como lo muestra la Figura 39. Documentación de Endpoint en Swagger.

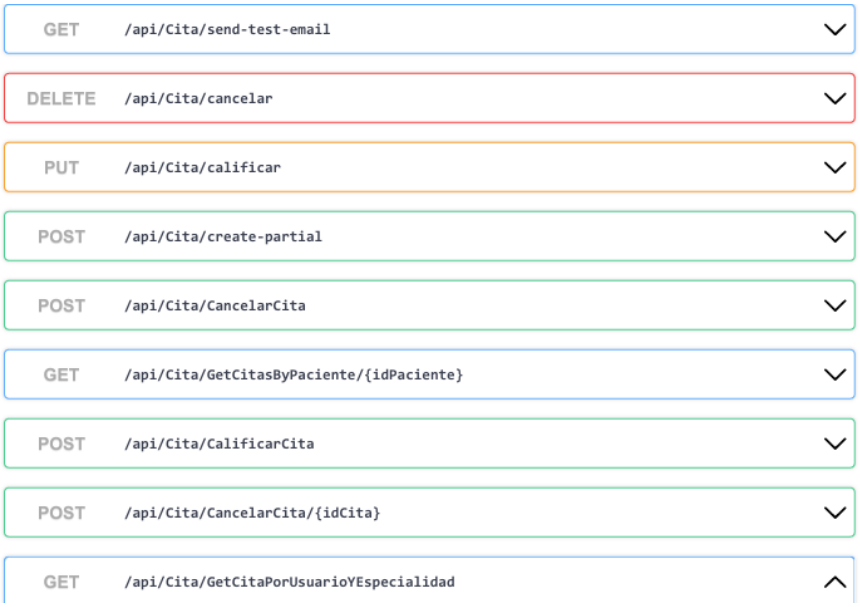


Figura 39. Documentación de Endpoint en Swagger

Los endpoints probados incluyen servicios de catálogos, citas, estados de citas, pacientes, personal y horarios. En la Figura 40. Respuesta de endpoint gate se puede observar que el endpoint /api/Catalogo devolvió resultados esperados con datos bien formateados indicando el correcto funcionamiento del proyecto con la implementación de Contenedores.

Figura 40. Respuesta de endpoint gate

El método GET en este caso también verificó parámetros adicionales como paginación y ordenación.

Las pruebas hechas con Swagger hacia el monolito ubicadas en el Anexo VII reflejan que el sistema está utilizando correctamente los estándares desarrollados como el uso de encabezados Authorization para la autenticación basada en JWT lo que nos asegura que el acceso a los endpoints está restringido a usuarios autenticados.

Las pruebas unitarias para cada microservicio creado se lo realizo en Postman utilizando su característica de Tests, que permite escribir scripts en JavaScript para validar la funcionalidad de los endpoints. Estas pruebas se ejecutan automáticamente después de que una solicitud se completa y son útiles para verificar respuestas específicas, por lo que en la Figura 41. Colección de Microservicios en Postman se puede visualizar las colecciones creadas de cada uno de los microservicios, se tiene más a detalle en el Anexo VI.

Figura 41. Colección de Microservicios en Postman

Para cada endpoint se realizó un script de prueba el cual validará aspectos específicos de la respuesta que son considerados como criterios de éxito, como los códigos de estado, contenido de la respuesta, encabezados y más como lo muestra la Figura 42. Prueba Unitaria en Postman.

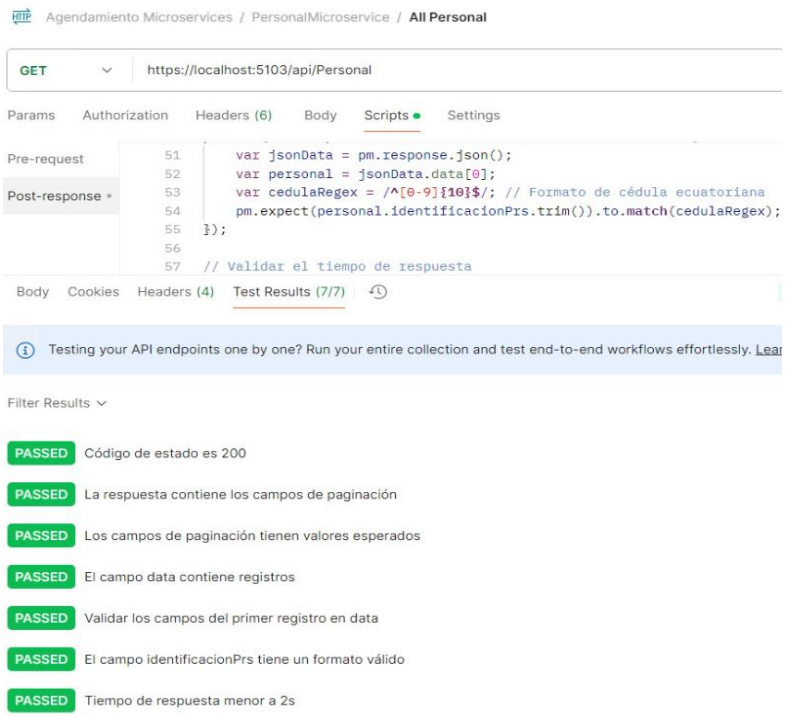


Figura 42. Prueba Unitaria en Postman

3 RESULTADOS, CONCLUSIONES Y RECOMENDACIONES

3.1 Resultados

Como parte de los resultados en las pruebas unitarias de cada microservicio se utilizó Newman para realizar un seguimiento estructurado y generar reportes automatizados al permitir ejecutar colecciones de pruebas diseñadas en Postman directamente desde la línea de comandos, más detalle de este reporte en el Anexo VIII.

Como indica la Figura 43. Solicitudes de Microservicios en Newman se realizaron diferentes solicitudes a cada uno de los microservicios con los scripts integrados para comprobar su funcionamiento.

Figura 43. Solicitudes de Microservicios en Newman

Newman ejecutó las colecciones de Postman asociadas a cada microservicio y el reporte refleja el estado de las solicitudes procesadas.

Catalogo Microservice:

Prueba: Validación del endpoint GET /api/Catalogo

- **Descripción**: Se probó el endpoint GET /api/Catalogo para obtener datos del catálogo utilizando encabezados que aceptan texto plano.
- **Criterios de éxito:**
    1. El código de estado debe ser 200 OK.
    2. La respuesta no debe estar vacía.
    3. El tiempo de respuesta debe ser menor a 2s.

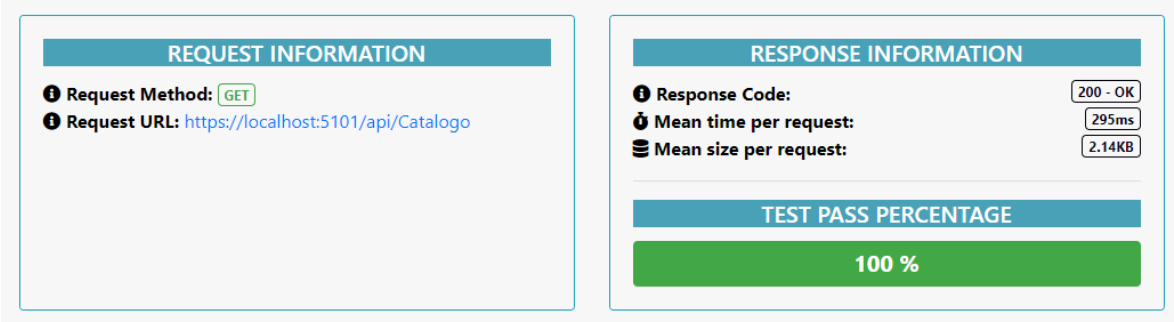
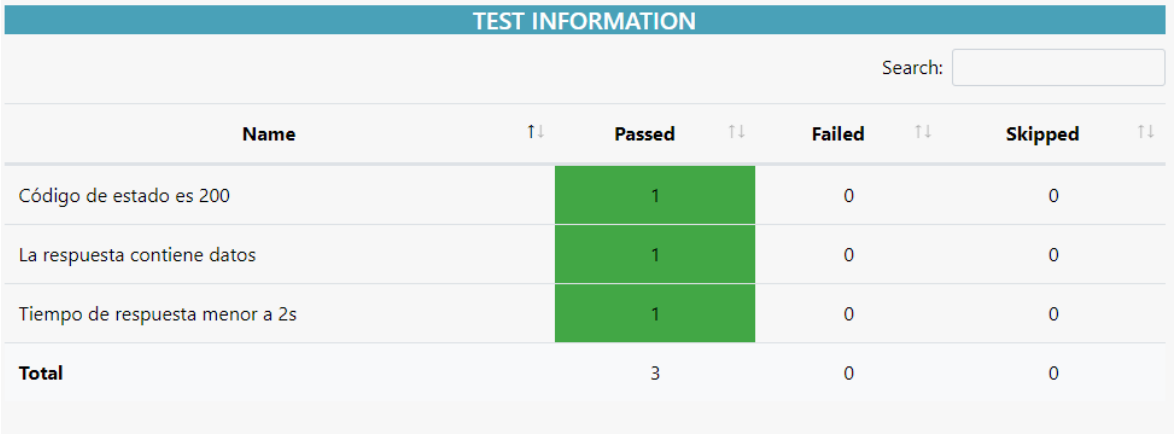


Figura 44. Validación del endpoint GET /api/Catalogo

Resultados:



TEST INFORMATION

Search:

Name	Passed	Failed	Skipped
Código de estado es 200	1	0	0
La respuesta contiene datos	1	0	0
Tiempo de respuesta menor a 2s	1	0	0
Total	3	0	0

Figura 45. Resultados del endpoint GET /api/Catalogo

Prueba: Validación del endpoint GET /api/Catalogo/1

- **Descripción:** Se probó el endpoint GET /api/Catalogo/1 para recuperar los datos de un catálogo especifico.
- **Criterios de éxito:**
    1. El código de estado debe ser 200 OK.
    2. La respuesta debe contener un valor valido.
    3. El tiempo de respuesta debe ser inferior a 500 ms.

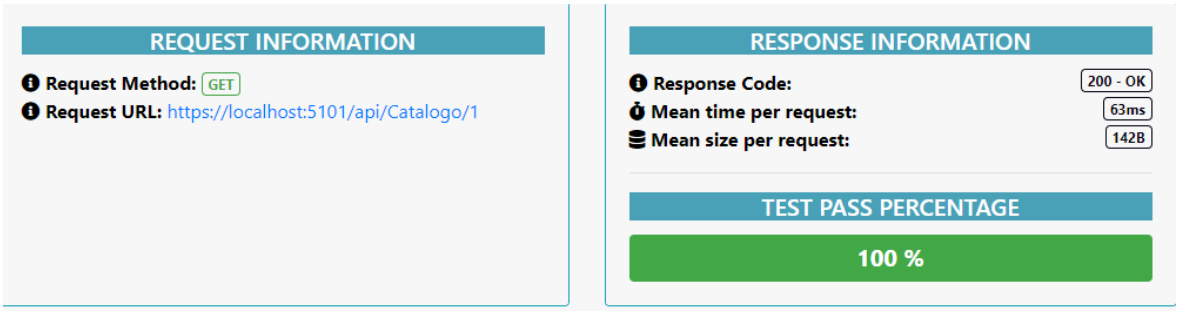
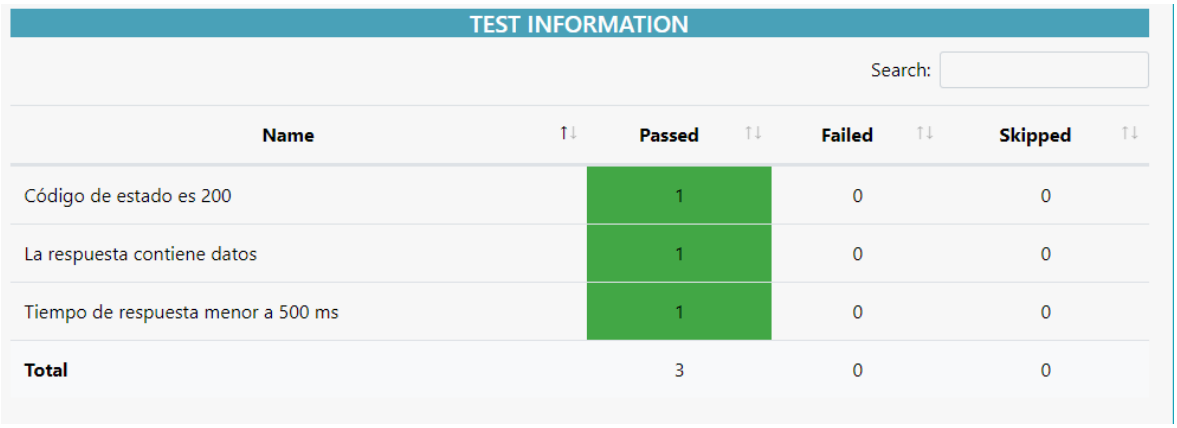


Figura 46. Validación del endpoint GET /api/Catalogo/1

Resultados:



Name	Passed	Failed	Skipped
Código de estado es 200	1	0	0
La respuesta contiene datos	1	0	0
Tiempo de respuesta menor a 500 ms	1	0	0
Total	3	0	0

Figura 47.Resultados del endpoint GET /api/Catalogo/1

Prueba: Validación del endpoint GET /api/CatalogoHoras

- **Descripción:** Se probó el endpoint GET /api/CatalogoHoras para comprobar la funcionalidad de los datos paginados de las horas disponibles en el catálogo.
- **Criterios de éxito:**
    1. El código de estado debe ser 200 OK.

2. La respuesta debe contener los campos: pageIndex, pageSize, count, pageCount, y data.
3. Los valores de los campos deben ser coherentes con una respuesta.
4. El tiempo de respuesta debe ser inferior a 2s.

Figura 48. Validación del endpoint GET /api/CatalogoHoras

Resultados:

TEST INFORMATION

Search:

Name	Passed	Failed	Skipped
Código de estado es 200	1	0	0
La respuesta contiene los campos básicos	1	0	0
Tiempo de respuesta menor a 2 s	1	0	0
Validar contenido para respuesta vacía	1	0	0
Total	4	0	0

Figura 49.Resultados del endpoint GET /api/CatalogoHoras

Cita Microservice:

Prueba: Validación del endpoint GET /api/Cita

- **Descripción:** Se probó el endpoint GET /api/Cita para validar que devuelve una lista de citas con la estructura esperada.
- **Criterios de éxito:**
    1. El código de estado debe ser 200 OK.

2. La respuesta debe permitir visualizar la cita con la estructura esperada.
3. Cada objeto debe contener los campos: idCita, idPersonal, idPaciente, idCatalogohoras, idTipocita, asistencia, fecha y recorridos.
4. El tiempo de respuesta debe ser inferior a 2s.

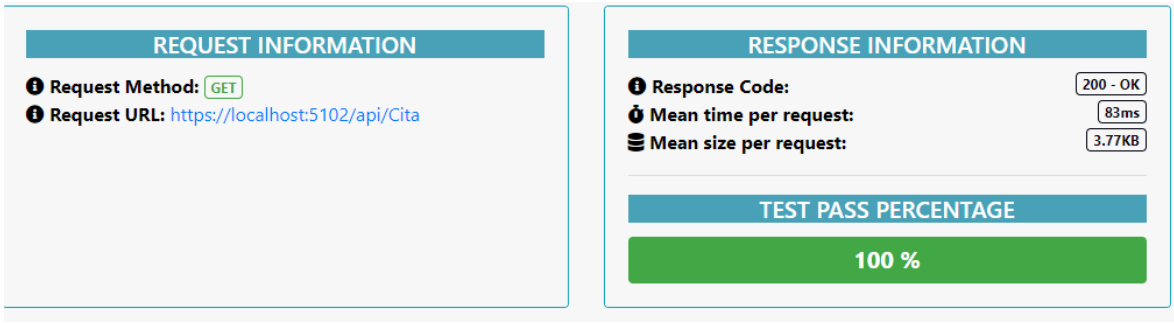


Figura 50. Validación del endpoint GET /api/Cita

Resultados:

Search:

Name	Passed	Failed	Skipped
Código de estado es 200	1	0	0
El array no está vacío	1	0	0
La respuesta es un array	1	0	0
Tiempo de respuesta menor a 2s	1	0	0
Validar los campos del primer objeto	1	0	0
Total	5	0	0

Figura 51.Resultados del endpoint GET /api/Cita

Prueba: Validación del endpoint GET /api/CitaEstadoActual

- **Descripción:** Se probó el endpoint GET /api/CitaEstadoActual para validar la funcionalidad de recuperación de estados actuales de citas.
- **Criterios de éxito:**
    1. El código de estado debe ser 200 OK.
    2. La respuesta debe contener los campos de paginación (pageIndex, pageSize, count, pageCount) y un campo data que es un array.

3. Cada objeto en data debe incluir los campos: idCita, idCatalogoHoras, fecha, idPaciente, idEstadoActual, horaInicio, horaFin, fechaCambio, nombreEstado, idPersonal, nombreEspecialidad.

4. El tiempo de respuesta debe ser inferior a 2s.

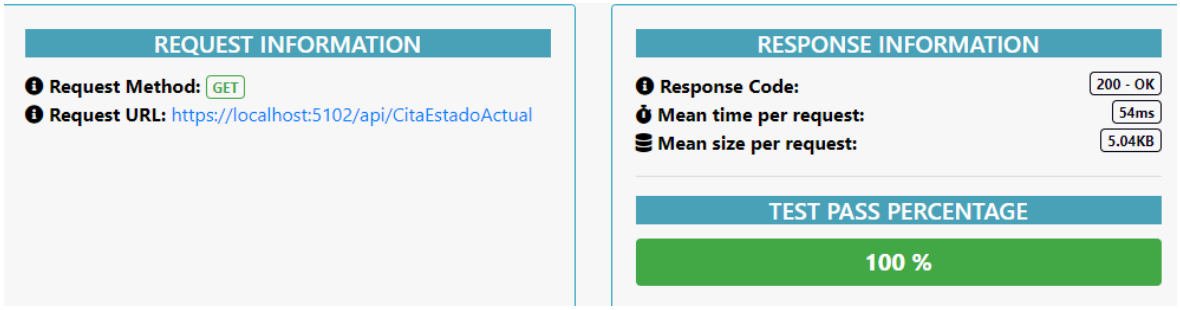
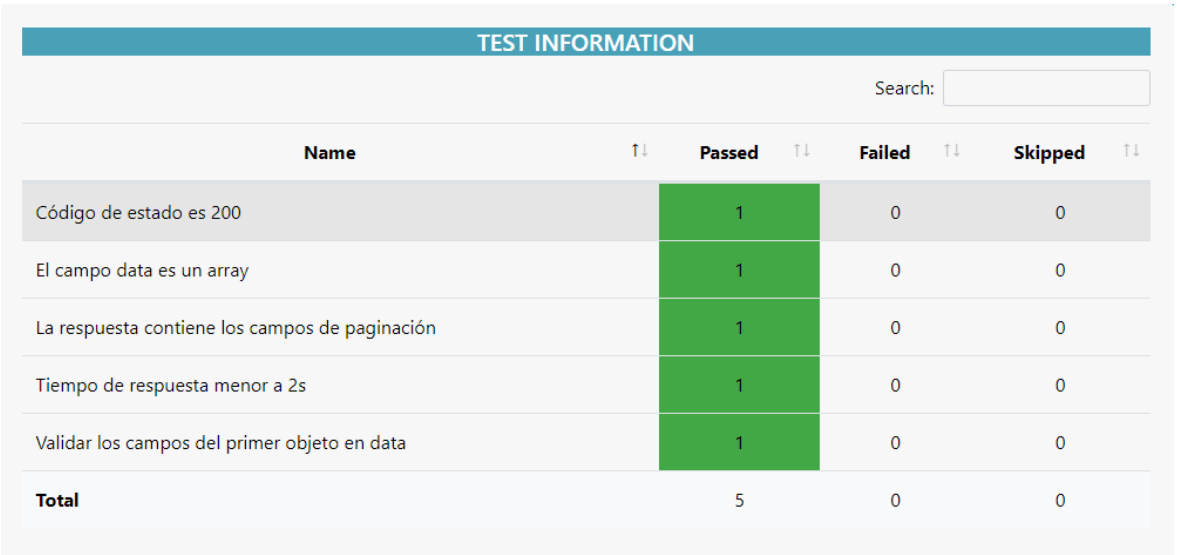


Figura 52. Validación del endpoint GET /api/CitaEstadoActual

Resultados:



TEST INFORMATION

Search:

Name	Passed	Failed	Skipped
Código de estado es 200	1	0	0
El campo data es un array	1	0	0
La respuesta contiene los campos de paginación	1	0	0
Tiempo de respuesta menor a 2s	1	0	0
Validar los campos del primer objeto en data	1	0	0
Total	5	0	0

Figura 53.Resultados del endpoint GET /api/CitaEstadoActual

Prueba: Validación del endpoint GET /api/Horario

- **Descripción:** Se probó el endpoint GET /api/Horario con los parámetros IdEspecialidad=1 e IdCatalogoHoras=1 para validar la funcionalidad de recuperación de horarios disponibles de acuerdo a una especialidad y a un Catálogo seleccionado.

- **Criterios de éxito:**

  1. El código de estado debe ser 200 OK.

2. La respuesta debe contener los horarios de acuerdo con la especialidad y catálogo.
3. Cada objeto en el array debe incluir los campos: idEspecialidad, idCatalogohoras, fecha, disponibilidad, horaInicio, y horaFin.
4. El tiempo de respuesta debe ser inferior a 2s.

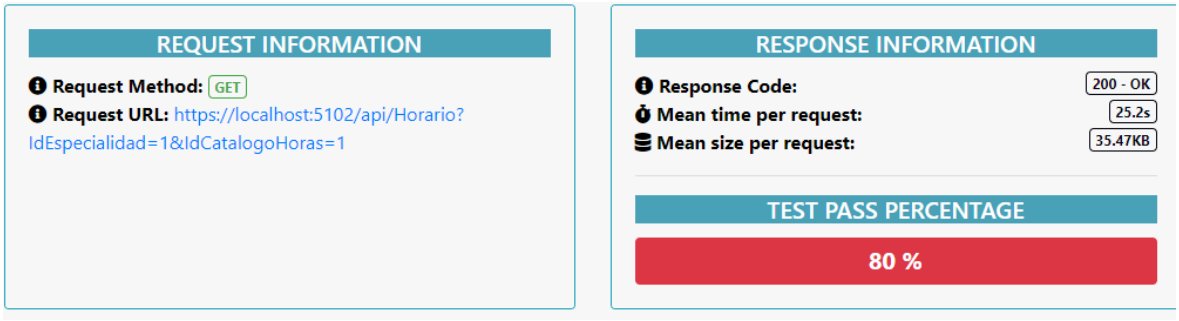
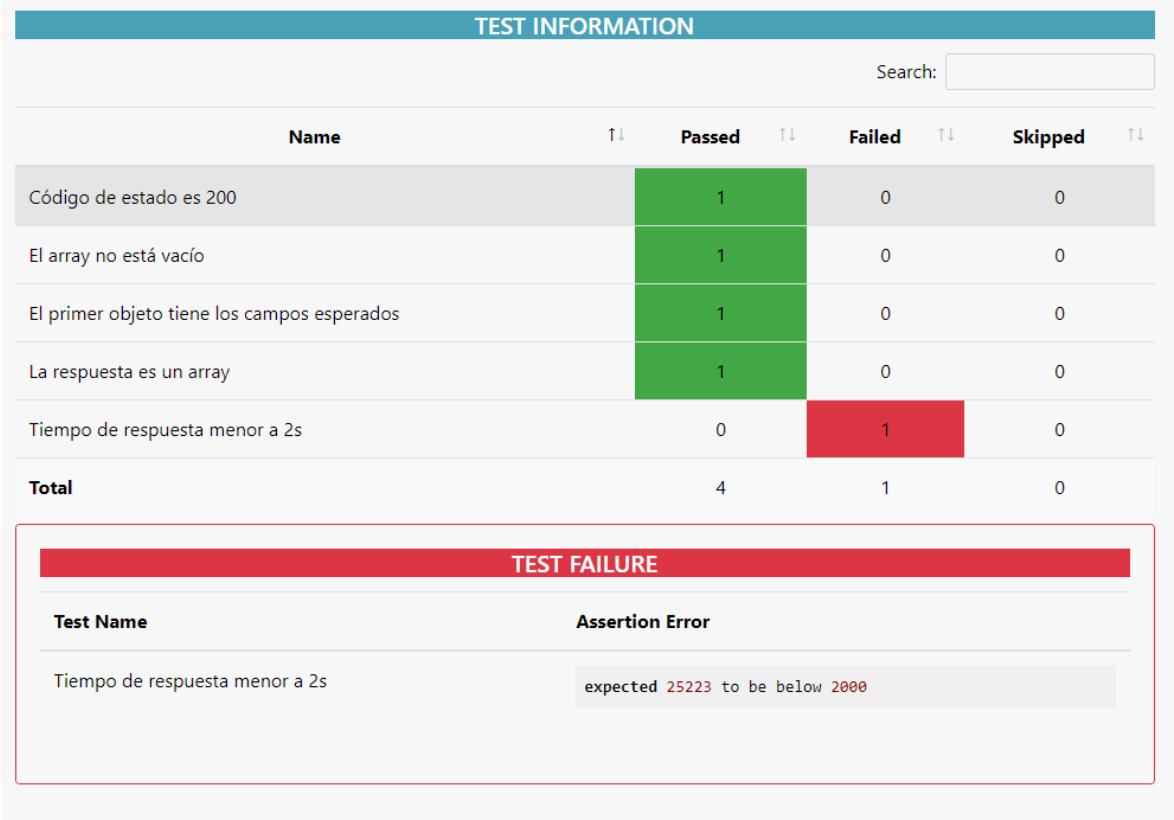


Figura 54. Validación del endpoint GET /api/Horario

Resultados:



TEST INFORMATION

Search:

Name	Passed	Failed	Skipped
Código de estado es 200	1	0	0
El array no está vacío	1	0	0
El primer objeto tiene los campos esperados	1	0	0
La respuesta es un array	1	0	0
Tiempo de respuesta menor a 2s	0	1	0
Total	4	1	0

TEST FAILURE

Test Name	Assertion Error
Tiempo de respuesta menor a 2s	`expected 25223 to be below 2000`

Figura 55.Resultados del endpoint GET /api/Horario

Prueba fallida:

Tiempo de respuesta menor a 2s: Esta prueba falló porque el tiempo de respuesta fue de 25223 ms (25.2 segundos), excediendo el límite establecido de 2 segundos. La razón se detalla en la sección Test Failure como un error de aserción: el tiempo real supera el esperado.

El tiempo excesivo de respuesta se atribuye a la arquitectura de la base de datos. Actualmente como requisito de la DGIP todos los microservicios comparten una base de datos unificada. Esto genera algunos problemas que afectan el rendimiento:

- **Consultas complejas**: Al ser una base de datos centralizada las consultas que involucran múltiples tablas o filtros avanzados tardan más en ejecutarse.
- **Carga compartida**: La base de datos está siendo utilizada simultáneamente por todos los microservicios lo que incrementa la latencia cuando hay múltiples solicitudes.
- **Falta de particionamiento**: La ausencia de separación de datos por microservicio impide la optimización de consultas para casos específicos.

Solución:

Implementar bases de datos independientes para cada microservicio reduciendo la competencia por recursos.

Paciente Microservice:

Prueba: Validación del endpoint GET /api/Paciente

- **Descripción:** Se probó el endpoint GET /api/Paciente con los parámetros pageNumber=1 y pageSize=50 para validar la funcionalidad de recuperación de datos de pacientes con paginación.
- **Criterios de éxito:**
    1. El código de estado debe ser 200 OK.
    2. La respuesta debe contener a los pacientes.
    3. Cada objeto debe contener los campos: idPaciente, idTipopaciente, identificacionPac, apellido1Pac, apellido2Pac, nombre1Pac, nombre2Pac, estado, correo, citasIdPersonal, citasIdPersonalderivacion, rol.

4. El campo identificacionPac debe tener un formato válido (10 dígitos).

5. El tiempo de respuesta debe ser inferior a 2s.

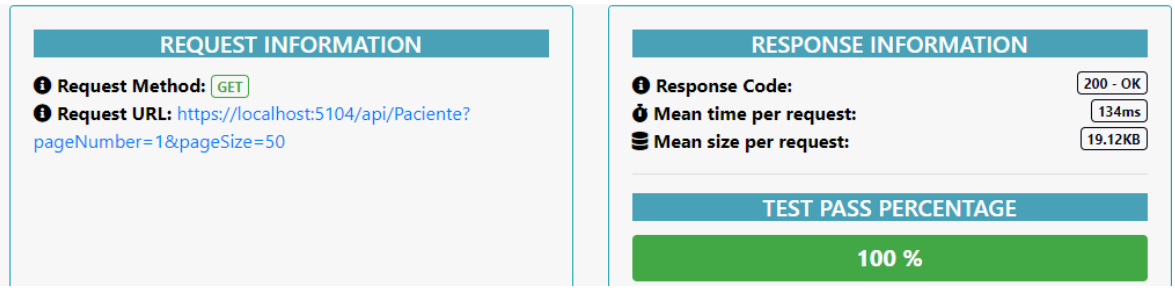
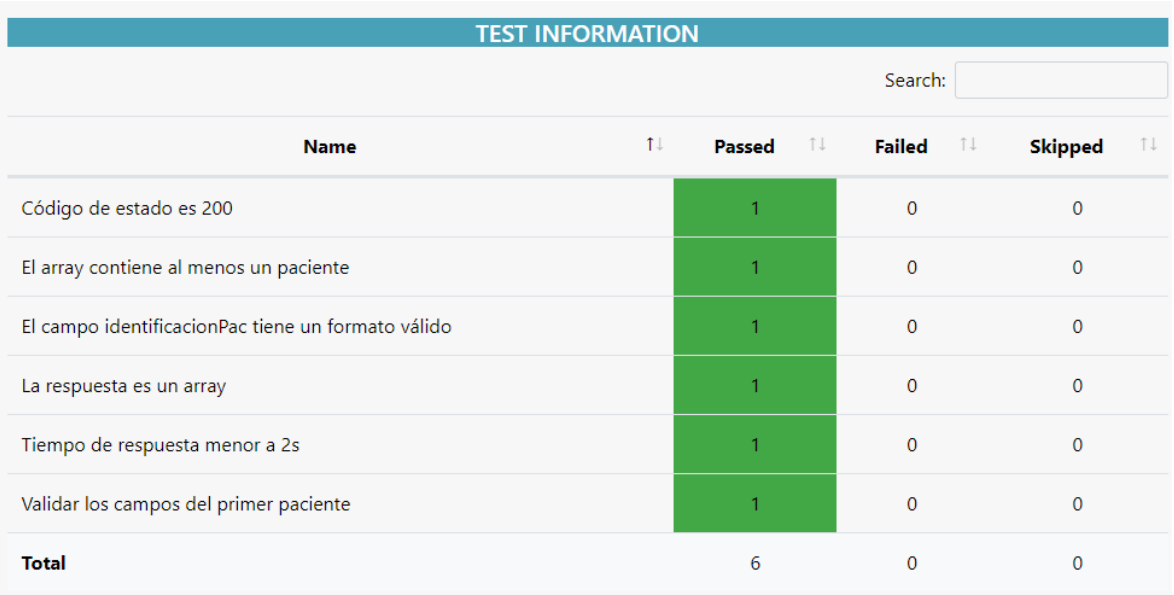


Figura 56. Validación del endpoint GET /api/Paciente

Resultados:



TEST INFORMATION

Search:

Name	Passed	Failed	Skipped
Código de estado es 200	1	0	0
El array contiene al menos un paciente	1	0	0
El campo identificacionPac tiene un formato válido	1	0	0
La respuesta es un array	1	0	0
Tiempo de respuesta menor a 2s	1	0	0
Validar los campos del primer paciente	1	0	0
Total	6	0	0

Figura 57.Resultados del endpoint GET /api/Paciente

Prueba: Validación del endpoint GET /api/ListaPaciente

- **Descripción:** Se probó el endpoint GET /api/ListaPaciente para validar el manejo de la paginación y la respuesta en casos donde no hay datos disponibles.
- **Criterios de éxito:**
    1. El código de estado debe ser 200 OK.
    2. La respuesta debe incluir los campos de paginación: pageIndex, pageSize, count, pageCount y data.

3. Los valores de los campos deben reflejar un estado vacío (count y pageCount igual a 0).
4. El campo data debe ser un array vacío.
5. El tiempo de respuesta debe ser inferior a 2s.

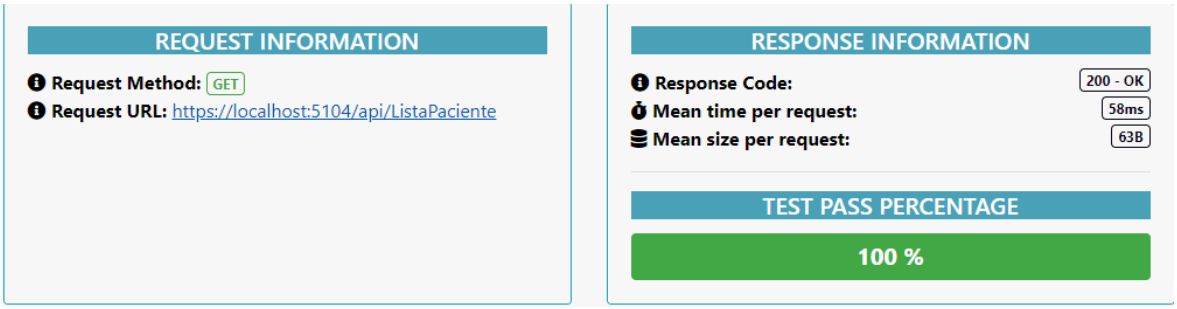
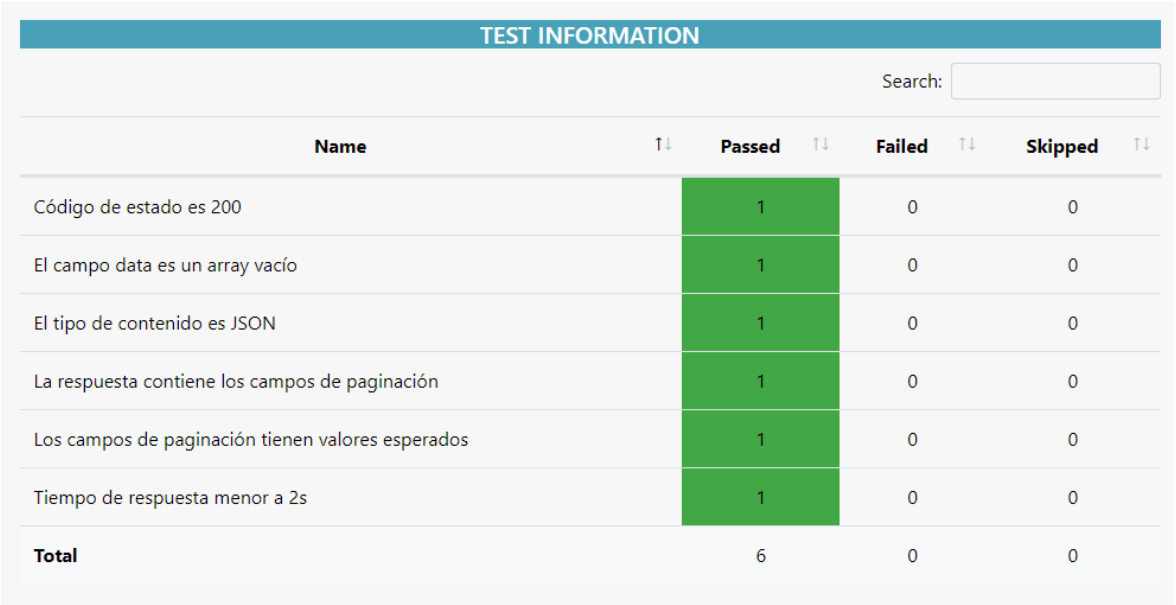


Figura 58. Validación del endpoint GET /api/ListaPaciente

Resultados:



TEST INFORMATION

Search:

Name	Passed	Failed	Skipped
Código de estado es 200	1	0	0
El campo data es un array vacío	1	0	0
El tipo de contenido es JSON	1	0	0
La respuesta contiene los campos de paginación	1	0	0
Los campos de paginación tienen valores esperados	1	0	0
Tiempo de respuesta menor a 2s	1	0	0
Total	6	0	0

Figura 59.Resultados del endpoint GET /api/ListaPaciente

Personal Microservice:

Prueba: Validación del endpoint GET /api/Personal

- **Descripción:** Se probó el endpoint GET /api/Personal para validar la recuperación de registros del personal, incluyendo la funcionalidad de paginación.
- **Criterios de éxito:**
    1. El código de estado debe ser 200 OK.
    2. La respuesta debe incluir los campos de paginación: pageIndex, pageSize, count, pageCount y data.
    3. El campo data debe contener un array con objetos que incluyen los campos: idPersonal, idTipoespecialidad, apellido1Prs, apellido2Prs, nombre1Prs, nombre2Prs, telefonosPrs, identificacionPrs, estado, direccionPrs, nacionalidadPrs, email.
    4. El campo identificacionPrs debe tener un formato válido (10 dígitos).
    5. El tiempo de respuesta debe ser inferior a 2s.

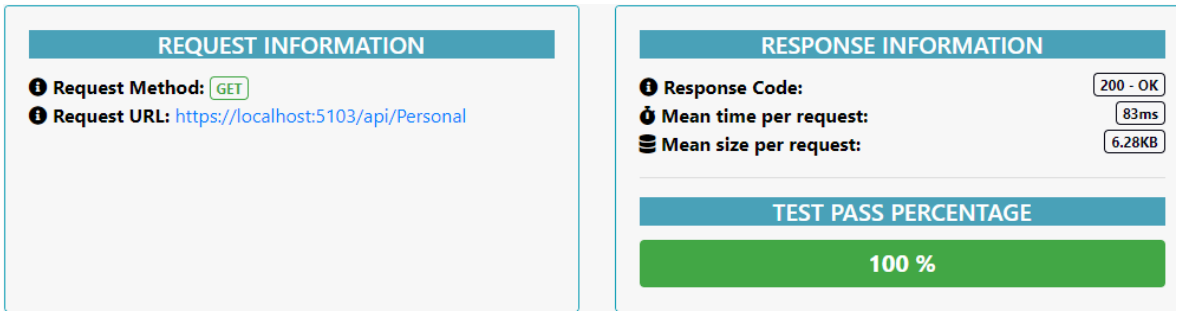


Figura 60. Validación del endpoint GET /api/Personal

Resultados:

Name	Passed	Failed	Skipped
Código de estado es 200	1	0	0
El campo data contiene registros	1	0	0
El campo identificacionPrs tiene un formato válido	1	0	0
La respuesta contiene los campos de paginación	1	0	0
Los campos de paginación tienen valores esperados	1	0	0
Tiempo de respuesta menor a 2s	1	0	0
Validar los campos del primer registro en data	1	0	0
Total	7	0	0

Figura 61.Resultados del endpoint GET /api/Personal

Prueba: Validación del endpoint GET /api/Especialidad

- **Descripción:** Se probó el endpoint GET /api/Especialidad para validar la recuperación de datos de especialidades médicas.
- **Criterios de éxito:**
    1. El código de estado debe ser 200 OK.
    2. La respuesta debe contener especialidades.
    3. Cada objeto debe incluir los campos: idEspecialidad, idTipoespecialidad, nombreEspecialidad, duracion, horaInicio, horaFin, listaNegra, listaEspera, maximoInasistencias, estado.
    4. El tiempo de respuesta debe ser inferior a 2s.

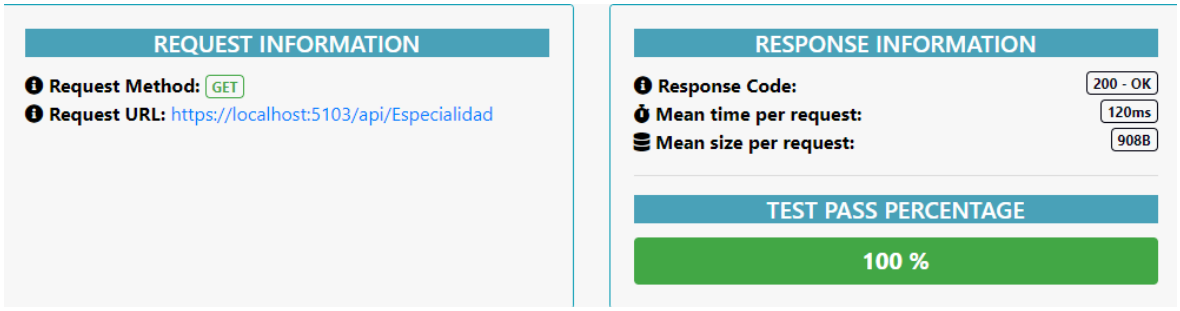
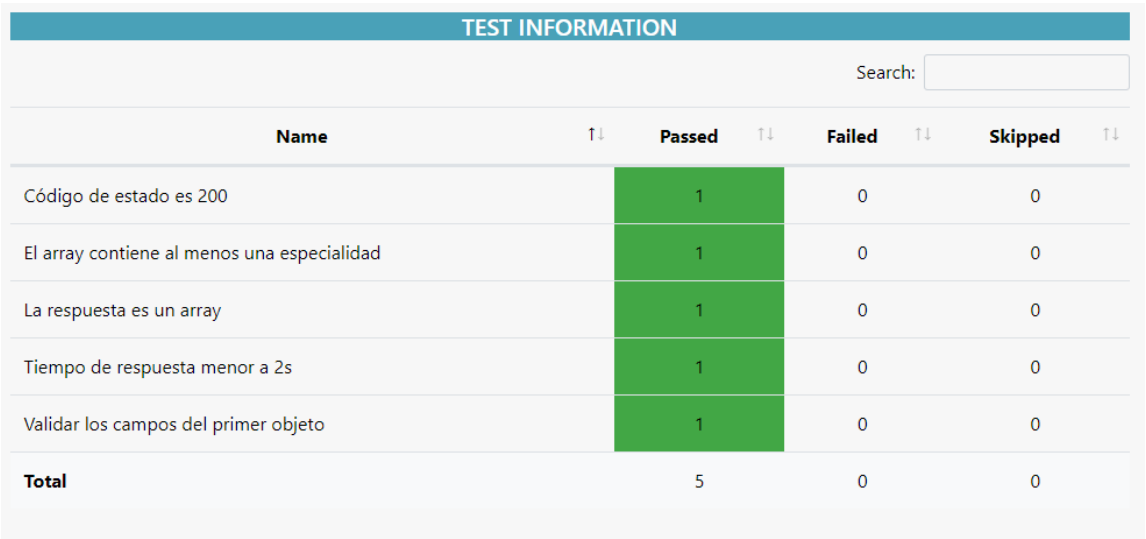


Figura 62. Validación del endpoint GET /api/Especialidad

Resultados:



TEST INFORMATION

Search:

Name	Passed	Failed	Skipped
Código de estado es 200	1	0	0
El array contiene al menos una especialidad	1	0	0
La respuesta es un array	1	0	0
Tiempo de respuesta menor a 2s	1	0	0
Validar los campos del primer objeto	1	0	0
Total	5	0	0

Figura 63.Resultados del endpoint GET /api/Especialidad

Prueba: Validación del endpoint GET /api/Permisos/GetPermisosByPersonal/3906

- **Descripción:** Se probó el Endpoint:

  GET /api/Permisos/GetPermisosByPersonal/3906 para validar la recuperación de permisos asociados al personal con ID 3906.

- **Criterios de éxito:**

  1. El código de estado debe ser 200 OK.
  2. La respuesta debe ser un array que contenga datos del permiso del personal.
  3. Cada objeto debe incluir los campos: idPermisoEsp, idPersonal, horaInicioPermiso, horaFinPermiso, fechaInicioPermiso, fechaFinPermiso, motivo, estado.
  4. El campo estado debe ser un booleano.
  5. El tiempo de respuesta debe ser inferior a 2s.

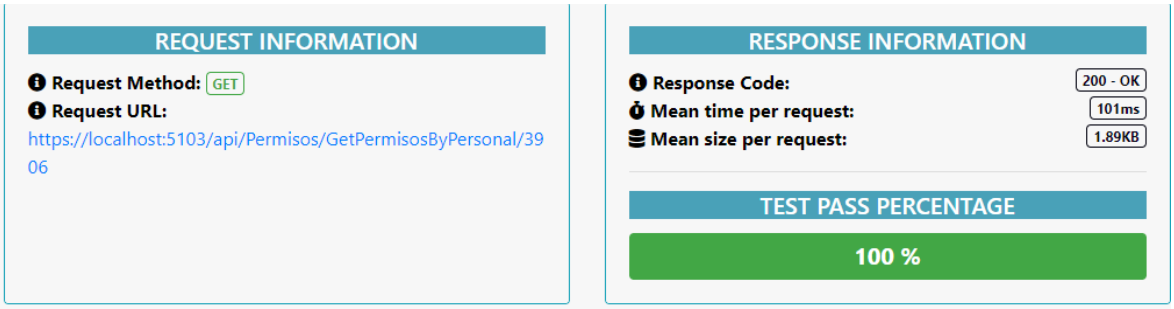


Figura 64. Validación del endpoint GET /api/Permisos/GetPermisosByPersonal/3906

Resultados:

TEST INFORMATION			
Search:			
Name	**Passed**	**Failed**	**Skipped**
Código de estado es 200	1	0	0
El array contiene al menos un permiso	1	0	0
El campo estado es un booleano	1	0	0
La respuesta es un array	1	0	0
Tiempo de respuesta menor a 2s	1	0	0
Validar los campos del primer permiso	1	0	0
Total	6	0	0

Figura 65.Resultados del endpoint GET /api/Permisos/GetPermisosByPersonal/3906

Los resultados registraron respuestas HTTPS con código 200 lo que refleja un funcionamiento exitoso de los servicios. En conjunto estas pruebas proporcionan evidencia clara y auditable de la funcionalidad de los microservicios cumpliendo con los requisitos técnicos del proyecto y de las normativas establecidas por la DGIP.

3.2 Conclusiones

Se desarrolló un Core de Servicios Web utilizando .NET, cumpliendo con los principios y mejores prácticas de una arquitectura RESTful para estructurar servicios eficientes y bien definidos con la disponibilidad de poder interoperar entre los sistemas informáticos del ecosistema académico.

Se aseguró mecanismos que fomentan el modularidad del sistema permitiendo el desarrollo y despliegue independiente de componentes. También se integró principios escalables que facilitan la incorporación de futuras aplicaciones académicas sin comprometer el rendimiento del sistema.

Se diseñó e integró medidas de seguridad como autenticación y autorización robustas que garantizaron la protección de los datos sensibles y los servicios del sistema, cumpliendo con los estándares requeridos para aplicaciones académicas.

Se implementó protocolos de transmisión segura mediante el uso de HTTPS para asegurar la protección contra posibles vulnerabilidades durante la comunicación entre servicios y usuarios.

La migración de un monolito a una arquitectura basada en microservicios para crear un núcleo de servicios que puedan ser reutilizados en un futuro por la FIS proporcionó una visión profunda sobre cómo estructurar y evolucionar un sistema monolítico a un sistema más robusto y escalable. En este proceso se logró desacoplar las diferentes funcionalidades del sistema en contextos delimitados que representan unidades autónomas del negocio y se pudo estudiar en el proceso los distintos eventos de integración consumo de datos mediante manejadores de eventos que aseguraron una comunicación desacoplada entre los servicios.

Para la FIS esta transformación representa un modelo práctico de cómo aplicar principios teóricos en un contexto real. La adopción de microservicios con DDD refuerza la idea de diseñar sistemas centrados en los dominios de negocio para fomentar una comprensión más profunda de las necesidades específicas de la facultad.

Este modelo de núcleo de servicios desarrollado facilita la reutilización de cada uno de estos servicios en diferentes sistemas o módulos ya que cada microservicio expone su funcionalidad a través de APIs bien documentadas. La FIS en un futuro puede reutilizar el microservicio de gestión de citas médicas para otras áreas de servicio público que requieran agendamiento o incluso para integrarlo con aplicaciones móviles de usuarios finales.

La implementación de estándares definidos en la arquitectura de microservicios abre una puerta hacia un futuro más dinámico y eficiente para la FIS ya que mediante estas directrices se pueden desarrollar nuevas aplicaciones que funcionen como servicios lo cual significaría extender las capacidades de los sistemas mediante nuevas funcionalidades sin necesidad de afectar el rendimiento o funcionamiento original. Por ejemplo, agregar servicios de analítica o notificaciones en tiempo real se puede realizar creando nuevos microservicios que consuman los eventos ya existentes. Con esta forma de desarrollar aplicaciones se pueden aplicar incluso tecnologías emergentes como inteligencia artificial o big data integrándolas fácilmente en el ecosistema existente de la Facultad de sistemas.

En términos de sostenibilidad esta arquitectura prepara a la DGIP para un crecimiento a largo plazo. Al adoptar microservicios orquestados en Kubernetes los recursos computacionales pueden ser ajustados dinámicamente en función de la demanda para reducir costos y asegurando una experiencia óptima para los usuarios.

3.3 Recomendaciones

Se recomienda que el Core de Servicios Web desarrollado sea ampliado para incluir nuevas funcionalidades que respondan a las necesidades emergentes del Subdecanato y la comunidad académica.

Dado el entorno dinámico y los riesgos de ciberseguridad en constante evolución se aconseja realizar auditorías periódicas de las medidas de seguridad implementadas, incluyendo autenticación, autorización y transmisión de datos.

Se recomienda extender el uso de protocolos como HTTPS a todos los servicios relacionados para garantizar la seguridad en la comunicación. Esto incluye educar a los desarrolladores y responsables de tecnología en el uso de estándares de seguridad modernos y mantenerlos actualizados.

Para asegurar el crecimiento sostenible del ecosistema de aplicaciones académicas es importante que los futuros proyectos mantengan la modularidad y escalabilidad como principios clave. Se recomienda estandarizar las prácticas y lineamientos definidos durante este proyecto para garantizar una integración fluida y eficiente de nuevas aplicaciones.

Se recomienda que la FIS adopte una arquitectura basada en servicios como estándar para el desarrollo de proyectos futuros, ya que permite que los servicios puedan desplegarse y actualizarse de forma independiente capaces de adaptarse a los cambios dentro de la facultad fomentando una innovación constante en los sistemas de información.

Es crucial establecer un conjunto de estándares internos que definan las mejores prácticas para el desarrollo de microservicios. Como se recomienda el uso de un patrón para desarrollarlos, en este caso se usó Domain-Driven Design (DDD), sin embargo, para proyectos más pequeños se puede utilizar otro tipo de diseño para los servicios.

La implementación de APIs RESTful bien documentadas y la comunicación eficiente entre servicios mediante eventos hace que la integración sea fluida entre los servicios y brinde una calidad uniforme en los desarrollos de las aplicaciones de la FIS.

Es clave optimizar la infraestructura de bases de datos para evitar dependencias centralizadas. Durante el desarrollo se evidenció que la centralización en una única base de datos unificada puede causar tiempos de respuesta altos para consultas complejas. Se sugiere adoptar bases de datos distribuidas diseñadas específicamente para cada microservicio, lo que permite mejorar el rendimiento en general y que también incremente la resiliencia del sistema frente a posibles fallos.

4 REFERENCIAS BIBLIOGRÁFICAS

[1] F. Devin, “Web-Oriented Architecture - How to design a RESTFull API”, en *TORUS 1 - Toward an Open Resource Using Services: Cloud Computing for Environmental Data*, wiley, 2020, pp. 191–206. doi: 10.1002/9781119720492.ch14.

[2] I. Ahmad, E. Suwarni, R. I. Borman, Asmawati, F. Rossi, y Y. Jusman, “Implementation of RESTful API Web Services Architecture in Takeaway Application Development”, *2021 1st International Conference on Electronic and Electrical Engineering and Intelligent System, ICE3IS 2021*, pp. 132–137, 2021, doi: 10.1109/ICE3IS54102.2021.9649679.

[3] S. H. Toman, “Review of Web Service Technologies: REST over SOAP”, *Journal of Al-Qadisiyah for Computer Science and Mathematics*, vol. 12, núm. 4, nov. 2020, doi: 10.29304/jqcm.2020.12.4.715.

[4] O. Zanevych, “ADVANCING WEB DEVELOPMENT: A COMPARATIVE ANALYSIS OF MODERN FRAMEWORKS FOR REST AND GRAPHQL BACK-END SERVICES”, *Grail of Science*, núm. 37, pp. 216–228, mar. 2024, doi: 10.36074/GRAIL-OF-SCIENCE.15.03.2024.031.

[5] M. K. Kumar y V. Anirudh, “Basic Components of Web Services Architecture: A Survey”, *International Journal of Engineering Technology and Management Sciences*, vol. 4, núm. 4, pp. 26–29, jul. 2020, doi: 10.46647/ijetms.2020.v04i04.005.

[6] E. Haro, T. Guarda, A. O. Z. Peñaherrera, y G. N. Quiña, “Backend development for web applications, restful web services: Node.js vs spring boot[Desarrollo backend para aplicaciones web, servicios web restful: Node.js vs spring boot]”, *RISTI - Revista Iberica de Sistemas e Tecnologias de Informacao*, núm. E17, pp. 309–321, ene. 2019.

[7] O. Chaplia y H. Klym, “Designing a Node.js Project Architecture for RESTful Microservices”, *Proceedings of the IEEE International Conference on Intelligent Data Acquisition and Advanced Computing Systems: Technology and Applications, IDAACS*, pp. 808–811, 2023, doi: 10.1109/IDAACS58523.2023.10348681.

[8] A. A. Prayogi, M. Niswar, y M. Rijal, “Design and Implementation of REST API for Academic Information System”, *The 3rd EPI International Conference on Science and Engineering 2019 (EICSE2019)*, 2019, doi: 10.1088/1757-899X/875/1/012047.

[9] J. Heikkilä, “Implementing a RESTful API with .NET Core 2.1”, 2021.

[10] S. Kotstein y J. Bogner, “Which RESTful API Design Rules Are Important and How Do They Improve Software Quality? A Delphi Study with Industry Experts”, *Communications in Computer and*

Information Science, vol. 1429 CCIS, pp. 154–173, 2021, doi: 10.1007/978-3-030-87568-8_10/TABLES/4.

[11] I. Ahmad, E. Suwarni, R. I. Borman, Asmawati, F. Rossi, y Y. Jusman, “Implementation of RESTful API Web Services Architecture in Takeaway Application Development”, *2021 1st International Conference on Electronic and Electrical Engineering and Intelligent System, ICE3IS 2021*, pp. 132–137, 2021, doi: 10.1109/ICE3IS54102.2021.9649679.

[12] L. Jackson, “The Complete ASP.NET Core 3 API Tutorial: Hands-On Building, Testing, and Deploying”, *The Complete ASP.NET Core 3 API Tutorial: Hands-On Building, Testing, and Deploying*, pp. 1–461, ene. 2020, doi: 10.1007/978-1-4842-6255-9.

[13] Y. Romani, O. Tibermacine, y C. Tibermacine, “Towards Migrating Legacy Software Systems to Microservice-based Architectures: A Data-Centric Process for Microservice Identification”, *2022 IEEE 19th International Conference on Software Architecture Companion, ICSA-C 2022*, pp. 15–19, 2022, doi: 10.1109/ICSA-C54293.2022.00010.

[14] M. Saadatmand, “Towards Automating Integration Testing of .NET Applications Using Roslyn”, en *2017 IEEE International Conference on Software Quality, Reliability and Security Companion (QRS-C)*, IEEE, jul. 2017, pp. 573–574. doi: 10.1109/QRS-C.2017.99.

[15] O. Zanevych, “ADVANCING WEB DEVELOPMENT: A COMPARATIVE ANALYSIS OF MODERN FRAMEWORKS FOR REST AND GRAPHQL BACK-END SERVICES”, *Grail of Science*, núm. 37, pp. 216–228, mar. 2024, doi: 10.36074/GRAIL-OF-SCIENCE.15.03.2024.031.

[16] R. A. Schmidt y M. Thiry, “Microservices identification strategies : A review focused on Model-Driven Engineering and Domain Driven Design approaches”, *Iberian Conference on Information Systems and Technologies, CISTI*, vol. 2020-June, jun. 2020, doi: 10.23919/CISTI49556.2020.9141150.

[17] E. N. Preeth, J. P. Mulerickal, B. Paul, y Y. Sastri, “Evaluation of Docker containers based on hardware utilization”, *2015 International Conference on Control, Communication and Computing India, ICCC 2015*, pp. 697–700, mar. 2016, doi: 10.1109/ICCC.2015.7432984.

[18] S. A. Mohammed y A. L. Ralescu, “Future Internet Architectures on an Emerging Scale—A Systematic Review”, el 1 de mayo de 2023, *MDPI*. doi: 10.3390/fi15050166.

[19] B. Gupta, P. Mittal, y T. Mufti, “A Review on Amazon Web Service (AWS), Microsoft Azure & Google Cloud Platform (GCP) Services”, European Alliance for Innovation n.o., mar. 2021. doi: 10.4108/eai.27-2-2020.2303255.

5 ANEXOS

5.1 ANEXO I. Código Fuente del Proyecto de Agendamiento de Citas Médicas en .NET

Enlace Código Fuente del Proyecto:

[POC Agendamiento Citas Medicas]

Enlace Archivos Dockerfile para los Microservicios:

[Dockers Microservicios]

Configuración Docker Compose para el Monolito:

docker-compose.yml

```yaml
version: "3.8"
services:
  dbp-agendamiento-backend:
    image: ${DOCKER_REGISTRY-}dbpagendamientobackend
    build:
      context: .
      dockerfile: dbp-agendamiento-backend/Dockerfile
    ports:
      - "5000:8080" # HTTP
      - "5001:8081" # HTTPS
    environment:
      - AzureAd__Instance=https://login.microsoftonline.com/
      - AzureAd__TenantId=682a4e6a-a77f-4958-a3ac-9e266d18aa37
      - AzureAd__ClientId=049b9e6f-05e7-4d9c-a0d4-7b12a39634a5
      - JwtSettings__Key=TOKEN_SECRET_EXAMPLE_FOR_DBP_ATENCION_APP
      - JwtSettings__Issuer=epn.edu.ec
      - JwtSettings__Audience=dbp.atencion.epn.edu.ec
      - ConnectionStrings__Postgres2DBContext=Host=####:####;Database=###;Username=####;Password=#####
      - ASPNETCORE_ENVIRONMENT=Development
    networks:
      - external-network

networks:
  external-network:
    driver: bridge
```

Código del Middleware para Autenticación JWT:

```
public class HmacAuthenticationMiddleware
{
    private readonly RequestDelegate _next;

    public HmacAuthenticationMiddleware(RequestDelegate next)
    {
        _next = next;
    }

    public async Task Invoke(HttpContext context)
    {
        // Verificar si la solicitud contiene la firma en las cabeceras
        if (!context.Request.Headers.TryGetValue("X-Signature", out var receivedSignature))
        {
            context.Response.StatusCode = 400; // Bad Request
            await context.Response.WriteAsync("Falta Firma HMAC.");
            return;
        }

        // Obtener el JWT Token del encabezado Authorization
        if (!context.Request.Headers.TryGetValue("Authorization", out var authorizationHeader))
        {
            context.Response.StatusCode = 400; // Bad Request
            await context.Response.WriteAsync("Falta autorización de cabezera.");
            return;
        }

        var token = authorizationHeader.ToString().Replace("Bearer ", "");

        // Generar el HMAC a partir del token y compararlo con el que viene en la cabecera
        var secretKey = "b584b4065c478ef5aec5d9fca11cf4aa2ee8c65d434cdc118f8b49e69efc25db";
        var computedSignature = GenerateHMAC(token, secretKey);

        if (!computedSignature.Equals(receivedSignature))
        {
            context.Response.StatusCode = 401; // No Autorizado
            await context.Response.WriteAsync("Firma HMAC invalida.");
            return;
        }

        // Si todo está bien, continúa con la siguiente acción
        await _next(context);
    }
```

Configuración de perfiles launchSettings.json para Microservicios:

```
{
  "profiles": {
    "CatalogoMicroService": {
      "commandName": "Project",
      "launchBrowser": true,
      "launchUrl": "swagger",
      "environmentVariables": {
        "ASPNETCORE_ENVIRONMENT": "Development"
      },
      "dotnetRunMessages": true,
      "applicationUrl": "http://localhost:5101"
    },
    "CitaMicroService": {
      "commandName": "Project",
      "launchBrowser": true,
      "launchUrl": "swagger",
      "environmentVariables": {
        "ASPNETCORE_ENVIRONMENT": "Development"
      },
      "dotnetRunMessages": true,
      "applicationUrl": "http://localhost:5102"
    },
    "PersonalMicroService": {
      "commandName": "Project",
      "launchBrowser": true,
      "launchUrl": "swagger",
      "environmentVariables": {
        "ASPNETCORE_ENVIRONMENT": "Development"
      },
      "dotnetRunMessages": true,
      "applicationUrl": "http://localhost:5103"
    },
    "PacienteMicroService": {
      "commandName": "Project",
      "launchBrowser": true,
      "launchUrl": "swagger",
      "environmentVariables": {
        "ASPNETCORE_ENVIRONMENT": "Development"
      },
      "dotnetRunMessages": true,
      "applicationUrl": "http://localhost:5104"
    },
    "Container (Dockerfile)": {
      "commandName": "Docker",
      "launchBrowser": true,
      "launchUrl": "{Scheme}://{ServiceHost}:{ServicePort}/swagger",
      "environmentVariables": {
        "ASPNETCORE_HTTPS_PORTS": "8081",
        "ASPNETCORE_HTTP_PORTS": "8080"
      },
```

Configuración del API Gateway con Ocelot:

ApiGateway > {} ocelot.json > ...

```
2     "ReRoutes": [
3       {
5         "DownstreamScheme": "http",
6         "DownstreamHostAndPorts": [
7           {
8             "Host": "localhost",
9             "Port": 5101
10          }
11        ],
12        "UpstreamPathTemplate": "/catalogo/{everything}",
13        "UpstreamHttpMethod": [ "Get", "Post", "Put", "Delete" ]
14      },
15      {
16        "DownstreamPathTemplate": "/api/cita/{everything}",
17        "DownstreamScheme": "http",
18        "DownstreamHostAndPorts": [
19          {
20            "Host": "localhost",
21            "Port": 5102
22          }
23        ],
24        "UpstreamPathTemplate": "/cita/{everything}",
25        "UpstreamHttpMethod": [ "Get", "Post", "Put", "Delete" ]
26      },
27      {
28        "DownstreamPathTemplate": "/api/personal/{everything}",
29        "DownstreamScheme": "http",
30        "DownstreamHostAndPorts": [
31          {
32            "Host": "localhost",
33            "Port": 5103
34          }
35        ],
36        "UpstreamPathTemplate": "/personal/{everything}",
37        "UpstreamHttpMethod": [ "Get", "Post", "Put", "Delete" ]
38      },
39      {
40        "DownstreamPathTemplate": "/api/paciente/{everything}",
41        "DownstreamScheme": "http",
42        "DownstreamHostAndPorts": [
43          {
44            "Host": "localhost",
45            "Port": 5104
46          }
47        ],
48        "UpstreamPathTemplate": "/paciente/{everything}",
49        "UpstreamHttpMethod": [ "Get", "Post", "Put", "Delete" ]
50      }
51    ],
52    "GlobalConfiguration": {
53      "BaseUrl": "http://localhost:5000"
```

Archivo docker-compose.yml para Microservicios1:

```yaml
version: '3.4'

services:
  apigateway:
    image: ${DOCKER_REGISTRY-}apigateway
    build:
      context: .
      dockerfile: ApiGateway/Dockerfile
    ports:
      - "5000:8080"
      - "5001:8081"
    networks:
      - apigateway-network
    environment:
      - AzureAd__Instance=https://login.microsoftonline.com/
      - AzureAd__TenantId=682a4e6a-a77f-4958-a3ac-9e266d18aa37
      - AzureAd__ClientId=049b9e6f-05e7-4d9c-a0d4-7b12a39634a5
      - JwtSettings__Key=TOKEN_SECRET_EXAMPLE_FOR_DBP_ATENCION_APP
      - JwtSettings__Issuer=epn.edu.ec
      - JwtSettings__Audience=dbp.atencion.epn.edu.ec
      - ConnectionStrings__Postgres2DBContext=Host=######;Database=######;Username=#######;Password=#####
      - ASPNETCORE_ENVIRONMENT=Development
      - ASPNETCORE_URLS=http://+:8080
    depends_on:
      - catalogomicroservice
      - citamicroservice
      - personalmicroservice
      - pacientemicroservice

  catalogomicroservice:
    image: ${DOCKER_REGISTRY-}catalogomicroservice
    build:
      context: .
      dockerfile: CatalogoMicroService/Dockerfile
    ports:
      - "5101:80"
    networks:
      - apigateway-network
    environment:
      - ASPNETCORE_ENVIRONMENT=Development
      - ASPNETCORE_URLS=http://+:80
```

Archivo docker-compose.yml para Microservicios 2:

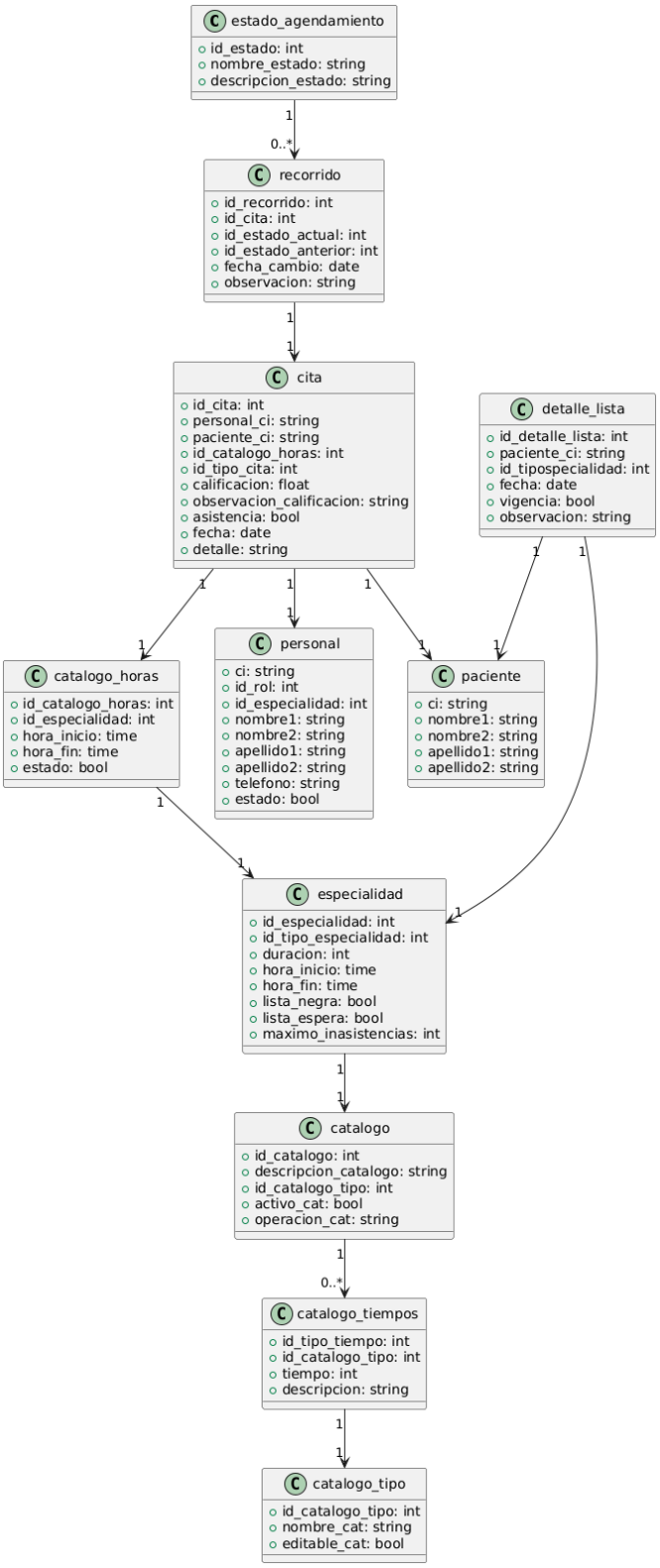
```yaml
  citamicroservice:
    image: ${DOCKER_REGISTRY-}citamicroservice
    build:
      context: .
      dockerfile: CitaMicroService/Dockerfile
    ports:
      - "5102:80"
    networks:
      - apigateway-network
    environment:
      - ASPNETCORE_ENVIRONMENT=Development
      - ASPNETCORE_URLS=http://+:80

  pacientemicroservice:
    image: ${DOCKER_REGISTRY-}pacientemicroservice
    build:
      context: .
      dockerfile: PacienteMicroService/Dockerfile
    ports:
      - "5104:80"
    networks:
      - apigateway-network
    environment:
      - ASPNETCORE_ENVIRONMENT=Development
      - ASPNETCORE_URLS=http://+:80

  personalmicroservice:
    image: ${DOCKER_REGISTRY-}personalmicroservice
    build:
      context: .
      dockerfile: PersonalMicroService/Dockerfile
    ports:
      - "5103:80"
    networks:
      - apigateway-network
    environment:
      - ASPNETCORE_ENVIRONMENT=Development
      - ASPNETCORE_URLS=http://+:80

networks:
  apigateway-network:
    driver: bridge
```

5.2 ANEXO II. Diagrama de Clases del Proyecto de Agendamiento de Citas Médicas



5.3 ANEXO III. Estándares de Desarrollo del Proyecto

Enlace al documento de estándares de desarrollo de software:

EPN-DGIP-EstandaresDesarrolloSoftware.docx

5.4 ANEXO IV. Metodología de Desarrollo Utilizada

Enlace documento de gestión de desarrollo de software:

POLÍTICAS Y GESTIÓN DE DESARROLLO DE SOFTWARE.docx

5.5 ANEXO V. Documentación caso de estudio

Enlace de documentación sobre el caso de estudio:

Informe General del Sistema Agendamiento DBP.docx

5.6 ANEXO VI. Colecciones de Pruebas de API en Postman

Enlace al lugar de trabajo de Postman:

TestMicroservicios.url

5.7 ANEXO VII. Reporte Generado con Swagger

Enlace al reporte de Swagger:

Reporte Swagger.pdf

5.8 ANEXO VIII. Resultados de Pruebas de API utilizando Newman

Enlace al reporte generado de los microservicios:

ReporteTestMicroservicios.url